

生成AI、機械学習の仕組みを学習する(超入門)

[[TOC]]

～ 第5回:ニューラルネットワーク ～

nab@garegga.net

2025年6月25日

§ 1. 学習理由、目標、進め方

学習理由など

学習理由

ニューラルネットワーク(以降NN)を学習する理由は「**現在のAI、機械学習において最もメジャーな手法のため**」です。

目標

ローレベルで実装する場面は少ないと思いますが、そこで現れる概念や用語は普通に使われるのである程度の**内部動作を含めての理解**を目指します

進め方

- 1) 最初にどのようなモデルかの説明をします。
- 2) 次に数字画像の分類というシナリオベースで実装します。**最初の実装は敢えて50～70年程前の技術を想定したもの**になります。

3) その後、色々な技法の紹介を通して精度や速度の

記述の方針

基本、解説→実装の流れで進めていきます

関連情報

ページの右上には関連情報へのリンクを載せています。アイコンの意味は次の通りです

-  : 書籍系
-  : オンラインの情報
-  : のうち、特に勉強になったもの
-  : 間接的な参考情報
-  : プログラミング関係

§ 2. ニューラルネットワークの概要

ニューラルネットワークのイメージと簡単な実装を紹介します

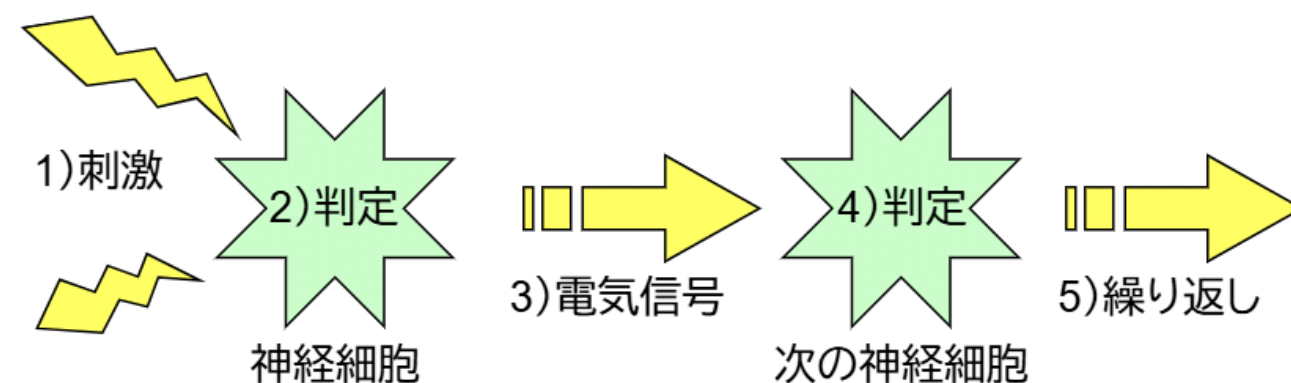
ニューラルネットワーク(NN)とは

ニューラルネットワークとは

ニューラルネットワークとは機械学習の手法の1つで、生物の神経メカニズムをモデルとしたものです

生物の神経メカニズム

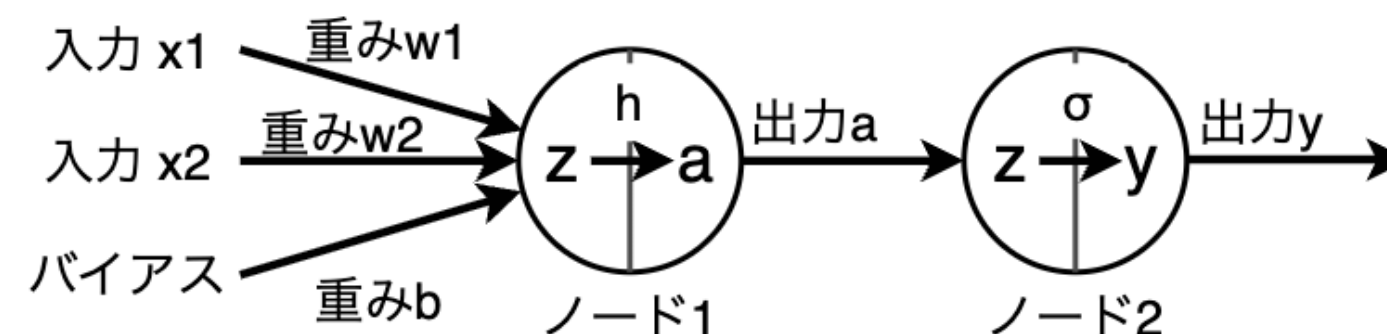
生物の神経は神経細胞への刺激が電気信号として他の神経細胞へ伝達する構造になっています



2), 4)の判定とは**一定の閾値を超えたら電気信号を発する**という動作を意味しています

数理モデル化

1)の刺激を入力データとし、神経細胞をノード(丸)、電気信号を一方向のエッジ(矢印)としてグラフ化します。ついでに名前をつけます(下図参照)



各記号とその意味は次の通りです

- ノード1、ノード2: 神経細胞のモデル化
- x_1, x_2 : 入力データ。神経細胞への刺激
- バイアス: 入力データの底上げ
- w_1, w_2, b : 重み係数。入力データにかけ算する

重み係数が学習によって更新されるパラメータです

- z : ノード内での途中出力。 $z = \sum_i w_i x_i$
- a, y : ノードからの出力。 $a = h(z), z = \sigma(z)$
- h, σ : 活性化関数。 z から出力値を決める

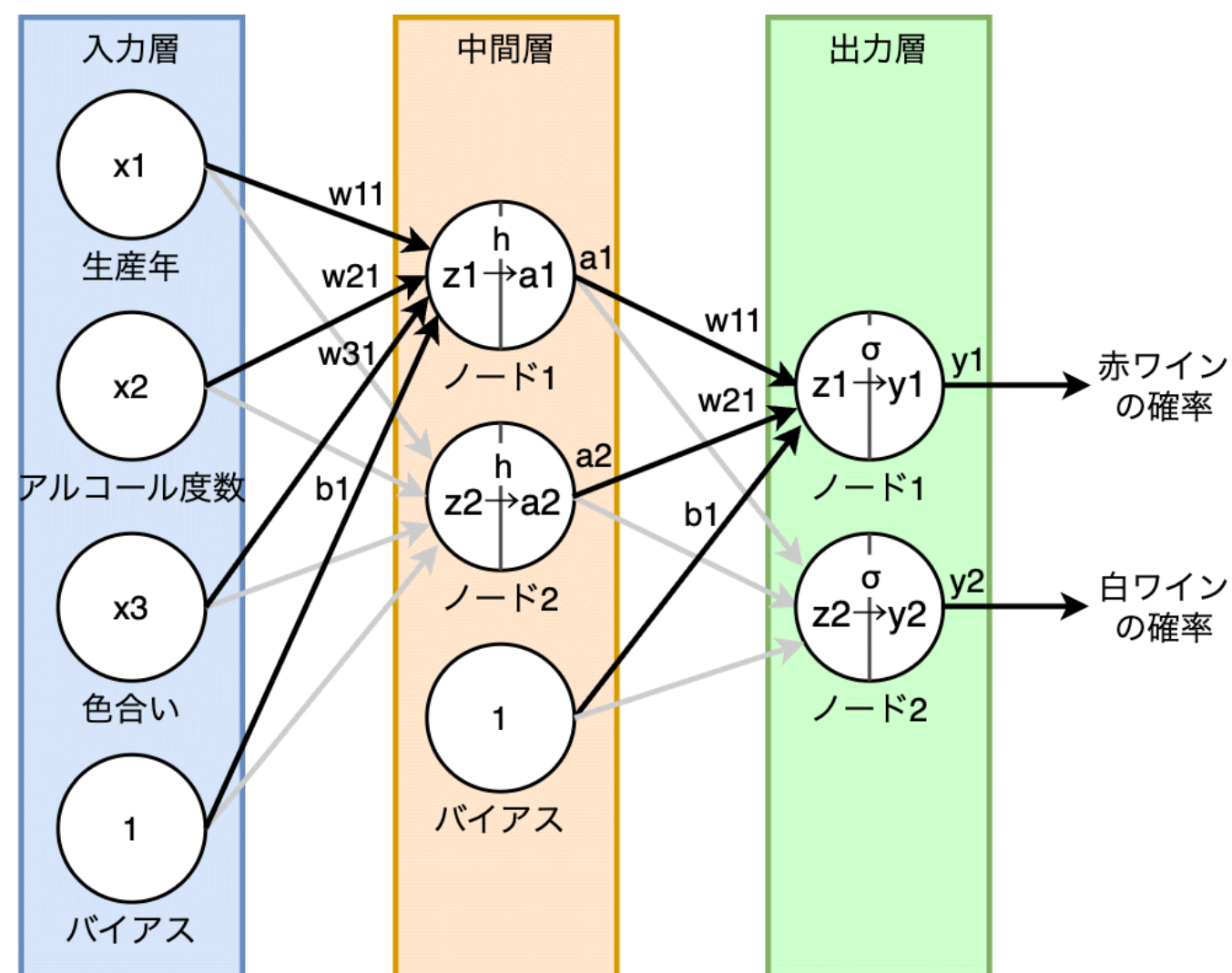
モデルの構成例

シナリオ

「あるワインが赤か白かをアルコール度数、生産年度、色合いから予測する」というシナリオを考えます

予測モデル(イメージ)

予測モデルの例として次図のような構成となります



全体構造とデータ方向

モデル全体としては左から右に一方向にデータが受け渡されていく構造です。この構造を**順伝播型ニューラルネットワーク**と呼びます。(英名: feed-forward neural network, FFNN)

入力層、隠れ層、出力層

ノード(円)の縦の並びを層と呼びます。一番左の層を**入力層**、一番右の層を**出力層**、その間の層を**隠れ層(中間層)**と呼びます

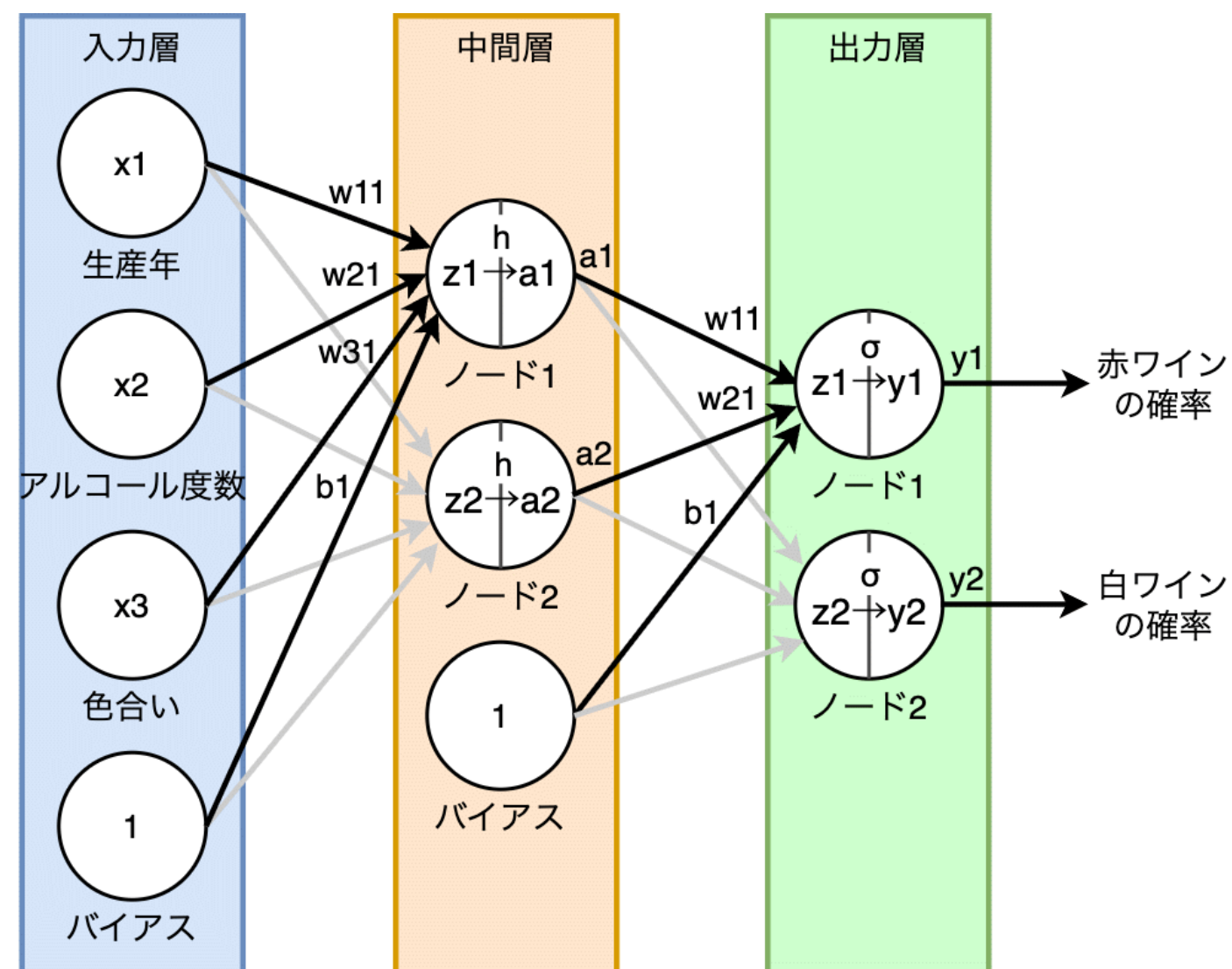
入力層は入力データです。左図の場合、ワインの生産年やアルコール度数などです

隠れ層は前の層からの出力を受け取り、何らかの処理をして次の層へ出力します。何らかの処理の部分是指定も制限もありません

出力層は前の層からの出力を受け取り、色々処理して

用語補足

モデル例(再掲)



n層ニューラルネットワーク

上図は2層ニューラルネットワークと呼びます
 入力層を含めません。ただし、見た目通り3層と呼ぶ場合もあるようです

全結合

ある層のノードから次の層のノードへすべてのペアが矢印で接続されているものを全結合と呼びます。左図のモデルは全結合です。

全結合における隠れ層は全結合層とも呼びます。
 Dense層や線形層とも呼ばれます

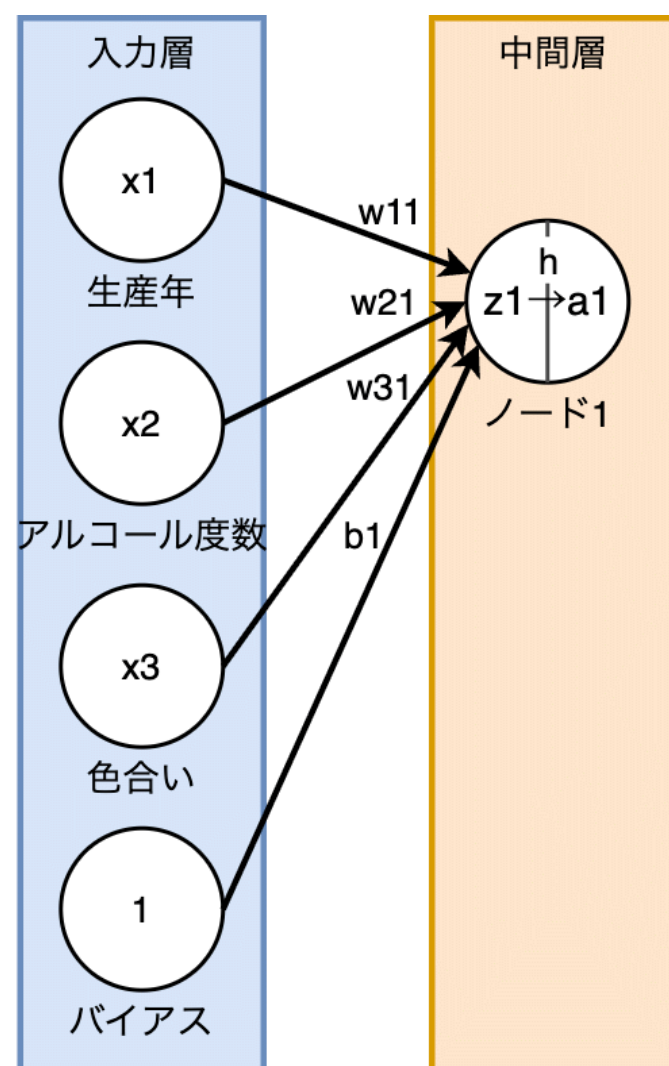
バイアス

要は平行移動。データの中心点(平均値)設定を担います

記号(ギリシャ文字)

- σ : シグマ(小文字)
- δ : デルタ(小文字)
- η : イータ(小文字)

入力層、隠れ層(前半)



入力層

入力層は予測に使用する入力データ x_i とバイアス(定数1)から構成されます。データは次の層にそのまま渡されます

隠れ層(前半)

前の層の出力(今回は x_i と1)を受け取ります。その際重み w_{i1} を掛けます

w_{ij} は前の層の i 番目のノードから次の層の j 番目のノードへの重みとなります。 b_j は次の層の j 番目のノードにそのまま加算される数値となります

ノードは受け取った数値を全て足し合わせます

$$z_1 = w_{11}x_1 + w_{21}x_2 + w_{31}x_3 + b_1 = \sum_i w_{i1}x_i + b_1$$

💡 z_1 はノード内の次の処理(h)に渡されます

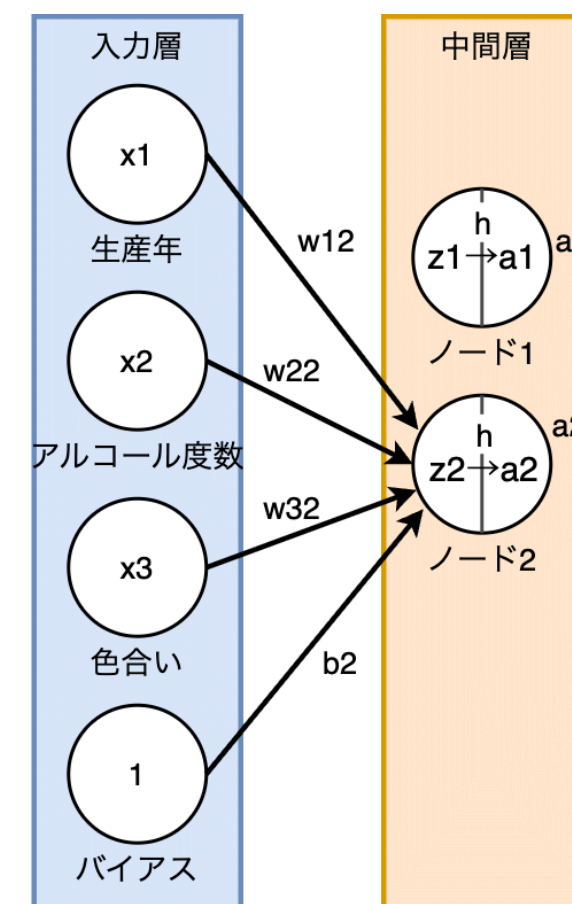
ベクトル化

$$z_1 = [x_1 \ x_2 \ x_3] \begin{bmatrix} w_{11} \\ w_{21} \\ w_{31} \end{bmatrix} + b_1$$

他ノードも同様

→

$$z_2 = [x_1 \ x_2 \ x_3] \begin{bmatrix} w_{12} \\ w_{22} \\ w_{32} \end{bmatrix} + b_2$$



行列化

$$z_1 = [x_1 \ x_2 \ x_3] \begin{bmatrix} w_{11} \\ w_{21} \\ w_{31} \end{bmatrix} + b_1 \quad z_2 = [x_1 \ x_2 \ x_3] \begin{bmatrix} w_{12} \\ w_{22} \\ w_{32} \end{bmatrix} + b_2$$

$$\iff [z_1 \ z_2] = [x_1 \ x_2 \ x_3] \begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \\ w_{31} & w_{32} \end{bmatrix} + [b_1 \ b_2]$$

入力層、隠れ層(前半、実装)

対象を整理(適当に記号化)

$$\begin{bmatrix} z_1 & z_2 \end{bmatrix} = \begin{bmatrix} x_1 & x_2 & x_3 \end{bmatrix} \begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \\ w_{31} & w_{32} \end{bmatrix} + \begin{bmatrix} b_1 & b_2 \end{bmatrix}$$

$\begin{bmatrix} z_1 & z_2 \end{bmatrix}$ → 出力値

$\begin{bmatrix} x_1 & x_2 & x_3 \end{bmatrix}$ → 入力値

$\begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \\ w_{31} & w_{32} \end{bmatrix}$ → 学習パラメータ。初期値は乱

$\begin{bmatrix} b_1 & b_2 \end{bmatrix}$ → 学習パラメータ。初期値0

実装(数式そのまま)

```
import numpy as np
W = np.random.randn(3, 2) # 重み行列
b = np.zeros([1, 2]) # バイアス項
x = np.array([1, 2, 3]) # 入力値(適当)
```

説明

コード全体としては行列を左記の通りに定義して、かけ算($X @ W$)と足算($+ b$)をしてるだけです

`np.random.randn(行数, 列数)` は乱数で行列生成します。`randn` は正規分布、`rand` だと一様分布です

`zeros([行数, 列数])` はゼロ行列を生成します

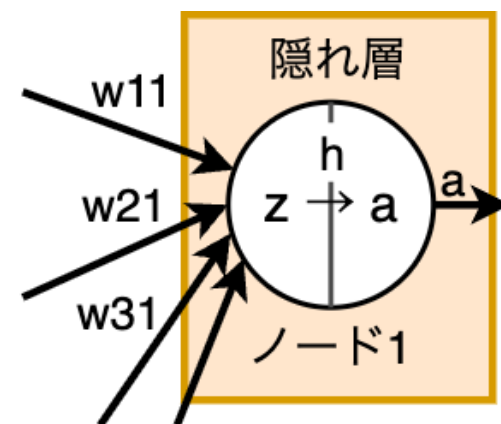
実装(あとで使う形に修正)

行数と列数を変数(`input_size` と `hidden_size`)にします。重み行列とバイアスは使いやすさの為、辞書 `p` にまとめます

```
import numpy as np
input_size, hidden_size = 3, 2 # 入力層の次元、隠れ層の次元
p = {}
p['W1'] = np.random.randn(input_size, hidden_size) # 重み行列
p['b1'] = np.zeros([1, hidden_size]) # バイアス項を追加
x = np.array([1, 2, 3]) # 入力値(適当)
z = x @ p['W1'] + p['b1'] # x:入力, z:ノード内出力
print(a)
```

隠れ層(後半)

隠れ層の前半と後半



隠れ層のノードは次のように動作します

1. 前の層からの出力を受け取る(→)
2. 重みとバイアスを掛けて足す(z_1)
3. 活性化関数を適用する($a_1 = h(z_1)$)
4. 出力結果(a_1)を次の層へ渡す

1、2は前ページの「前半」です。3、4が「後半」です

活性化関数(Activation Function)

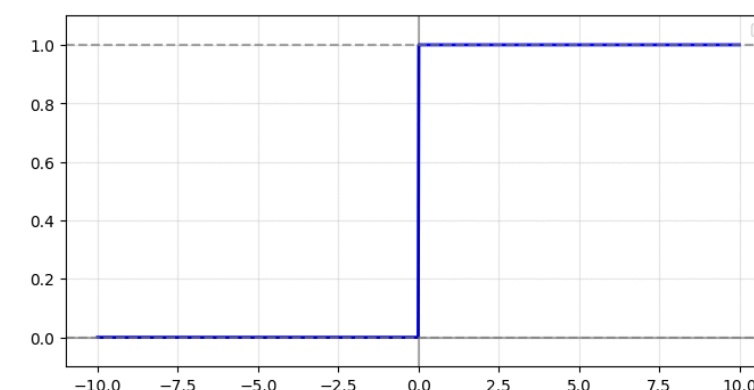
活性化関数とは隠れ層と出力層において、受け取った入力データに対する出力を決定する関数です

記号的にはノード内出力 z を引数として $a = h(z)$ という形の関数となります。次のような性質の関数が多用されています

- 入力の数値1つで出力も数値1つ
- 入力値が大きくなると出力値も大きくなる
- 特に0付近で大きく変化する

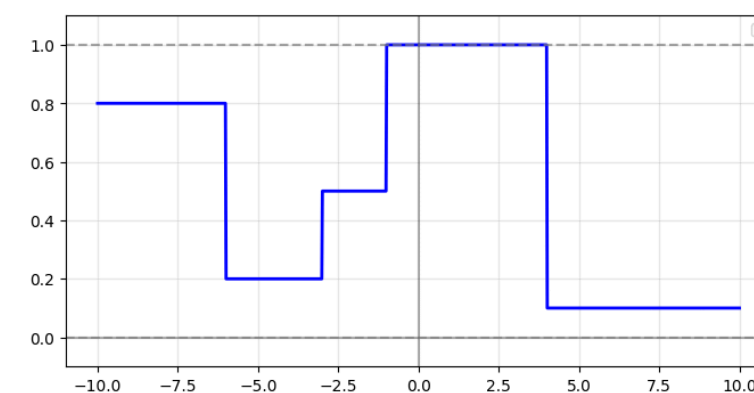
大小関係が変わらない

階段関数(Step Function)



活性化関数として使われる階段関数は左図のように0以上で1、0以下で0となる関数です

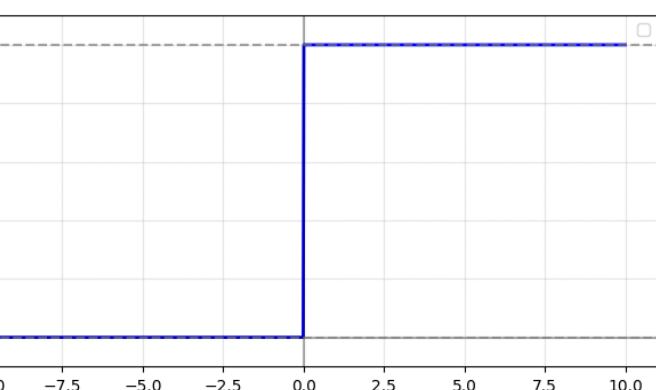
$$y = \begin{cases} 0 & , x \leq 0 \\ 1 & , x > 0 \end{cases}$$



一般に階段関数は左図の様な凸凹の関数です(潜在的にはあらゆる関数を表現する能力があります)

隠れ層(後半、実装)

階段関数(再掲)



$$y = \begin{cases} 0 & , x \leq 0 \\ x & , x > 0 \end{cases}$$

階段関数(実装)

```
step_function(x):
y = x > 0 # x>0ならTrue、そうでなければFalse
return y.astype(int) # Trueなら1、Falseなら0
```

は数値単体でも数値のリストでもOKです

隠れ層全体(実装)

隠れ層の前半と後半を合体すると次の通りです
(後半部分は最後の1行のみです)

```
import numpy as np
# ノード数を設定
input_size, hidden_size = 3, 2 # 入力層の次元、隠れ層の次元

# 重みとバイアスを初期化
p={}
p['W1'] = np.random.randn(input_size, hidden_size) # 重み
p['b1'] = np.zeros([1, hidden_size]) # バイアス項を追加

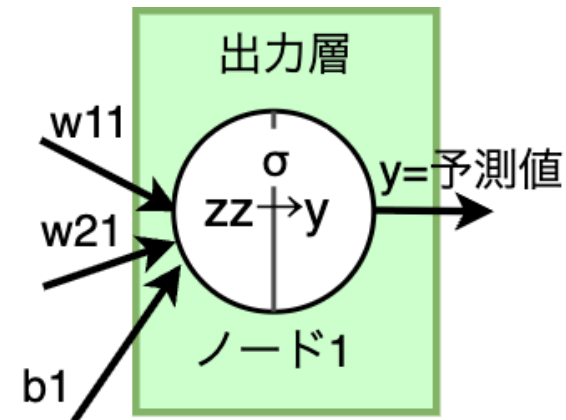
# 入力値を設定
x = np.array([1, 2, 3]) # 入力値 (適当)

# ノード内出力zを算出
z1 = x @ p['W1'] + p['b1'] # x:入力, z1:ノード内出力

# 活性化関数を適用
a1 = step_function(z1) # a1:ノード外への出力
```

出力層

出力層



動作は隠れ層と同じです

活性化関数が階段関数から
softmax関数に変更となります

出力層(前半、実装)

前半部分の実装は隠れ層と同じです
隠れ層の実装の続きとしては下記の通りです
(隠れ層の出力が `a1` です)

```
output_size = 2 # 出力層の次元
p['W2'] = np.random.randn(hidden_size, output_size)
p['b2'] = np.zeros([1, output_size])
z2 = a1 @ p['W2'] + p['b2'] # a1:入力, z2:ノード内出力
```

出力層の活性化関数

出力層の活性化関数は用途により様々です。今回は分類問題と仮定します

分類問題

分類問題とは、入力データに対してそれがどのクラスに属するかを判定する問題です。

例えば0～9の数字の手書き画像データを入力として、それがどの数字かを判定する様な問題です。

この例の場合、出力層の次元は10となり出力値は確率になるよう設計します

出力値を確率としてアウトプットする関数の1つとしてソフトマックス関数があり、今回これを使用します

softmax関数

softmax関数とは

値のセットを確率化する関数の1つです

具体的な操作(理論と実装)

値のセットを $X = [x_1, x_2, \dots, x_n]$ とします
各値を指数関数にかけます

$$X' = [e^{x_1}, e^{x_2}, \dots, e^{x_n}]$$

全ての要素を足し算します

$$S = e^{x_1} + e^{x_2} + \dots + e^{x_n}$$

全ての要素を合計値 S で割ります

$$Z = \left[\frac{e^{x_1}}{S}, \frac{e^{x_2}}{S}, \dots, \frac{e^{x_n}}{S} \right]$$

Z の各要素はそのまま確率となります

式通り実装したのが下記(実用上は問題あり)

```
def softmax_simple(x):  
    exp_x = np.exp(x) # 各要素を指数関数にかけ
```

問題点と解決策

e^x は x が少し大きくなるだけですぐにコンピュータで扱える数値の範囲を超えます。そのため、 e^x が小さい数値に収まるように調整する必要があります

具体的には $X' = [e^{x_1}, e^{x_2}, \dots, e^{x_n}]$ 最大値 e^M で各要素を割ります。
 $\frac{e^{x_i}}{e^M} = e^{x_i - M}$

つまり、事前に x_i から最大値 M を引いておきます

```
def softmax(x):  
    M = np.max(x) # 最大値Mを取得  
    exp_x = np.exp(x - M) # xからMを引いてから指数関数にかけ  
    S = np.sum(exp_x) # あとは同じ処理  
    return exp_x / S
```

FAQ

- 最大値で引き算して結果変わらない?
 - 確率(数値)は変わりますが大小関係は同じ
- そもそも確率なの?
 - 数値セットは足して割ったら確率扱い可能

ここまでのまとめ

理論ベース

x を W と b で線形変換します $z = Wx + b$

力 z を活性化関数 h にかけます $a = h(z)$

ると $y = h(Wx + b)$

$z = Wx + b$ の実装(縦横注意)

$x @ W + b$

活性化関数

ステップ関数

```
def step_function(x):
    y = x > 0 # x>0ならTrue、そうでなければFalse
    return y.astype(int) # Trueなら1、Falseなら0
```

softmax関数

```
def softmax(x):
    M = np.max(x) # 最大値Mを取得
    exp_x = np.exp(x - M) # xからMを引いてから指数関数にかける
```

順伝播全体

```
import numpy as np

input_size = 3 # 入力層の次元
hidden_size = 2 # 隠れ層の次元
output_size = 2 # 出力層の次元

# 重み行列とバイアスを初期化
p = {}
p['W1'] = np.random.randn(input_size, hidden_size)
p['b1'] = np.zeros([1, hidden_size])
p['W2'] = np.random.randn(hidden_size, output_size)
p['b2'] = np.zeros([1, output_size])

# 入力値 (適当)
x = np.array([1, 2, 3])

# ニューラルネットワーク
z = x @ p['W1'] + p['b1'] # x:入力, z:ノード内出力
a = step_function(z) # a:ノード外への出力
z = a @ p['W2'] + p['b2'] # a:入力, z:ノード内出力
y = softmax(z) # y:最終出力
```

y は確率出力です。 y 内の最大値のインデックスは a で取得します

ここまでは順伝播の簡単な実装例です
次ページからは具体的な問題(シナリオ)を用意し
その問題への順伝播の適用と
学習機能を解説、実装します

§ 3. 数字画像の分類

具体的な問題へのNNの適用を解説します。コードは02.py参照です

シナリオ

概要

0から9までの手書きの数字画像について、各画像がどの数字に対応するかを判定します
これが今回のシナリオです

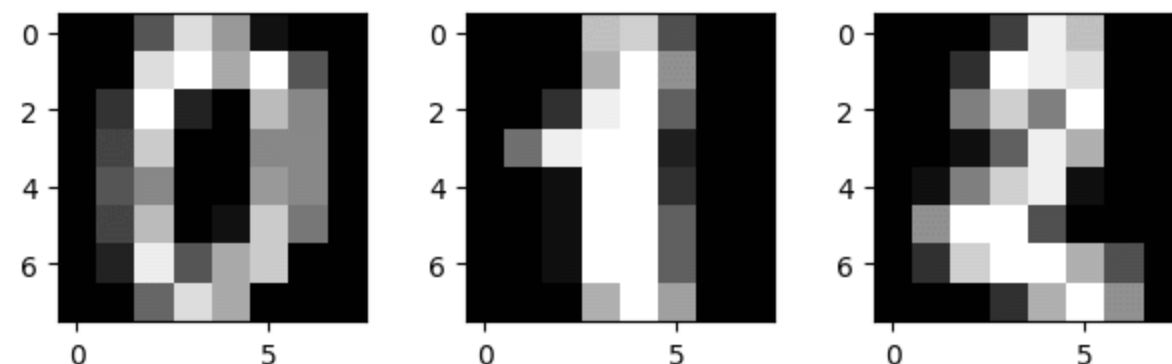
手法としてはニューラルネットワーク(以下NN)を用いどのように学習するのかなど実装含め説明します

使用する数字画像データ

この手のシナリオで定番なのは [MNIST](#) です

しかし今回はscikit-learnの [digits](#) を使用します

画像サンプルは下記のようになります



画像データの特徴

- 画像サイズ: 8x8 (MNISTは28x28)
- 値域: 0-16 (MNISTは0-255)
- サンプル数: 1797 (MNISTは計70000)

要はデータサイズが小さくて扱いやすいため
今回はdigitsを使います

ポイント

このシナリオを通じてのポイントは次の通りです

- 具体的な問題に対するNNの構成と実装
- 学習がどのように行われるのかの技術的理解
- どうやって精度を改善するかの手法をいくつか
(このため初期実装は敢えてレトロバージョンです)

✎ 前回の線形回帰は200年以上前の技術で更にレトロです

📌 最も重要なのは上記ポイントの発想の部分です

画像データの準備

コード

02-01-digits.py

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

""" digitsデータ読み込み """
digits = load_digits()
X_orig, Y_orig = digits.data, digits.target
X_train, X_test, Y_train, Y_test = train_test_split(
    X_orig, Y_orig, test_size=0.2, random_state=19)

def print_data():
    # <省略>
print_data()
```

画像データの読み込みと確認

`digits = load_digits()` で画像データを全て読み込みます。
`digits.data` は画像データ本体、`digits.target` は各画像の正解ラベルです

`print_data()` は読み込んだデータの大きさや画像表示の確認

データの分割

`train_test_split` はデータ全体を学習データと検証データに分割します(検証データの割合が20%)

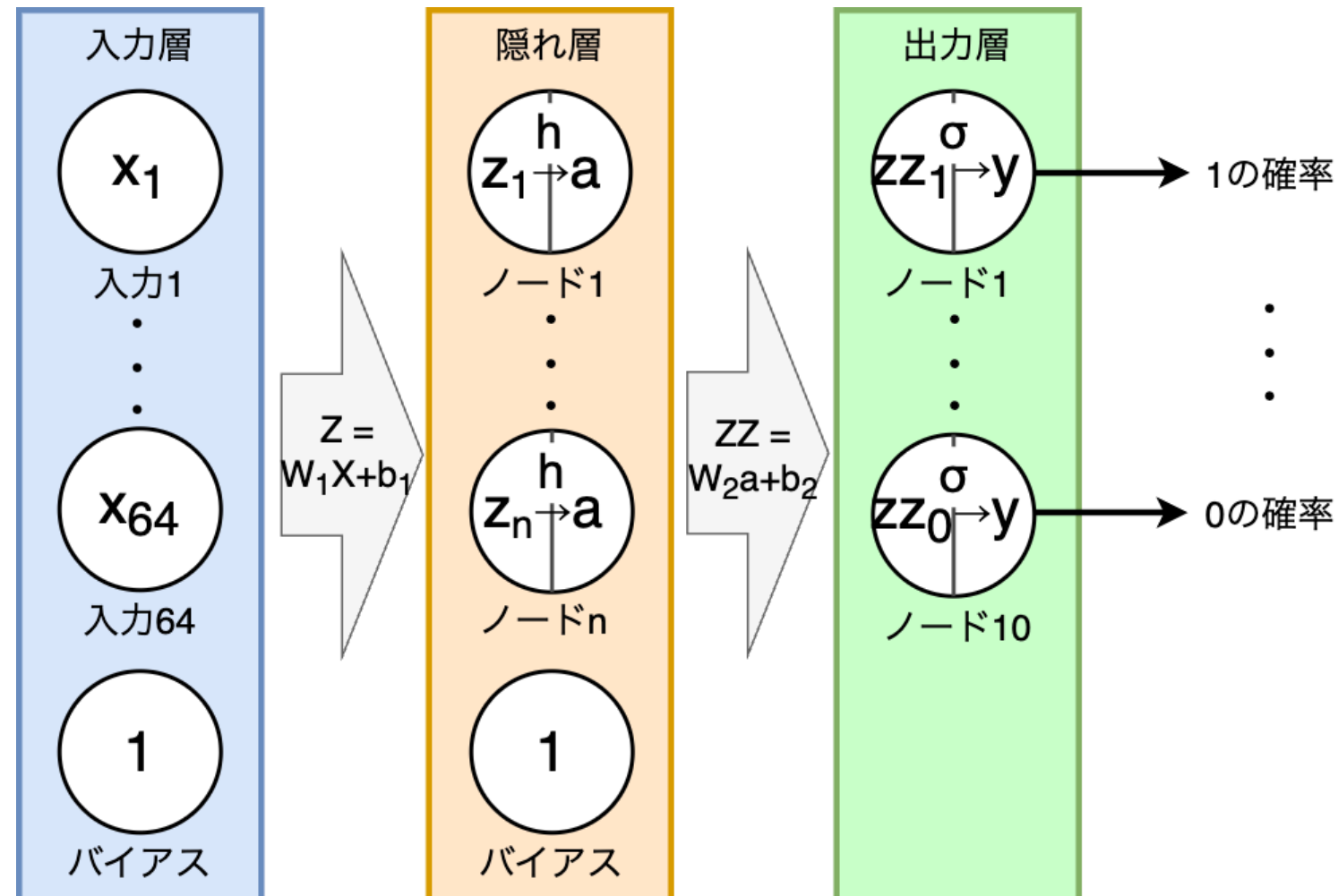
- `X_train` : 学習データ。画像データ
- `Y_train` : 学習データ。正解ラベル
- `X_test` : 検証データ。画像データ
- `Y_test` : 検証データ。正解ラベル

`X_train` を入力値として画像がどの数字なのかを予測し、`Y_train` と比較した結果からパラメータを更新して学習します

`X_test` と `Y_test` は上記の学習結果からどの程度数字の識別ができるようになったかを確認するために使います

ニューラルネットワークの構成

構成



情報整理

- 入力層のデータは $\text{bm}x = [x_1, x_2, \dots, x_{64}]$ (8x8の画像データを1次元化したもの)
- 入力層から隠れ層への重みは W_1 、バイアスは b_1
- 隠れ層
 - 隠れ層内の出力データは $z1 = W_1 \text{bm}x + b_1$
 - 隠れ層からの出力データは $a = h(z1)$ (h は階段関数)
- 出力層
 - 出力層内の出力データは $z2 = W_2 a + b_2$
 - 出力層からの出力データは $y = f(z2)$ (f はsoftmax関数)

上図のように2層NNとします。次数は次の通り

- 入力層は8x8=64次元 (画像データサイズ)
- 隠れ層はn=64次元 (適当に決めました)
- 出力層は10次元 (文字の種類は0-9の10種)

隠れ層の活性化関数はstep関数、
出力層の活性化関数はsoftmax関数です

✎ > 上記式、 $W_1 \text{bm}x$ などは縦ベクトルベースの表記です

実装は横ベクトルベースで縦横や順序が変わります
その辺は適当に読み替えてください

順伝播を実装

コード

先ページの整理結果から実装したものがこちらです

02-02-forward.py

```
import numpy as np

""" 初期設定 """
np.random.seed(19) # 乱数初期化
input_size = 8 * 8 # 入力画像のサイズ
hidden_size = 64 # 隠し層、サイズは適当
output_size = 10 # 分類したい個数

""" 重みの初期化 """
p={}
p['W1'] = np.random.randn(input_size, hidden_size)
p['b1'] = np.zeros([1, hidden_size])
p['W2'] = np.random.randn(hidden_size, output_size)
p['b2'] = np.zeros([1, output_size])

def forward(x):
    """ 順伝播 """
    z1 = x @ p['W1'] + p['b1'] # x:入力, z1:ノード内出力
    a = step_function(z1) # a:ノード外への出力
    z2 = a @ p['W2'] + p['b2'] # a:入力, z2:ノード内出力
    y = softmax(z2) # y:ノード外への出力
    return y # yが最終出力
```

解説

初期設定 は入力画像のサイズ、隠し層のサイズ、出力層のサイズを定義します

重みの初期化 は重み行列を定義しています。辞書形式なのは後の実装をしやすいするために、キーの **W1**、**b1**、**W2**、**b2** はそれぞれは隠れ層、出力層での重み行列、バイアスを表します

def forward(x): が順伝播の本体です。**x** は入力値、**z1** は隠れ層の内部出力、**a** は隠れ層からの次の層への出力、**z2** は出力層の内部出力、**y** は出力層の最終的な外部出力を表します

step_function, softmax は活性化関数で前節で実装した物をそのまま利用しています

順伝播を実行(とりあえず1件)

コード

`forward` を実行した結果がどうなるか確認

02-03-test.py

```
i = 0 # 何番目のテストデータを使うかを指定
Y_pred = forward(X_test[i]) # i番目の画像を判別
print(Y_pred.shape) # リストサイズ確認→列数が10個ならOK
print(Y_pred) # 具体的な中身を表示→数字が表示されたならOK

k = np.argmax(Y_pred) # 一番確率の高い数字を取得
print(k, Y_test[i]) # 正解と並べて表示→8 7
```

i 今までのコードは数字順に全て実行していること前提です

実行結果と解説

`print(Y_pred.shape)` の結果は `(1, 10)` です。これは「判定したデータ件数が1件」で「各データごとに10個の出力値がある」という意味です

`print(Y_pred)` の結果は `[[数字 数字 数字 ... 数字]]` で数字10個のリストで、リストの順番に対応する数字である確率を表します。例えば `[[0.3 0.1 ... ]]` だと0の確率が0.3、1

続き

`k = np.argmax(Y_pred)` は `Y_pred` の中で一番確率の高い数字を取得します。今回の結果は8

`Y_test[i]` は `i` 番目のテストデータの正解ラベルを表します。今回の結果は7

なぜ不正解なのか？

現時点では一切学習していないのでサイコロ状態です
如何に学習するかは次節で説明します

正解率

正解率(Accuracy)とは

正解率とは分類問題における指標の1つで、**与えられたデータについて正解件数を全件数で割ったもの**です。最大値は1(100%)、最小値は0(0%)。イメージ的には100点満点のテストで何点だった？です

実装

データ1件1件ごとにループしながら正解をカウントしています。最後にデータ件数 `len(Y)` で割っています

02-04-accuracy.py

```
def accuracy(Y_pred, Y):  
    """ 正解率 """  
    cnt = 0 # 正解数  
    for i in range(len(Y_pred)):  
        k = np.argmax(Y_pred[i]) # 最も確率が高いindexを取得  
        if Y[i] == k: # i番目の正解Y[i]と予測値kを比較  
            cnt += 1 # 正解ならカウント+1  
  
    return cnt / len(Y)  
  
Y_pred = forward(X_train) # X_trainから予測値取得  
print(accuracy(Y_pred, Y_train))
```

実行結果

実行結果は `0.0953 ...` で約9.5%です。まだ学習していないので乱数で(サイコロ振って当てるみたいな感じで) 10%前後に落ち着いた状態です

i 次ページからどうやって正解率をあげるのかを模索します

§ 4. 学習(=パラメータの更新)

一般的な語句としての学習とは「いかに正解率をあげるか」ですが
機械学習的な本質は「どうやって(何を根拠に)パラメータを更新するか」です

正解率をあげるには

現状まとめ

`forward()` を実装したので画像識別自体は実行できる。でも、正解率(識別率)は10%でサイコロ(乱数)状態なので**正解率を上げたい**

正解率をあげる方法を検討

- $WX + b$ で W と b は初期値が乱数なので、正しく判定できるような値に更新したい
- まず現在の**判定精度を測る指標(数値指標)**が必要
- その指標を最適化するようにパラメータを更新する
→ **勾配降下法**

勾配降下法とは

数値指標の関数 L を最小化(最適化)するため、ある t 時点でのパラメータ $W^{(t)}$ を用いて次式でパラメータ W を更新する手法です

$$\bm{W}^{(t+1)} := \bm{W}^{(t)} - \eta \nabla L(\bm{W}^{(t)})$$

損失関数(＝精度の数値指標)

数値指標を出力する関数を損失関数と呼びます。

正解率は使えない

損失関数として先に取り上げた正解率が思い浮かぶかもしれませんが、正解は1、不正解は0の2択が基本のため、指標としては粗く学習には不適です

MSE(平均二乗誤差)

ということで、まずはMSEを使用します

正解が i の時、 i に対する確率が $p(i)$ とすると $(1 - p(i))^2$ が評価値となり、 $p(i)$ が大きいほど損失関数は低くなります($\because (1 - p(i)) \rightarrow 0$)

逆に i 以外に対する確率を $p(j)$ とすると $(p(j))^2$ が評価値で $p(j)$ が小さいほど損失関数は低くなります

上記のように確率がベースなので正解不正解の2択/101

損失関数(MSE)

MSE(平均二乗誤差)

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$$

- N : サンプル数
- $\hat{y}_i$: i 番目のサンプルに対する予測値
- y_i : i 番目の正解ラベル

上記定義は数値予測に対するものとなります

今回のような分類問題では次のようになります

分類問題へのMSEの適用

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N \sum_{j=1}^K \left(\hat{y}_i^{(j)} - y_i^{(j)} \right)^2$$

数式が異なることの補足

MSEの定義はスカラーの差の二乗和(の平均)です。分類問題では予測値がベクトルなので各成分を網羅するよう二乗和を計算しています

実装

02-05-MSE.py

```
def one_hot_vec(k):
    """ k番目の成分だけ1のワンホットベクトル """
    v = np.zeros(10, dtype=int) # 全成分がゼロのベクトルを用意
    v[k] = 1 # k番目の成分だけ1にする
    return v

def one_hot_batch(k_list):
    """ n個のリストから(n,10)サイズのone_hot_vecリストを作る """
    return np.array([one_hot_vec(k) for k in k_list])

def loss_mse(Y_pred, Y):
    """ 平均二乗誤差 """
    Y_vec = one_hot_batch(Y) # 正解をワンホットベクトル化
    rss = np.sum((Y_pred - Y_vec)**2, axis=1) # 列方向に二乗和
    return np.mean(rss)

def loss_func(X, Y):
    """ 損失関数(コスト関数) """
```

損失関数を最小化したい

現状まとめ

`forward()` の精度の指標としてMSEを実装。次はこのMSEを最小化するようパラメータを更新したい

パラメータの更新＝最適化

今回、最適化手法として勾配降下法を用います

勾配降下法(理論)

パラメータを $\mathbf{W}$ 、指標となる関数を L とすると
 $\mathbf{W}^{(t+1)} := \mathbf{W}^{(t)} - \eta \nabla L(\mathbf{W}^{(t)})$ (η は学習率)

意味は「現在値から $\eta \times$ 微分値を引き算する」です
 パラメータ1つ (w_{ij}) だけに着目して書き換えます

$$w_{ij}^{(t+1)} := w_{ij}^{(t)} - \eta \frac{\partial L}{\partial w_{ij}^{(t)}}$$

L の微分をどうするか

今回、損失関数としてMSEを用いるので

$$L = \text{MSE} = \frac{1}{N} \sum_{i=1}^N \sum_{j=1}^K \left(\hat{y}_i^{(j)} - y_i^{(j)} \right)^2$$

つまり、 L は $\hat{y}_i^{(j)}$ を変数にとっています。しかし、求めたいのは $\frac{\partial L}{\partial w_{ij}}$ です。微分の定義に戻ってみます

$$\frac{\partial L}{\partial w_{ij}} = \lim_{h \rightarrow 0} \frac{L(w_{ij} + h) - L(w_{ij} - h)}{2h}$$

理論的にはこの極限を考える必要がありますが、実装視点では十分小さな h を設定して近似値が算出できればOKです(求めたいのは理論式でなく数値)

$$\frac{\partial L}{\partial w_{ij}} \approx \frac{L(w_{ij} + h) - L(w_{ij} - h)}{2h}$$

この手法を数値微分と呼びます。次ページで実装します

数値微分(実装、単パラメータ)

今やりたいこと

$$\frac{\partial L}{\partial w_{ij}} \approx \frac{L(w_{ij} + h) - L(w_{ij} - h)}{2h} \text{ を実装したい}$$

パラメータ1つに注目して検討

微分変数を1つに固定 & その現在値を保存

```
= 'W1' # 微分対象のパラメータを絞る (今回はW1のどれか)
i, j = 2, 2 # W1の(2,2)成分に着目する (2,2にしたのは適当)
w_ij = p[W][i][j] # ij成分を保存 (この後の計算のため)
```

分子に表れる h を設定 (値は適当)

```
h = 1e-5 # 微小値hを設定
```

$L(w_{ij} + h)$ を計算する

```
p[W][i][j] = w_ij + h # 先に微分対象のパラメータを+hしとく
L1 = loss_func(X_train, Y_train) # その状態でLを計算
```

$L(w_{ij} - h)$ も同様に計算

```
p[W][i][j] = w_ij - h # 先に微分対象のパラメータを-hしとく
```

続き

$$\frac{L(w_{ij} + h) - L(w_{ij} - h)}{2h} \text{ を計算}$$

```
grad = (L1 - L2) / (2 * h) # ij成分での微分値を算出
```

パラメータ単体の数値微分のまとめ

02-06-grad.py

```
W = 'W1' # 微分対象のパラメータを絞る (今回はW1のどれか)
i, j = 2, 2 # W1の(2,2)成分に着目する (2,2にしたのは適当)
w_ij = p[W][i][j] # ij成分を保存 (この後の計算のため)
h = 1e-5 # 微小値hを設定 (ちょい粗めの設定)
```

```
# L(W+h)の計算
```

```
p[W][i][j] = w_ij + h # 先に微分対象のパラメータを+hしとく
L1 = loss_func(X_train, Y_train) # その状態でLを計算
```

```
# L(W-h)の計算
```

```
p[W][i][j] = w_ij - h # 先に微分対象のパラメータを-hしとく
L2 = loss_func(X_train, Y_train) # その状態でLを計算
```

```
# ij成分での微分値を算出
```

```
grad = (L1 - L2) / (2 * h)
```

```
# ij成分を元に戻す (今回は微分値の算出のみ、更新はしない)
```

数値微分(実装、重み行列単体)

現状

パラメータの w_{ij} に対する $\frac{\partial L}{\partial w_{ij}}$ は実装できた
重み行列1つ分に拡張したい

実装

02-07-grad_numeric.py

```
def grad_numeric(L, W):
    """ 微分 (L:微分対象の関数、W:微分変数) """
    h = 1e-1
    grad = np.zeros_like(p[W]) # 微分結果の格納先
    for i in range(p[W].shape[0]): # 行でループ
        for j in range(p[W].shape[1]): # 列でループ
            # 重みのij成分を保存しておく
            W_ij = p[W][i][j]
            # f(W+h)の計算
            p[W][i][j] = W_ij + h # W+hを算出
            L1 = L() # f(W+h)を算出 (L=f)
            # f(W-h)の計算
            p[W][i][j] = W_ij - h # W-hを算出
            L2 = L() # f(W-h)を算出
            # ij成分での微分値を算出 (微分の定義そのまま)
            grad[i][j] = (L1 - L2) / (2 * h)
```

解説

全体としては、前ページでの単パラメータの処理を行 `i` と `j` でループすることで重み行列1つ分を網羅するようにしている

計算結果は `grad` 変数に格納

先の損失関数は `loss_func(X_train, Y_train)` で固めたが、関数化にあたって引数 `L` で動的に変更できるように拡張

呼び出し方の例

02-07-grad_numeric.py

```
L = lambda: loss_func(X_train, Y_train) # 微分対象
grads = {}
grads['W1'] = grad_weight(L, 'W1')
print(grads['W1'].shape, np.sum(grads['W1']))
# (64, 27) 0.3258398751365621
```

`L = lambda: ~` とすることで `grad_weight` 内で呼び出す

数値微分(実装、全重み行列)

現状

重み行列単体に対する $\frac{\partial L}{\partial w_{ij}}$ は実装できた
重み行列を網羅して全パラメータに広げたい

実装

02-08-grad_weight_all.py

```
def grad_weight_all(X, Y):
    L = lambda: loss_func(X, Y) # 微分対象
    grads = {}
    grads['W1'] = grad_numeric(L, 'W1')
    grads['b1'] = grad_numeric(L, 'b1')
    grads['W2'] = grad_numeric(L, 'W2')
    grads['b2'] = grad_numeric(L, 'b2')
    return grads

# 呼び出し例
grads = grad_weight_all(X_train, Y_train)
print(grads['W2'].shape, np.sum(grads['W2']))
# (27, 10) -0.0003173014280410946
```

解説

`grad_numeric` で重み行列単体の微分ができるので、それを全重み行列に対して呼び出すように列挙している(`W1`, `b1`, `W2`, `b2`)
損失関数(微分対象)はいずれも同じ `L`

呼び出し例 では実行後に `grads` の行列サイズと微分値の合計がゼロでないことを確認

今何をやってるか(リマインド)

損失関数 `L` を最小化するため、下記の勾配降下法を実装したい

$$\bm{W}^{(t+1)} := \bm{W}^{(t)} - \eta \nabla L(\bm{W}^{(t)}) \quad (\eta \text{ は学習率})$$

そのため下記微分を求めたい(これが今完了)

$$\frac{\partial L}{\partial w_{ij}} \approx \frac{L(w_{ij} + h) - L(w_{ij} - h)}{2h}$$

勾配降下法(実装)

勾配降下法

ターゲットの勾配降下法は次式の通りです

$$\mathbf{W}^{(t+1)} := \mathbf{W}^{(t)} - \eta \nabla L(\mathbf{W}^{(t)})$$

$\nabla L(\mathbf{W}^{(t)})$ は `grads = grad_weight_all(X, Y)` で実装したので、上記の更新式は次のようにコーディングできます(η は `lr` に置き換えています)

```
for key in grads.keys(): # W1, b1, W2, b2を順にループ
    p[key] = p[key] - lr * grads[key] # パラメータ更新
```

学習部分のコード全体

02-09-train.py

```
def train(X, Y):
    print_metrics(X, Y) # 初期状態表示
    for i in range(max_epoch): # max_epoch = 400
        print(f'loop: {i+1} → ', end='')
        # 微分値を取得
        grads = grad_weight_all(X, Y)
        # パラメータ更新
        for key in grads.keys():
            p[key] = p[key] - lr * grads[key]
```

解説

`print_metrics` は損失値と正解率を表示する関数(次ページで説明します)

```
for i in range(max_epoch): #
max_epoch = 400
```

より400回学習を繰り返します。**学習用データ1セット分の学習をエポック**と言います

`grads = grad_weight_all(X, Y)` は微分値の取得(前ページまでで説明済み)

```
p[key] = p[key] - lr *
grads[key]
```

はパラメータ更新(勾配降下法)。`lr` は後で定義

学習状況表示、グラフ表示

学習状況表示

`metrics` で学習状況を `print` します。実行タイミングは学習中の毎エポックの直後です

print_metrics.py

```
def print_metrics(x, y):  
    # 現在値を取得  
    loss = loss_func(x, y) # 損失値lossを取得  
    acc = accuracy(forward(x), y) # 正解率accuracyを取得  
    now = time.perf_counter() # 現在時刻を取得
```

グラフ表示用に値を保存

```
loss_list.append(loss)  
acc_list.append(acc)  
time_list.append(now - start) # 開始からの経過時間
```

現在値表示

```
print(f'Loss: {loss:.10f}, Accuracy: {acc:.5f}, ' \n  
      f'Time: {(now-start)/60:.5f}[min]')  
return loss, acc, now # 一応戻り値にしておく
```

グラフ表示

全ループを回した後に `graph` でグラフ表示します

02-11-graph.py

```
def graph(~):  
    省略
```

グラフ表示のコードは本題ではないので省略します

実行コード

ここまで準備した学習コードと状況表示コードを合わせて、実行するのが次のコードとなります

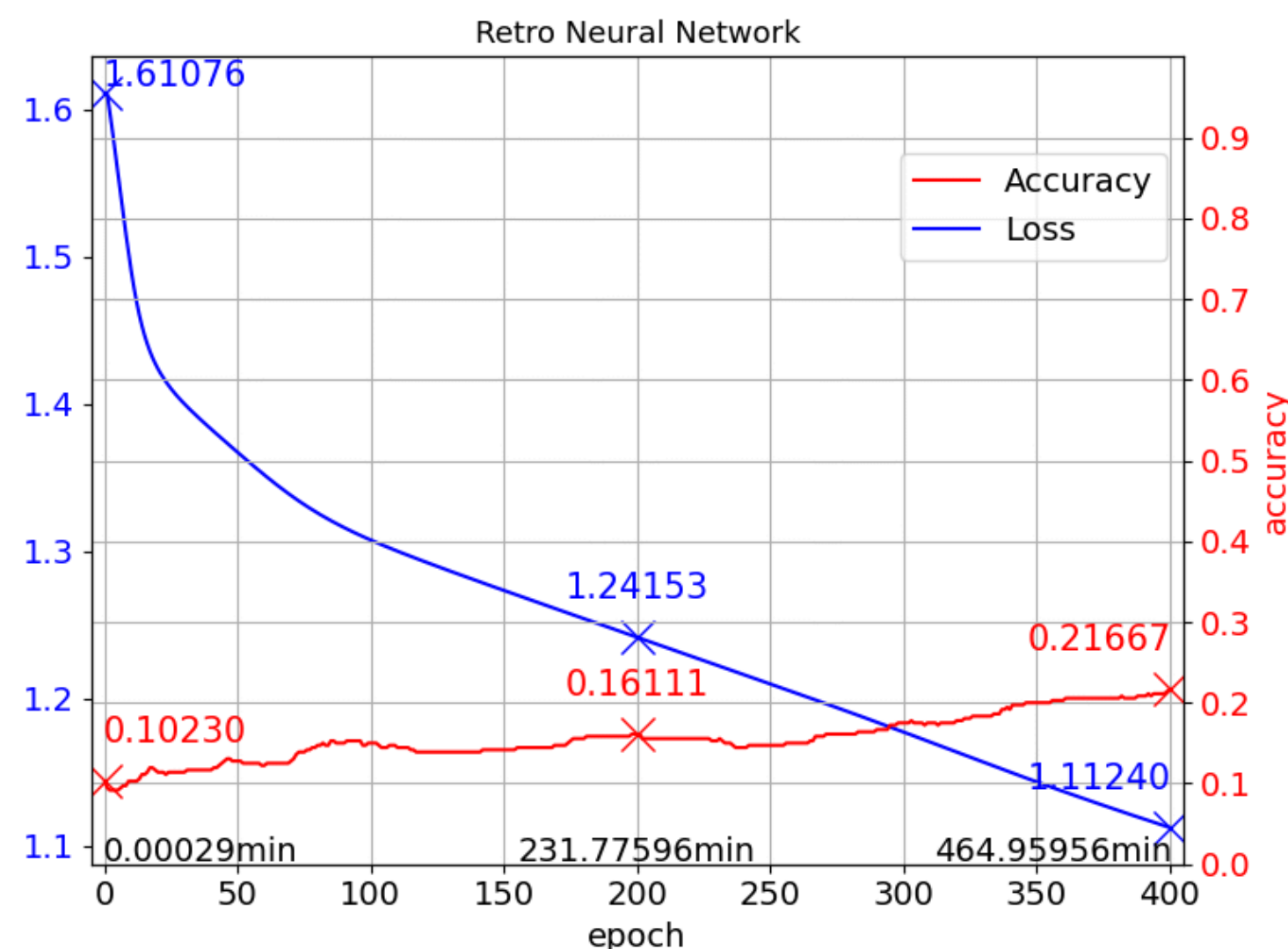
02-12-exec.py

```
import time  
max_epoch = 400 # 全データセットを400回学習  
lr = 1e-1 # 学習率、適当に設定  
loss_list, acc_list, time_list = [], [], [] # グラフ表示用
```

```
start = time.perf_counter() # 時間計測開始  
train(X_train, Y_train) # 学習はこの1コマンド  
graph(loss_list, acc_list, time_list)
```

実行結果(レトロ実装)

グラフとログ



```

Loss: 1.6107641142, Accuracy: 0.10230, Time: 0.00029[min]
loop: 1 → Loss: 1.6103457182, Accuracy: 0.09444, Time: 1.21338[min]
loop: 2 → Loss: 1.5976645880, Accuracy: 0.09167, Time: 2.33267[min]
<省略>
loop: 398 → Loss: 1.1135847417, Accuracy: 0.21111, Time: 462.56229[min]
loop: 399 → Loss: 1.1129939177, Accuracy: 0.21389, Time: 463.76959[min]
loop: 400 → Loss: 1.1124041312, Accuracy: 0.21667, Time: 464.95956[min]
  
```

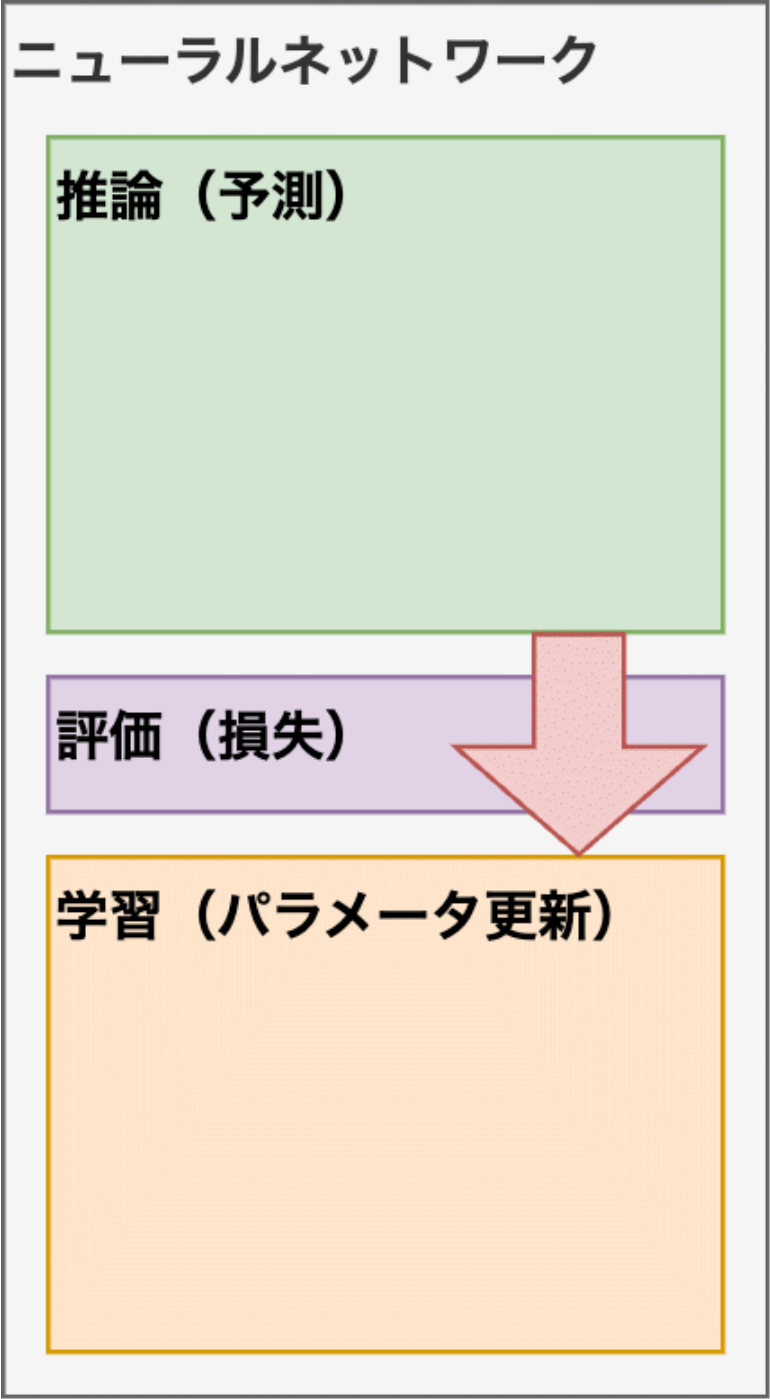
結果

- 損失値(最終 1.12)
 - 減少している → 学習している
 - **収束していない** → 学習の余地がある
- 正解率(最終 21.6%)
 - 上昇している → 学習している
 - **収束していない** → 学習の余地がある
- 時間(100エポック: 116分)
 - トータル 464.95956min

ニューラルネットワークの基本的な構成については
これで一通り説明が完了となります。
復習を兼ねてまとめておきます

ニューラルネットワークまとめ#1

大枠

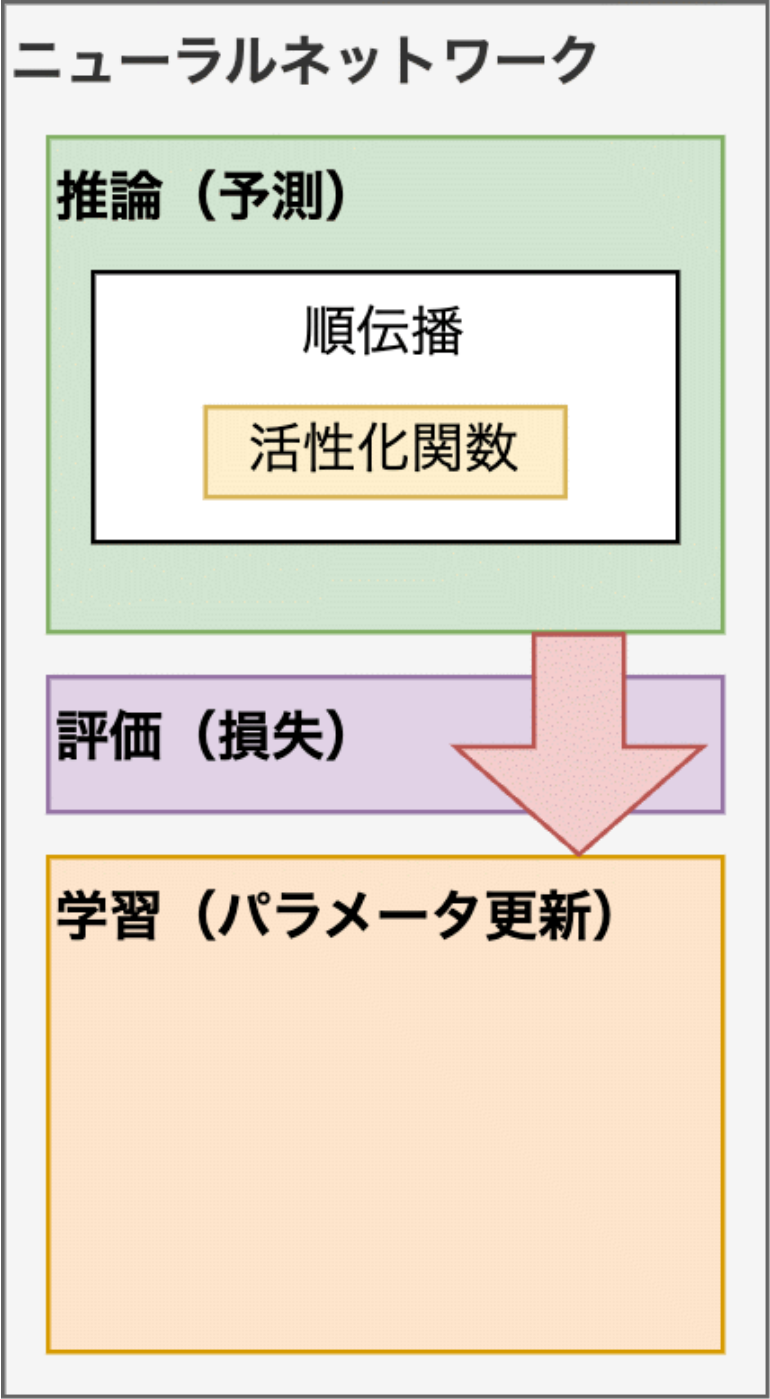


ニューラルネットワークは大まかに「推論(予測)」部分と「学習」部分に分けられます

この推論と学習を結びつけるのが「評価値(損失値)」となります

推論結果の評価値から学習を行います

推論



推論機能は「順伝播」によって行われます

順伝播は入力データ $\mathbf{x}$ に対して $\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$ とパラメータ

$\mathbf{W}$, $\mathbf{b}$ を適用し、最後に「活性化関数」 σ によって

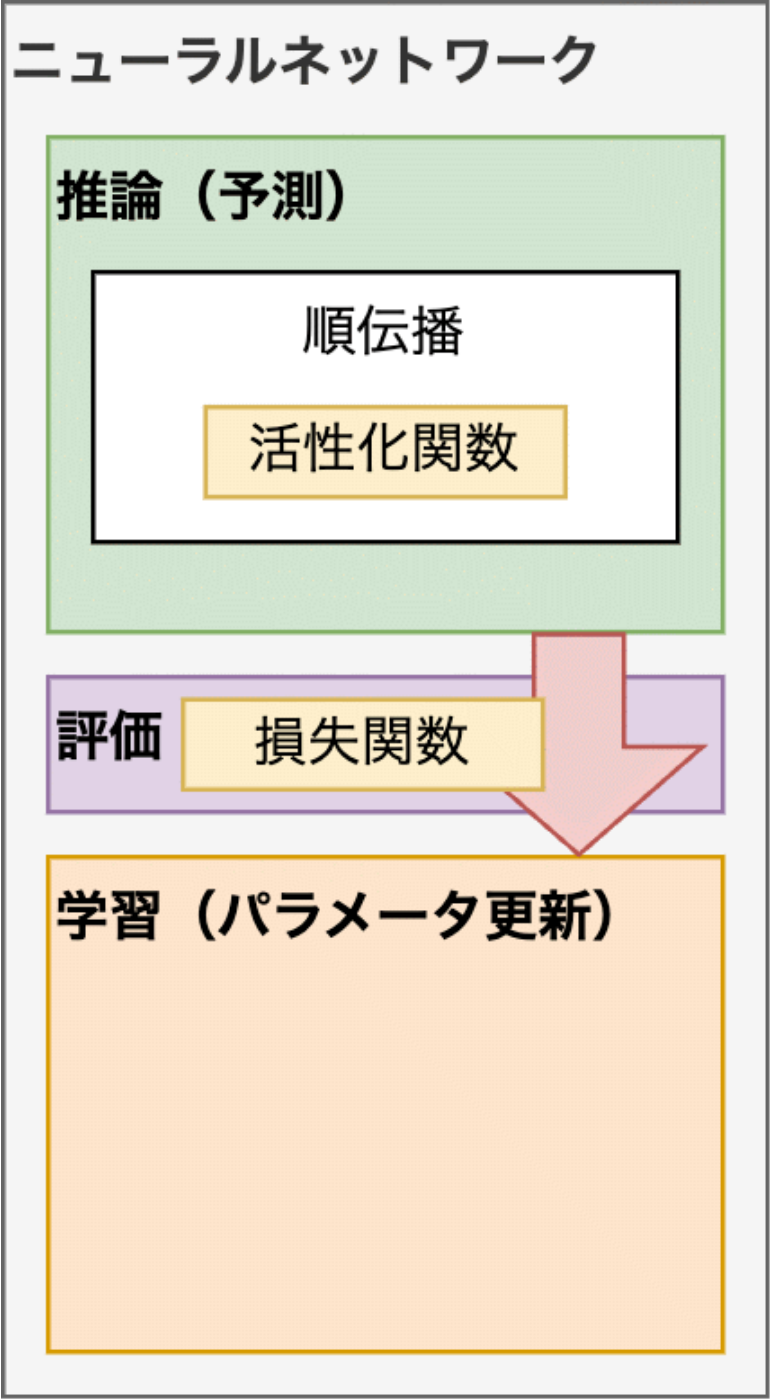
$$\mathbf{a} = \sigma(\mathbf{z})$$

となります

先の例では活性化関数としてステップ関数とsoftmax関数を用いました

ニューラルネットワークまとめ#2

評価値、損失値



推論(予測)結果に対してその精度を表す指標が「**評価値(損失値)**」となります

損失値を算出する関数 L を「損失関数」と呼びます

損失値

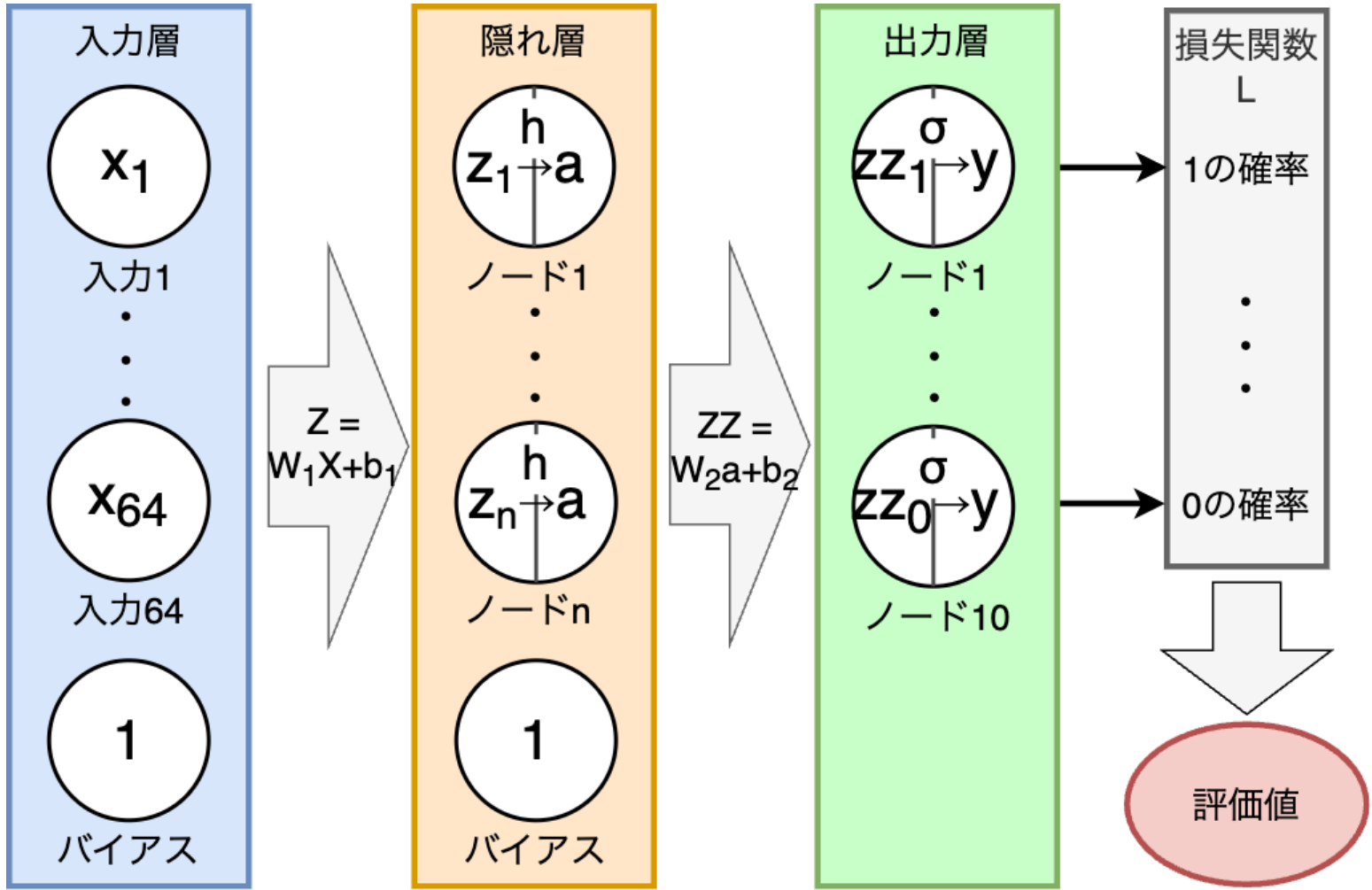
$$= L(\backslash \textcolor{red}{b}m_y, \backslash \textcolor{red}{b}\hat{m}_y)$$

($\backslash \textcolor{red}{b}m_y$:正解値、 $\backslash \textcolor{red}{b}\hat{m}_y$:予測値)

先の例では損失関数として平均二乗誤差(MSE)を用いました

補足: 損失関数のイメージ

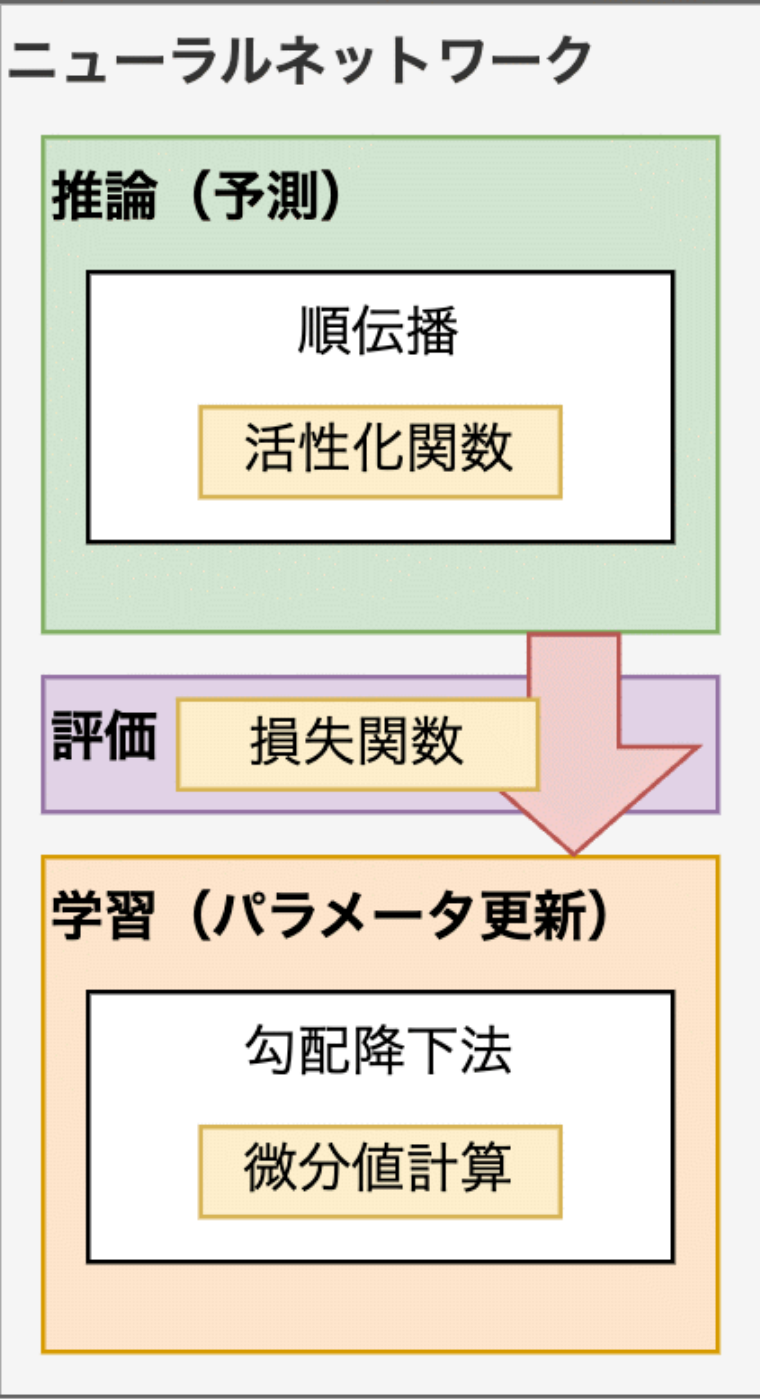
損失関数は出力層からの最終出力を直接の変数として受け取り、評価値(損失値)を計算する関数です



ニューラルネットワークまとめ#3

学習

補足:理解のポイント



学習とは端的に言うとパラメータの更新です。技法としては勾配降下法を用いてます

具体的には損失関数 L の微分 $\nabla L(\bm{W})$ より
 $\bm{W}^{(t+1)} := \bm{W}^{(t)} - \eta \nabla L(\bm{W}^{(t)})$

の計算式からパラメータを更新します

この時、微分 $\nabla L(\bm{W})$ の計算として先の例では数値微分を用いました

この後、様々な改善手法を紹介しますが、

左図構成のどの部分に対する改善か

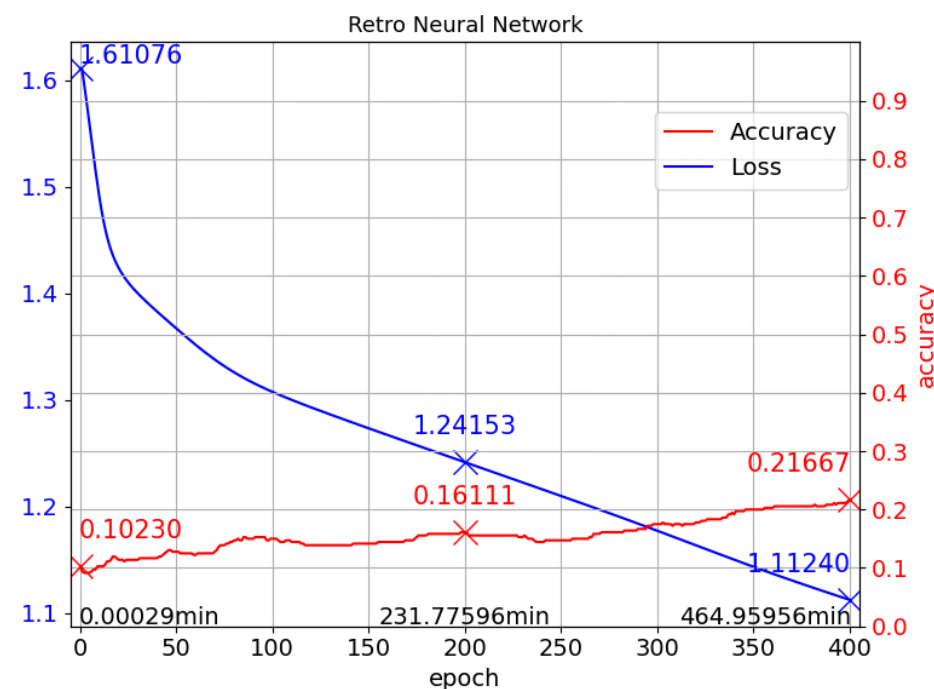
に注目頂けると理解が捗るかと思います

i 以上がNNの基本構成のまとめとなります
この後に紹介する改善手法もこの構成がベースです

§ 5. ミニバッチ学習

方針

前回の結果



正解率の上昇が遅い

着眼点は色々ですが
今回は**パラメータの更新頻度**
に注目します

どんなコードだったかを下記に
再掲

パラメータ更新の呼び出し箇所

02-09-train.py

```
def train(X, Y):
    print_metrics(X, Y) # 初期状態表示
    for i in range(max_epoch): # max_epoch = 400
        print(f'loop: {i+1} → ', end='')
        # 微分値を取得
        grads = grad_weight_all(X, Y)
        # パラメータ更新
        for key in grads.keys():
            p[key] = p[key] - lr * grads[key]
```

方針

- ループは `max_epoch` 回(今回は400)
- 更新回数は1epochごとに1回
- 1epoch辺の更新回数を増やせばどうか？
→ミニバッチ学習

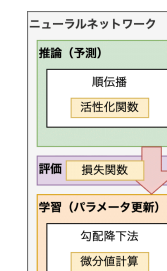
✎ 現時点ではあくまで仮説です。本来はここから検証です

i 1epochとは全データを1セット使い切ることを指します

具体的な対応

- **1epoch内で複数回パラメータ更新する**
- 逆に見ると1回のパラメータ更新には
1部のデータのみを用いる
- ついでに用いるデータの順番をシャッフルする

NN構成のどの部分か？



先に説明したNN構成図において、上記の
が該当する箇所はこの構成の外側となりま
(構成図中の要素は変わらず同じデータを学習する)

実装

実装

02-13-train-minibatch.py

```
def train_minibatch(X, Y):
    print_metrics(X_test, Y_test) # 初期状態表示
    # エポック毎にループ
    for i in range(max_epoch):
        print(f'epoch: {i+1} → ', end='')

        # 1エポック内で訓練データをシャッフル
        num = X.shape[0] # データ個数取得
        indices = np.arange(num) # データ個数分の順列を取得
        np.random.shuffle(indices) # シャッフル

        # バッチサイズごとにループ
        for j in range(0, num, batch_size):
            # バッチセットを取得
            batch_indices=indices[j:min(j+batch_size,num)]
            X_batch = X[batch_indices]
            Y_batch = Y[batch_indices]

            # 微分値を取得
            grads = grad_weight_all(X_batch, Y_batch)

            # パラメータ更新
            for key in grads.keys():
                p[key] = p[key] - lr * grads[key]
```

解説

`for j in range(0, num, batch_size):` より **1epoch内で更に複数回ループ**

- ループは `0` 番から `num` 番まで (`num` は1epoch分の総データ数)
- 1回のループごとに `batch_size` 個進む
- `batch_size` は任意の数値を指定可能

`X_batch=X[batch_indices]` は学習データ `X` から `bath_indices` で指定された番号のデータだけを取り出したもの (`Y_batch` も同様)

`微分値の計算、パラメータ更新` は前回と同じ。`X_batch` と `Y_batch` を**1setとしてパラメータ更新**

用語および実行コード

ミニバッチ学習

前ページで実装したように**1epoch**を**複数回のループに分割し各ループ毎にパラメータ更新する、という手法をミニバッチ学習**と呼びます

バッチサイズ

1回のループをバッチと呼び、そのバッチ1回で使用するデータの個数をバッチサイズと呼びます

確率的勾配降下法(SGD)

ミニバッチ学習において、使用するデータの順番をランダムにシャッフルすることを**確率的勾配降下法(SGD)**と呼びます

実行コード

02-14-exec-minibatch.py

```
import time

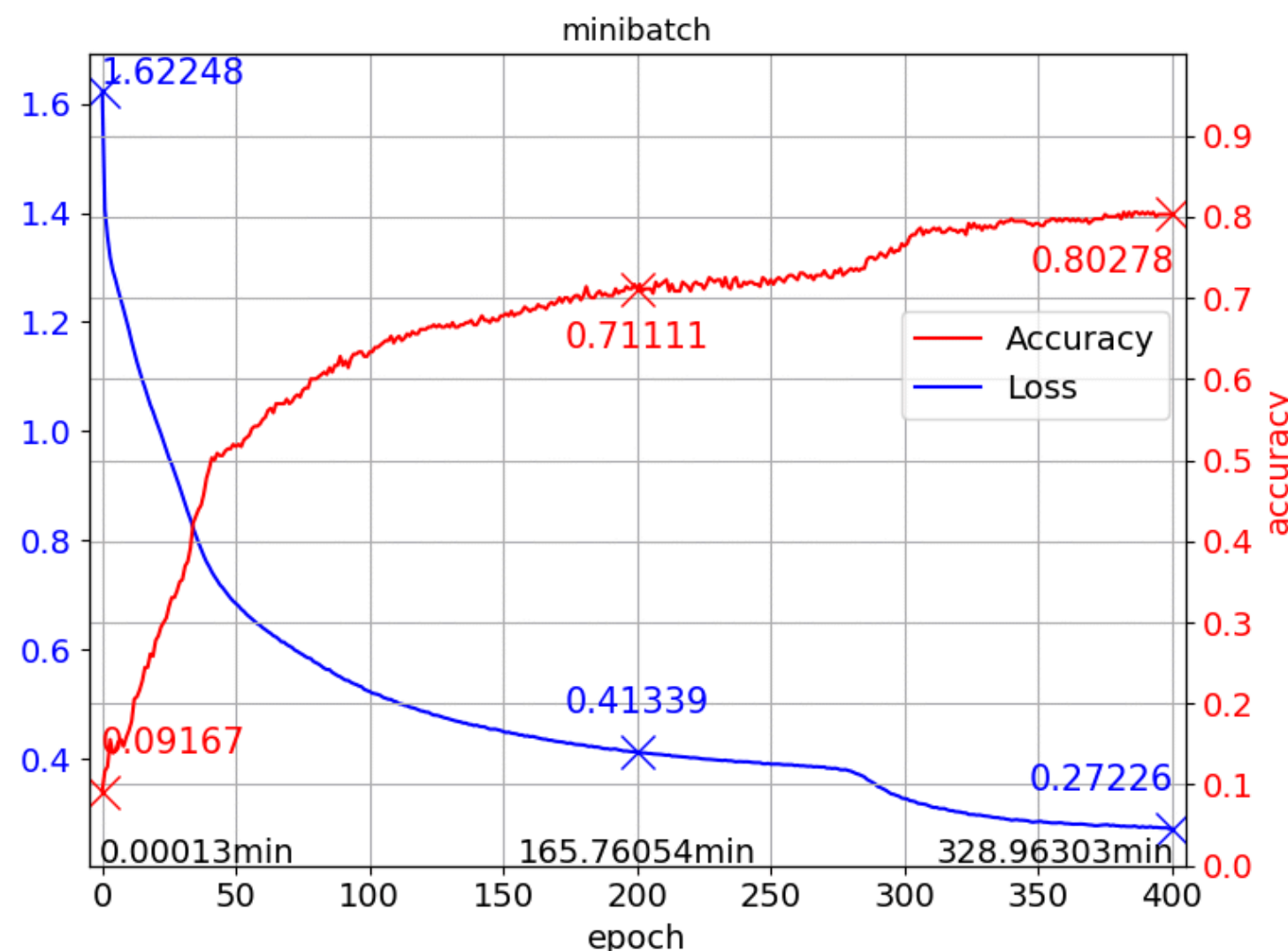
max_epoch = 400
lr = 1e-1
batch_size = 50 # これを追加
loss_list, acc_list, time_list = [], [], []

start = time.perf_counter() #計測開始
train_minibatch(X_train, Y_train)
graph(loss_list, acc_list, time_list, title='minibatch')
```

バッチサイズは50です

ミニバッチ学習(実行結果)

グラフとログ



活性化関数:step, 損失関数:MSE, バッチサイズ:50, 学習率:0.1, 隠し層:27

Loss:1.6888381853, Accuracy:0.09534, Time:0.00137[min]

epoch:1 → Loss:1.3904333381, Accuracy:0.07778, Time:0.32897[min]

epoch:2 → Loss:1.2521404843, Accuracy:0.08611, Time:0.67934[min]

<省略>

epoch:198 → Loss:0.3468973515, Accuracy:0.75833, Time:68.10825[min]

epoch:199 → Loss:0.3389557619, Accuracy:0.75278, Time:68.43988[min]

epoch:200 → Loss:0.3401196462, Accuracy:0.75556, Time:68.77476[min]

結果

- 損失値(最終 1.12→0.27)
 - 大きく減少している
 - 収束傾向
- 正解率(最終 21.6%→80.2%)
 - 初期の立ち上がり大きい
 - 大きく上昇している→学習している
 - 収束傾向
- 時間(100エポック: 116分→82分)
 - トータル328分= 約5.5時間

オンライン学習

オンライン学習

先ほどのミニバッチ学習ではバッチサイズを50にしました。結果、正解率が80%まで上昇しました。それではバッチサイズを1にしたらどうか？

次ページで実行結果を提示します

実行コード

02-15-exec-online.py

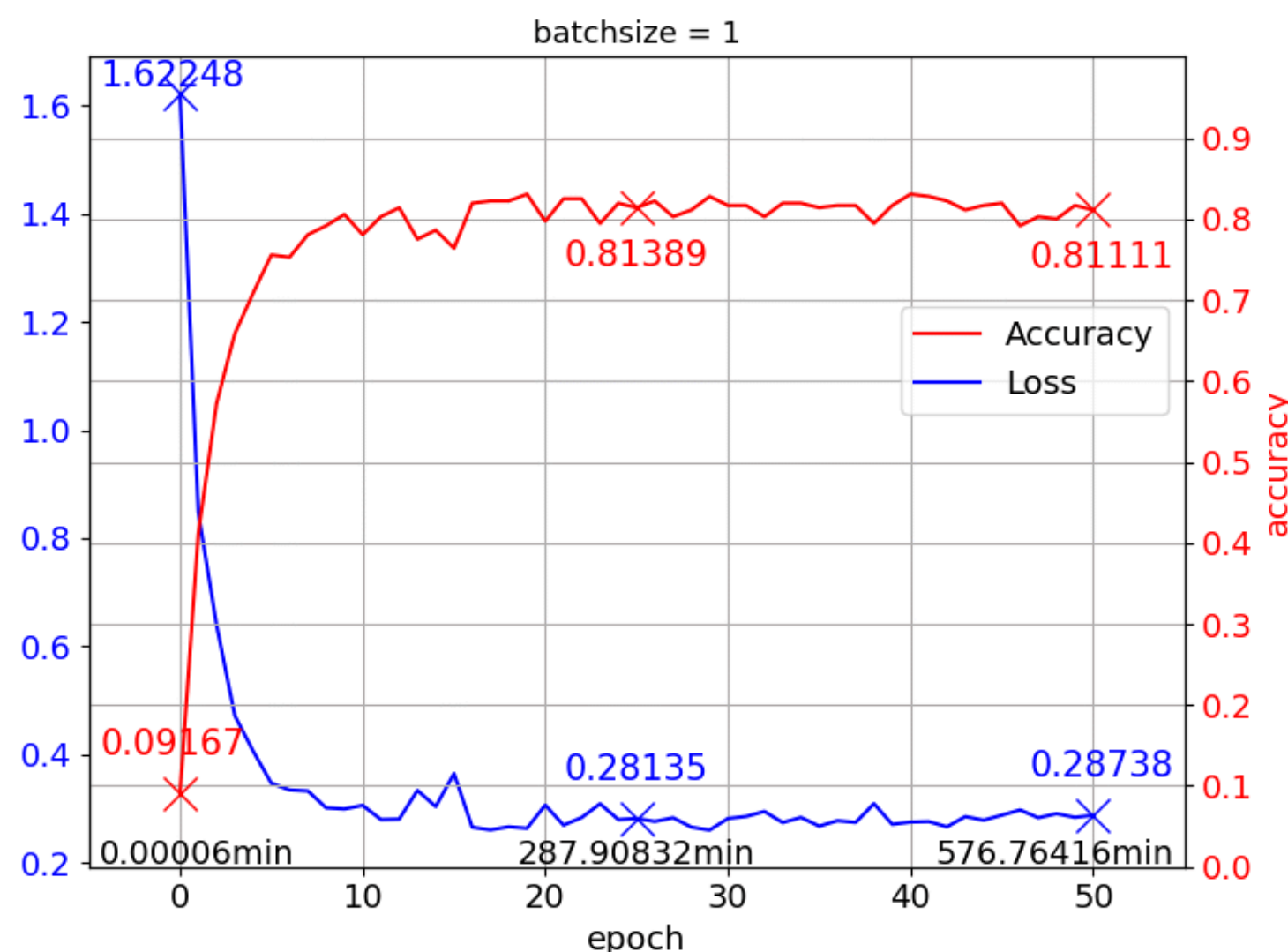
```
import time

max_epoch = 50
lr = 1e-1
batch_size = 1 # 変更はここだけ
loss_list, acc_list, time_list = [], [], []

start = time.perf_counter() #計測開始
train_minibatch(X_train, Y_train)
graph(loss_list, acc_list, time_list, title='')
```

オンライン学習(実行結果)

グラフとログ



```

Loss: 1.6224837625, Accuracy: 0.09167, Time: 0.00006[min]
epoch: 1 → Loss: 0.8482461501, Accuracy: 0.41111, Time: 11.51039[min]
epoch: 2 → Loss: 0.6394141518, Accuracy: 0.57222, Time: 22.96517[min]
<省略>
epoch: 48 → Loss: 0.2902170711, Accuracy: 0.80000, Time: 553.73990[min]
epoch: 49 → Loss: 0.2834401250, Accuracy: 0.81667, Time: 565.27932[min]
epoch: 50 → Loss: 0.2873804376, Accuracy: 0.81111, Time: 576.76416[min]
  
```

結果

- 損失値
(0.27→0.28)
- 正解率
(80.2%→81.1%)
 - ほぼ同じ
 - 収束はしている
 - ギザギザ傾向
- 時間(100エポック:
82分→1152分)
 - トータル576分=約9.5時間

評価

- 収束した(最初の10~20エポックで)
- 1エポックの時間が長すぎ 43/101

§ 6. リファクタリング

この後も色々な手法を紹介するのですが、
実装しやすくするためにまずコード全体を整理しておきます

リファクタリングとは

ざっくり言うと

リファクタリングとは「外部から見た振る舞いを保ったまま、ソフトウェアの内部構造を改善する」事です。下記のような利点があります

- コードが理解しやすくなる
- メンテしやすくなる
- バグ対応がしやすくなる
- 拡張しやすくなる

「振る舞いを保つ」とは？

入力に対する出力(=機能)が変わらない事です

ファイル書き換えや通信などといった副次的動作も基本は同じままです。目的は内部構造の改善です

i 「振る舞いを保つ」の基準は対象によります

具体的に何をするのか？

今回は下記を実施します

- Pythonらしいコードに変更
- クラス化

コードをPythonらしく変更

`accuracy()`

`python`と`numpy`はベクトルデータ、行列データをまとめて処理できるので下記のような書き方が可能です。慣れると便利です(私だ慣れてません)

02-16-refactor.py

```
def accuracy(Y_pred, Y):  
    """ 正解率 (Python的実装) """  
    return np.mean(np.argmax(Y_pred, axis=1) == Y)
```

`one_hot_batch()`

行列をワンホットベクトルのリストとみなして添え字に正解ラベル指定することで正解のワンホットベクトルを抜き出す、という操作をしています(Python的には一般的な感じ? 本当に、全然慣れま

02-16-refactor.py

```
def one_hot_batch(k_list, num_classes=10):  
    """ (n,1)のリストを(n,10)のone_hot_vecリストに変換 """  
    return np.eye(num_classes)[k_list]
```

`grad_numeric()`

二重ループを単ループに変更してます

02-16-refactor.py

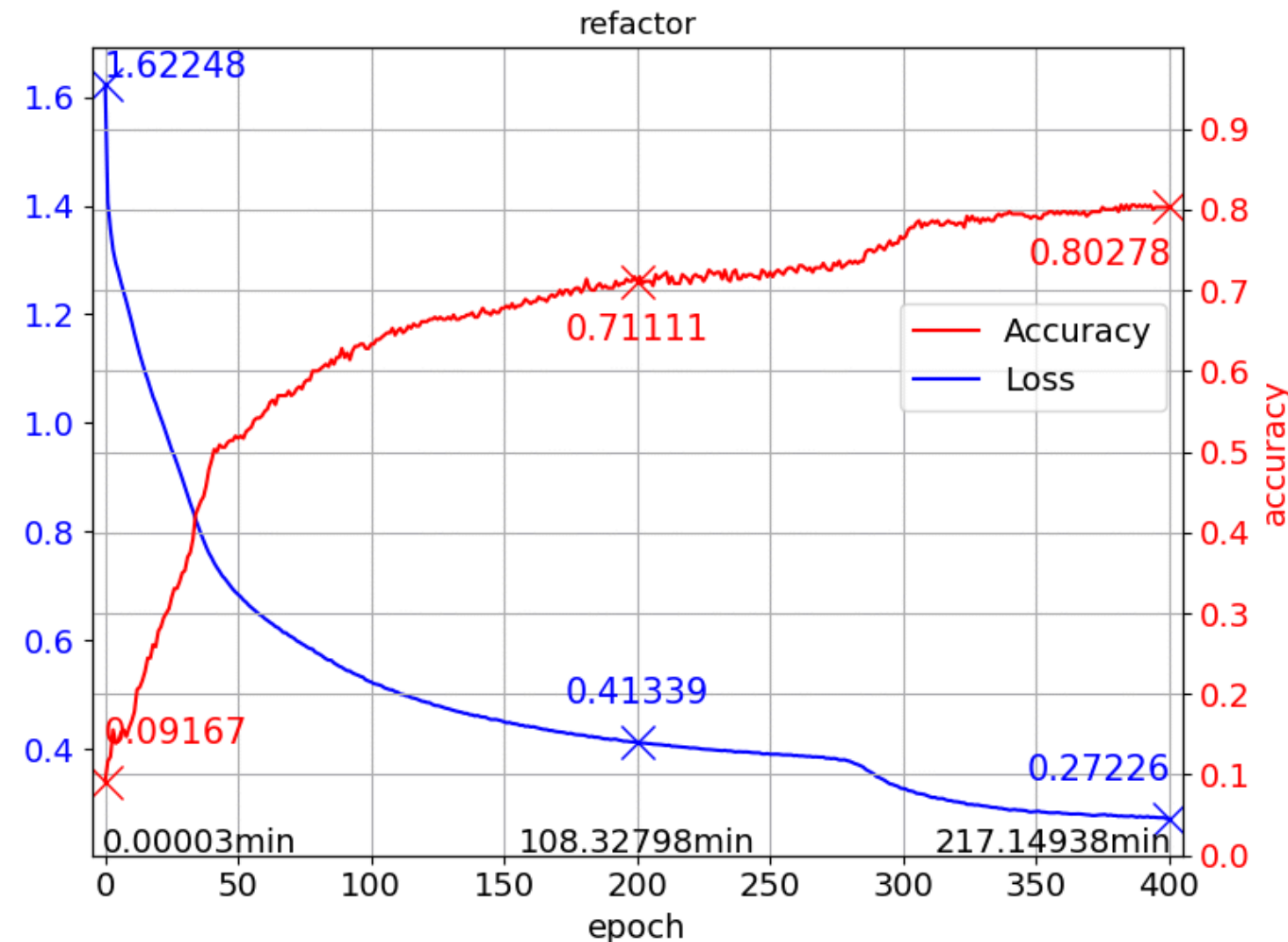
```
def grad_numeric(L, W):  
    """ 微分 (L:微分対象の関数、W:微分変数) """  
    h = 1e-1  
    grad = np.zeros_like(p[W])  
    iter = np.nditer(p[W], flags=['multi_index'])  
    while not iter.finished: # 単ループに変更  
        idx = iter.multi_index  
        original = p[W][idx].copy()  
        p[W][idx] = original + h  
        L1 = L()  
        p[W][idx] = original - h  
        L2 = L()  
        grad[idx] = (L1 - L2) / (2 * h)  
        p[W][idx] = original  
        iter.iternext()  
    return grad
```

実行コード

02-17-exec-refactor.py

関数リファクタ後(実行結果)

グラフとログ



```
Loss: 1.6224837625, Accuracy: 0.09167, Time: 0.00003[min]
epoch: 1 → Loss: 1.4060173063, Accuracy: 0.11667, Time: 0.54791[min]
epoch: 2 → Loss: 1.3562544863, Accuracy: 0.12222, Time: 1.11180[min]
<省略>
epoch: 398 → Loss: 0.2739740340, Accuracy: 0.80278, Time: 216.03700[min]
epoch: 399 → Loss: 0.2720686037, Accuracy: 0.80278, Time: 216.60802[min]
epoch: 400 → Loss: 0.2722565645, Accuracy: 0.80278, Time: 217.14938[min]
```

結果

- 損失値
(0.27→0.27)
 - 全く同じ値。OK
- 正解率
(80.2%→80.2%)
 - これも全く同じ値。OK
- 時間(100エポック:
82分→54分)
 - トータル217分=約3.5時間
 - やや減少

評価

- 実行結果(動作)をキープしたまま
コードの改善ができた

クラス化

クラス化の理由

- 改善手法の実装を簡単にするため
- どの機能への改善かを見通しよくするため

大枠の構造

- 全体を `MyNetwork` クラスにまとめる
- 隠れ層、出力層は `MyLinearLayer` クラスに分離
- `forward`, `grad`, `loss`, `train` はクラス内に格納
- 活性化関数と損失関数の本体は個別関数のまま
(あとで入れ替えしやすいように)

```
MyLinearLayer#__init__()
```

```
MyLinearLayer():
""" 線形層 """
def __init__(self, input_size, output_size, activation):
    self.activation, self.activation_prime \
        = funcs_map[activation]
    self.W = np.random.randn(input_size, output_size)
    self.b = np.array([np.zeros(output_size)])
```

つづき

`activation` は活性化関数です。`funcs_map` 辞書で取得します。

`activation_prime` は活性化関数の微分関数です
実装します

`funcs_map` は下記

03.py

```
# 活性化関数/損失関数の名前から関数と導関数を紐付け
funcs_map = {
    'step': (step_function, step_function_
    'softmax': (softmax, softmax_prime),
    'mse': (loss_mse, loss_mse_prime),
    'MSE': (loss_mse, loss_mse_prime),
}
# <中略>
""" 微分関数。名前だけ確保。あとで実装 """
def step_function_prime(x): # placeholder
    return
def relu_prime(x): # placeholder
    return
def softmax_prime(x): # placeholder
    return
def loss_mse_prime(x): # placeholder
```

クラス化#2

MyLinearLayer#forward()

```
def forward(self, X):
    """ 順伝播 """
    self.Z = X @ self.W + self.b # 線形出力 (N, output_size)
    self.A = self.activation(self.Z) # 活性化出力 (N, output_size)
    return self.A
```

forwardと同じ処理。ただし層1つ分のみ

MyLinearLayer#grad()

```
def grad(self, X, Y, loss_fn, param, h=1e-4):
    """ 数値微分 """
    grad = np.zeros_like(param)
    iter = np.nditer(param, flags=['multi_index'])
    while not iter.finished:
        idx = iter.multi_index
        original = param[idx].copy() # 現在値をコピっとく
        param[idx] = original + h
        L1 = loss_fn() # L(w + h)を算出
        param[idx] = original - h
        L2 = loss_fn() # L(w - h)を算出
```

MyLinearLayer#train_numeric

03.py

```
def train_numeric(self, X, Y, loss_fn, lr):
    """ 数値微分でパラメータ更新 """
    dW = self.grad(X, Y, loss_fn, self.W)
    db = self.grad(X, Y, loss_fn, self.b)
    self.W -= lr * dW
    self.b -= lr * db
```

勾配降下法を適用。対象は層1つ分

クラス化#3

MyNetwork#__init__()

```
def __init__(self, loss_name='cross_entropy', activation='relu'):
    self.loss_fn, self.loss_prime = funcs_map[loss_name]
    self.layers = [
        LinearLayer(input_size, hidden_size, activation='step'),
        LinearLayer(hidden_size, output_size, activation='softmax'),
    ]
```

`LinearLayer` を2つ含んでいます。1つ目が隠れ層、2つ目が出力層

`loss_prime` は損失関数とその微分で、`funcs_map` から取得します。

MyNetwork#forward()

```
def forward(self, X):
    """
    順伝播
    """
    for layer in self.layers:
        X = layer.forward(X)
```

MyNetwork#loss()

03.py

```
def loss(self, X, Y, L2=0):
    Y_pred = self.forward(X)
    loss_value = self.loss_fn(Y_pred, Y)
    return loss_value
```

損失関数の計算。実体は `loss_fn`

MyNetwork#train_numeric

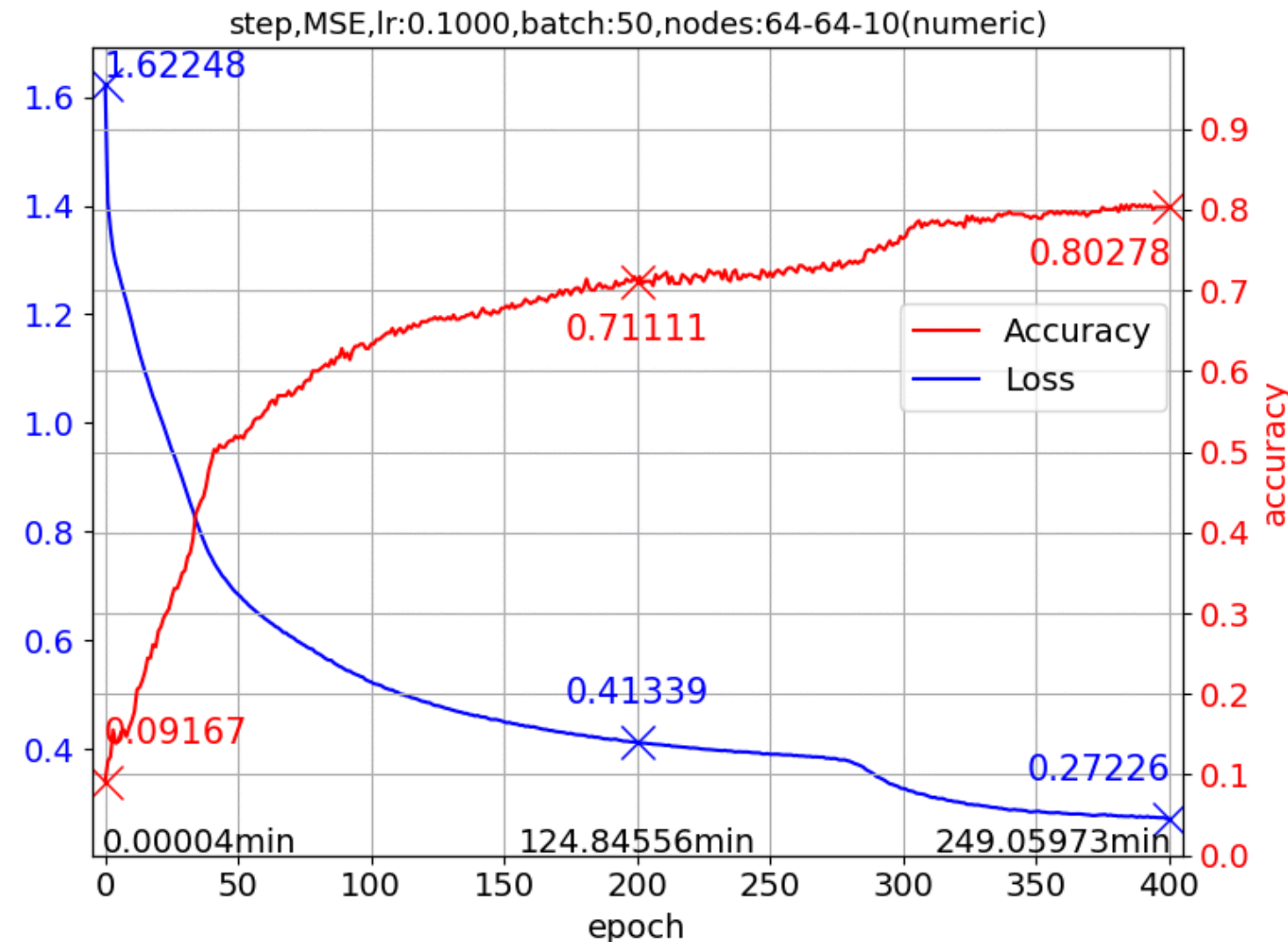
03.py

```
def train_numeric(self, X, Y, lr=1e-2):
    """ 数値微分でパラメータ更新 """
    L = lambda: self.loss(X, Y) # layer loss
    for layer in self.layers:
        layer.train_numeric(X, Y, L, lr)
    return L
```

パラメータ更新を `layers` 内の全ての層に対して行う。
先の `loss` で、`lambda` 式にすることで `layer` ごとに `loss` を計算する。

クラス化(実行結果)

グラフとログ



結果

- 損失値
(0.27→0.27)
 - 全く同じ値。OK
- 正解率
(80.2%→80.2%)
 - これも全く同じ値。OK
- 時間(100エポック:
54分→62分)
 - トータル249分=約4.0時間
 - やや増加

評価

- 実行結果(動作)をキープしたまま
コードの改善ができた

step,MSE,lr:0.1000,batch:50,nodes:64-64-10(numeric)

Loss: 1.6224837625, Accuracy: 0.09167, Time: 0.00004[min]

epoch: 1 → Loss: 1.4060173063, Accuracy: 0.11667, Time: 0.64135[min]

epoch: 2 → Loss: 1.3562544863, Accuracy: 0.12222, Time: 1.26345[min]

<省略>

epoch: 398 → Loss: 0.2739740340, Accuracy: 0.80278, Time: 247.84502[min]

epoch: 399 → Loss: 0.2720686037, Accuracy: 0.80278, Time: 248.45844[min]

epoch: 400 → Loss: 0.2722565645, Accuracy: 0.80278, Time: 249.05973[min]

§ 7. 損失関数の変更

損失関数をMSEから交差エントロピーに変更します

交差エントロピー誤差

定義と分類問題での解釈

分類問題に於いて選択肢の個数が C 個の場合の交差エントロピー誤差は次の通りです(今回は $C = 10$)

$$L_1 = - \sum_{i=1}^C y_i \log \hat{y}_i$$

$i = k$ を正解とすると $y_k = 1$ で他の $y_i = 0$ なので

$$L_1 = - \sum_{i=1}^C y_i \log \hat{y}_i = - \log \hat{y}_k$$

N 数の場合、足して割る

サンプル数が N 個の場合は次のようになります

$$L_N = - \frac{1}{N} \sum_{n=1}^N \sum_{i=1}^C y_{n,i} \log \hat{y}_{n,i}$$

実装

03.py

```
def cross_entropy(Y_pred, Y):  
    """ クロスエントロピー誤差 (改訂版) """  
    d = 1e-8  
    batch_size = Y.shape[0]  
    correct_probs = Y_pred[np.arange(batch_size), Y]  
    log_likelihood = -np.log(correct_probs + d)  
    loss = np.mean(log_likelihood)  
    return loss
```

交差エントロピーの出所

[最尤推定法](#)です。最尤推定法とは「最(一番)」「尤(もっともらしい)」「推定(値を予測)」という意味です。

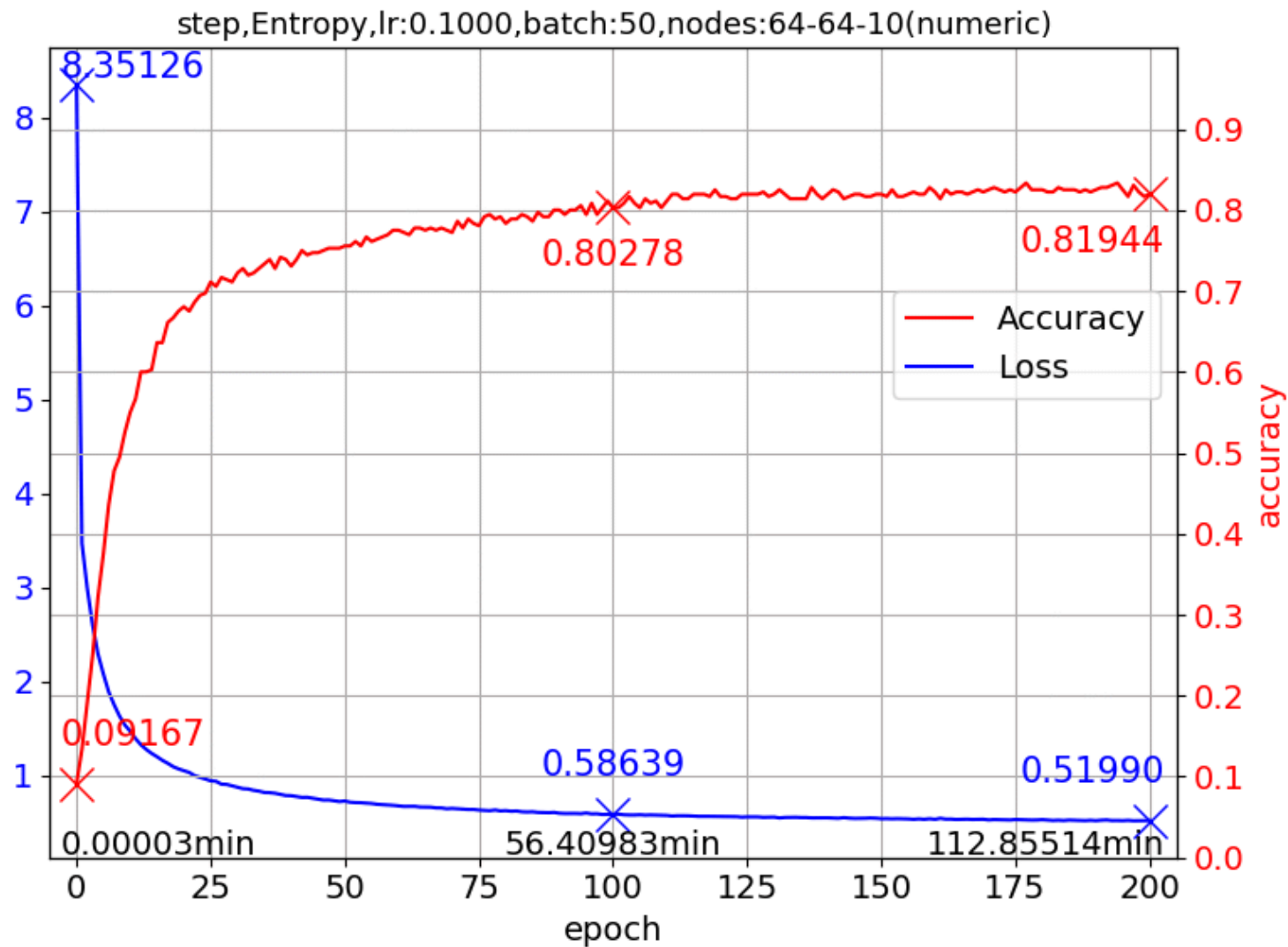
閑話:用語の補足

- エントロピーとついてるけど[物理](#)とは関係ない
- 情報学における初出は[シャノンの情報理論](#)

数式が物理におけるエントロピーと同形なのでエントロピーと537101

クロスエントロピー誤差(実行結果)

グラフとログ



結果

- 損失値
(0.27→0.51)
 - 関数を変えたので比較は意味なし
- 正解率
(80.2%→81.9%)
 - 少し上昇
- 時間(100エポック:
62分→56分)
 - トータル112分=約2.0時間
 - トータル半減

評価

- 正解率はほぼ同じだが、エポック数を400から200に減らして101

```
step,Entropy,lr:0.1000,batch:50,nodes:64-64-10(numeric)
Loss: 8.3512616600, Accuracy: 0.09167, Time: 0.00003[min]
epoch: 1 → Loss: 3.4750508441, Accuracy: 0.13333, Time: 0.56560[min]
epoch: 2 → Loss: 2.9898804363, Accuracy: 0.19167, Time: 1.10969[min]
<省略>
epoch: 198 → Loss: 0.5189744374, Accuracy: 0.82222, Time: 111.69308[min]
epoch: 199 → Loss: 0.5200585853, Accuracy: 0.81667, Time: 112.28882[min]
epoch: 200 → Loss: 0.5198975577, Accuracy: 0.81944, Time: 112.85514[min]
```

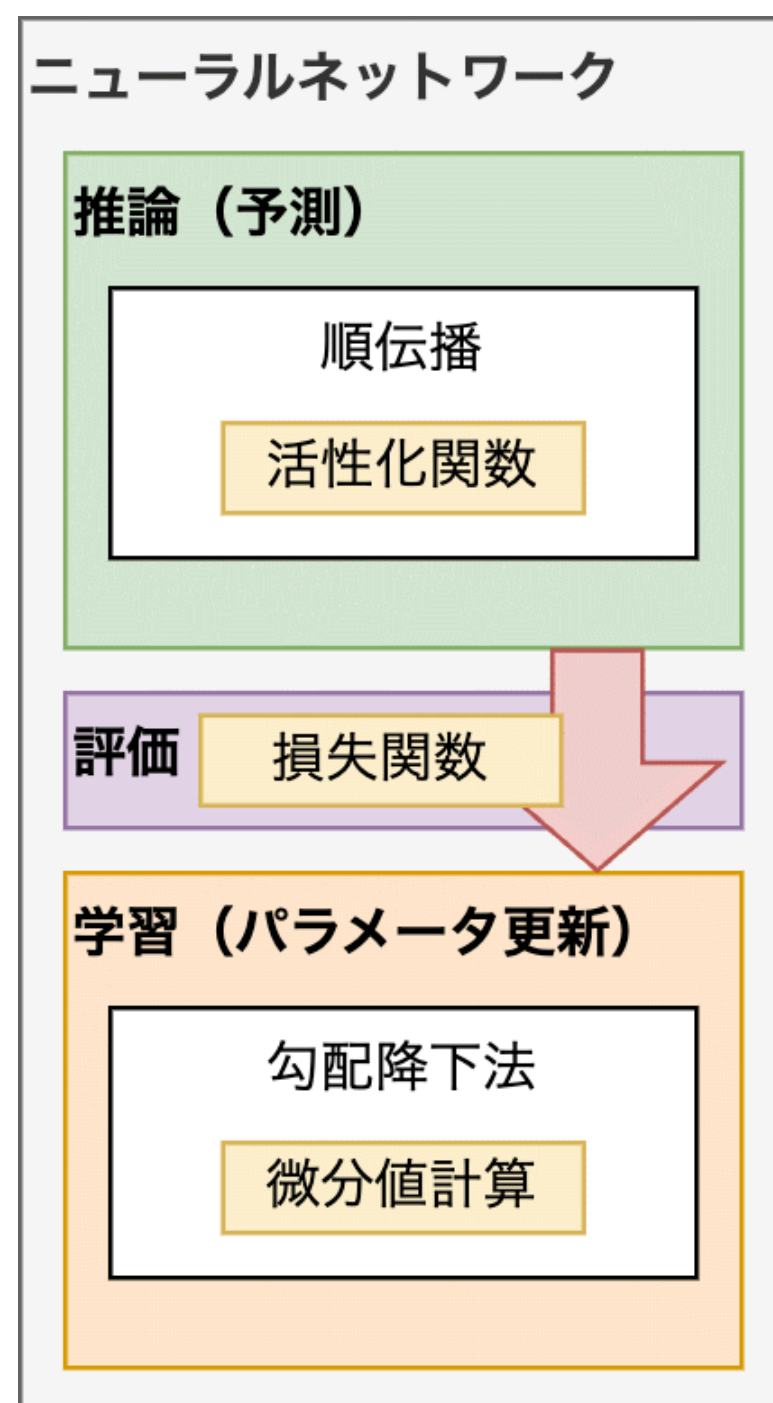

§ 8. 誤差逆伝播法

背景とアイデアと有効性が理解できればOKです。理論ベースです

背景

学習にかかる時間

ここまでの学習は1時間/100epoch程度でした
最も時間がかかっているのが微分値計算です



数値微分の計算量

数値微分はアイデアは素直ですが計算量が多いです

1パラメータの更新のために順伝播を2回実行する必要がある事と、その計算が全パラメータを絡めた計算として繰り返される事が原因です
ざっくり $O(N^2)$ です

この計算量の削減の手法のうち効果が限界突破的に高かったものが誤差逆伝播法です

初手は微分式の見直しです

✎ 誤差逆伝播法の位置付けは左図の「微分値計算」です
WEB上の情報眺めてると「学習全体」みたいな記述も見かけますが厳密には誤りです

≡ 100エポック1時間、昔だと？

50年前だと1年程度です。CPUを比較すると
今=4GHz、50年前=4MHz → 1000倍=6週間
今=64bit、50年前=8bit → 8倍=1年間
しかもマルチコアやその他の改良なども含めると...

微分式の見直し(出力層) #1

0. 微分式を検証してみる

予測値を $\hat{y}$ 、正解を y 、損失関数を $L(\hat{y}, y)$ とする。
出力層に注目してみると、

- 出力層への入力: $\mathbf{a}$
- 重み行列、バイアス: $\mathbf{W}$ 、 $\mathbf{b}$
- 出力層の内部出力:
 $\mathbf{z} = \mathbf{W}\mathbf{a} + \mathbf{b}$
- 出力層の出力: $\hat{\mathbf{y}} = \sigma(\mathbf{z})$

数値微分では $\frac{\partial L}{\partial \mathbf{W}}$ を定義式から直接算出しました。

今度は微分式をチェーンルールで展開します

まず、目的は重み $\mathbf{W}$ に対する損失 L の勾配

$\frac{\partial L}{\partial \mathbf{W}}$ を求めることです。チェーンルールを用い

て、以下のように分解できます:

$$\frac{\partial L}{\partial \mathbf{W}} = \frac{\partial L}{\partial \hat{\mathbf{y}}} \frac{\partial \hat{\mathbf{y}}}{\partial \mathbf{z}} \frac{\partial \mathbf{z}}{\partial \mathbf{W}}$$

1. $\frac{\partial L}{\partial \hat{\mathbf{y}}}$

損失関数 L の出力 $\hat{\mathbf{y}}$ による偏微分です。クロスエントロピー誤差の場合、正解ラベルを k とすると

$$L = -\frac{1}{N} \sum_{n=1}^N \sum_{j=1}^C y_{n,j} \log(\hat{y}_{n,j}) = -\frac{1}{N} \sum_{n=1}^N \log(\hat{y}_{n,k})$$

ある n における j 番目のクラスに対する微分は

$$\frac{\partial L}{\partial \hat{y}_{n,j}} = -\frac{y_{n,j}}{\hat{y}_{n,j}} \Rightarrow \frac{\partial L}{\partial \hat{y}_{n,j}} = \begin{cases} -\frac{1}{\hat{y}_{n,k}} & , j = k \\ 0 & , j \neq k \end{cases}$$

$\frac{\partial L}{\partial \hat{\mathbf{y}}}$ 全体はベクトルになりますが、正解の成分以外

は0になります

✎ 1つの成分だけ $-\frac{1}{\hat{y}_k}$ で他は 0 (大切なので2回書きました)

📌 チェーンルール展開 = 計算量削減ではないです
現時点では「ここから模索していく」状態です

微分式の見直し(出力層) #2

2. $\partial \hat{\mathbf{y}} / \partial \mathbf{z}$

softmax 関数のヤコビ行列 J になります

$$\hat{y}_i = \text{softmax}(z_i) = \frac{e^{z_i}}{S} \quad (S = \sum_{l=1}^n e^{z_l})$$

より

$$J = \frac{\partial \hat{\mathbf{y}}}{\partial \mathbf{z}} = \text{diag}(\hat{\mathbf{y}}) - \hat{\mathbf{y}} \hat{\mathbf{y}}^\top$$

算出

i) $\hat{y}_i$ を z_i で微分

$$\frac{\partial \hat{y}_i}{\partial z_i} = \frac{\partial}{\partial z_i} \frac{e^{z_i}}{S} = \frac{(e^{z_i})' S - e^{z_i} S'}{S^2}$$

ここで

$$(e^{z_i})' = \frac{\partial e^{z_i}}{\partial z_i} = e^{z_i}, S' = \frac{\partial S}{\partial z_i} = \frac{\partial \sum_l e^{z_l}}{\partial z_i} = e^{z_i}$$

より

$$\begin{aligned} \frac{\partial \hat{y}_i}{\partial z_i} &= \frac{(e^{z_i})' S - e^{z_i} S'}{S^2} = \frac{e^{z_i} S - e^{z_i} e^{z_i}}{S^2} \\ &= \frac{e^{z_i}}{S} - \frac{e^{z_i} e^{z_i}}{S^2} = \hat{y}_i - \hat{y}_i \hat{y}_i \end{aligned}$$

ii) $\hat{y}_i$ を z_j で微分 ($i \neq j$)

$$\frac{\partial \hat{y}_i}{\partial z_j} = \frac{\partial}{\partial z_j} \frac{e^{z_i}}{S} = \frac{(e^{z_i})' S - e^{z_i} S'}{S^2}$$

ここで

$$(e^{z_i})' = \frac{\partial e^{z_i}}{\partial z_j} = 0, S' = \frac{\partial S}{\partial z_j} = \frac{\partial \sum_l e^{z_l}}{\partial z_j} = e^{z_j}$$

より

$$\begin{aligned} \frac{\partial \hat{y}_i}{\partial z_j} &= \frac{(e^{z_i})' S - e^{z_i} S'}{S^2} = \frac{0 \cdot S - e^{z_i} e^{z_j}}{S^2} \\ &= 0 - \frac{e^{z_i} e^{z_j}}{S^2} = -\hat{y}_i \hat{y}_j \end{aligned}$$

行列としてまとめると $\frac{\partial \hat{\mathbf{y}}}{\partial \mathbf{z}} = \text{diag}(\hat{\mathbf{y}}) - \hat{\mathbf{y}} \hat{\mathbf{y}}^\top$

微分式の見直し(出力層) #3

3. $\partial L / \partial \backslash \mathbf{bm} z$

$$\frac{\partial L}{\partial \backslash \mathbf{bm} z} = \frac{\partial L}{\partial \backslash \mathbf{bm} y} \frac{\partial \backslash \mathbf{bm} y}{\partial \backslash \mathbf{bm} z} \text{ を求めます。 } \frac{\partial L}{\partial \backslash \mathbf{bm} y}$$

は k 番目の要素以外が 0 のベクトル、 $\frac{\partial \backslash \mathbf{bm} y}{\partial \backslash \mathbf{bm} z}$ は行列

です。よって、 $\frac{\partial L}{\partial \backslash \mathbf{bm} z} = \frac{\partial L}{\partial \hat{y}_k} \left(\frac{\partial \backslash \mathbf{bm} y}{\partial \backslash \mathbf{bm} z} \right)_k$ (つまり

k 行目だけ取り出し)

i 番目の成分を考えると $\frac{\partial L}{\partial \hat{y}_k} = -\frac{1}{\hat{y}_k}$ の項は i によら

ず一定。 $\left(\frac{\partial \backslash \mathbf{bm} y}{\partial \backslash \mathbf{bm} z} \right)_k$ は $i = k$ と $i \neq k$ で異なるので

i) $i = k$ の場合

$$\frac{\partial \hat{y}_k}{\partial z_k} = \hat{y}_k - \hat{y}_k \hat{y}_k \text{ となり}$$

続き

ii) $i \neq k$ の場合

$$\frac{\partial \hat{y}_k}{\partial z_i} = -\hat{y}_k \hat{y}_i \text{ となり}$$

$$\left(\frac{\partial L}{\partial \backslash \mathbf{bm} z} \right)_{i \neq k} = -\frac{1}{\hat{y}_k} (-\hat{y}_k \hat{y}_i) = \hat{y}_i = \hat{y}_i - 0$$

まとめると

$$\left(\frac{\partial L}{\partial \backslash \mathbf{bm} z} \right)_i = \begin{cases} \hat{y}_i - 1 & , i = k \\ \hat{y}_i - 0 & , i \neq k \end{cases}$$

ワンホットベクトルで表現すると

$$\frac{\partial L}{\partial \backslash \mathbf{bm} z} = \backslash \mathbf{bm} y - \backslash \mathbf{bm} y$$

微分式の見直し(出力層) #4

4. $\partial z / \partial \mathbf{W}$

$\mathbf{z} = \mathbf{W}\mathbf{a} + \mathbf{b}$ を $\mathbf{W}$ で微分します

$$\frac{\partial \mathbf{z}}{\partial \mathbf{W}} = \frac{\partial (\mathbf{W}\mathbf{a})}{\partial \mathbf{W}} = \mathbf{a}^T$$

✎ 出力はベクトルですが、縦横は後続の計算に合わせてます

③ 上記式の算出は省略

5. 出力層の勾配 $\partial L / \partial \mathbf{W}$

以上より
$$\frac{\partial L}{\partial \mathbf{W}} = \frac{\partial L}{\partial \hat{\mathbf{y}}} \frac{\partial \hat{\mathbf{y}}}{\partial \mathbf{z}} \frac{\partial \mathbf{z}}{\partial \mathbf{W}}$$

について

$$\frac{\partial L}{\partial \mathbf{W}} = \frac{\partial L}{\partial \mathbf{z}} \frac{\partial \mathbf{z}}{\partial \mathbf{W}} = (\hat{\mathbf{y}} - \mathbf{y}) \mathbf{a}^T$$

6. δ_{out} の導入

次のように δ_{out} を導入します

$$\frac{\partial L}{\partial \mathbf{W}} = \delta_{out} \mathbf{a}^T \quad \left(\delta_{out} = \frac{\partial L}{\partial \hat{\mathbf{y}}} \frac{\partial \hat{\mathbf{y}}}{\partial \mathbf{z}} = \frac{\partial L}{\partial \mathbf{z}} \right)$$

δ_{out} を使って他の微分項を求められます

$$\frac{\partial L}{\partial \mathbf{b}} = \frac{\partial L}{\partial \hat{\mathbf{y}}} \frac{\partial \hat{\mathbf{y}}}{\partial \mathbf{z}} \frac{\partial \mathbf{z}}{\partial \mathbf{b}} = \delta_{out} \mathbf{1}^T$$

$$\frac{\partial L}{\partial \mathbf{a}} = \frac{\partial L}{\partial \hat{\mathbf{y}}} \frac{\partial \hat{\mathbf{y}}}{\partial \mathbf{z}} \frac{\partial \mathbf{z}}{\partial \mathbf{a}} = \delta_{out} \mathbf{W}^T$$

$\frac{\partial L}{\partial \mathbf{b}}$ は $\mathbf{z} = \mathbf{W}\mathbf{a} + \mathbf{b}$ の $\mathbf{b}$ の
パラメータ更新に使います

$\frac{\partial L}{\partial \mathbf{a}}$ は前の層(隠れ層)の誤差逆伝播で使います

微分式の見直し(隠れ層) #1

0. 隠れ層

予測値を $\hat{y}$ 、正解を y 、損失関数を $L(\hat{y}, y)$ とする。
隠れ層について、

- 隠れ層への入力: $\mathbf{x}$
- 重み行列、バイアス: $\mathbf{W}$ 、 $\mathbf{b}$
- 隠れ層の内部出力:
 $\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$
- 隠れ層の出力: $\mathbf{a} = h(\mathbf{z})$
- 活性化関数: h

出力層について、
(文字が重複しているので定義し直し)

- 出力層への入力: $\mathbf{a}$
- 出力層の重みとバイアス: $\mathbf{W}_{out}$ 、 $\mathbf{b}_{out}$
- 出力層の内部出力: $\mathbf{z}_{out}$

1. チェーンルールによる分解

$$\frac{\partial L}{\partial \mathbf{W}} = \frac{\partial L}{\partial \hat{\mathbf{y}}} \frac{\partial \hat{\mathbf{y}}}{\partial \mathbf{z}_{out}} \frac{\partial \mathbf{z}_{out}}{\partial \mathbf{a}} \frac{\partial \mathbf{a}}{\partial \mathbf{z}} \frac{\partial \mathbf{z}}{\partial \mathbf{W}}$$

$\mathbf{a}$ は出力層にとっては入力となります。それに注意すると次のように式展開できます

$$\begin{aligned} \frac{\partial L}{\partial \mathbf{W}} &= \frac{\partial L}{\partial \hat{\mathbf{y}}} \frac{\partial \hat{\mathbf{y}}}{\partial \mathbf{z}_{out}} \frac{\partial \mathbf{z}_{out}}{\partial \mathbf{a}} \frac{\partial \mathbf{a}}{\partial \mathbf{z}} \frac{\partial \mathbf{z}}{\partial \mathbf{W}} \\ &= \left(\frac{\partial L}{\partial \hat{\mathbf{y}}} \frac{\partial \hat{\mathbf{y}}}{\partial \mathbf{z}_{out}} \frac{\partial \mathbf{z}_{out}}{\partial \mathbf{a}} \right) \frac{\partial \mathbf{a}}{\partial \mathbf{z}} \frac{\partial \mathbf{z}}{\partial \mathbf{W}} \\ &= \left(\delta_{out} \mathbf{W}_{out}^T \right) \frac{\partial \mathbf{a}}{\partial \mathbf{z}} \frac{\partial \mathbf{z}}{\partial \mathbf{W}} \end{aligned}$$

$\left(\delta_{out} \mathbf{W}_{out}^T \right)$ は出力層の解説で述べたものです
実装的にも出力層の処理の時に算出しておき、隠れ層の処理に引き継ぐようにします

i この辺、成分表示に頼ると逆に見通しが悪くなります

微分式の見直し(隠れ層) #2

2. $\frac{\partial \mathbf{a}}{\partial \mathbf{z}}$

$\frac{\partial \mathbf{a}}{\partial \mathbf{z}} = h'(\mathbf{z})$ は隠れ層の活性化関数の微分です

結果はヤコビ行列になりますが、**活性化関数が要素単位で独立している場合は対角行列**になります

具体的には $a_i = h(z_i)$ の形をしている場合、

$$\frac{\partial a_i}{\partial z_j} = \begin{cases} h'(z_j) & , i = j \\ 0 & , i \neq j \end{cases}$$

ReLU, Sigmoid, tanh などにはこれに該当します

3. $\frac{\partial L}{\partial \mathbf{z}}$

$$\frac{\partial L}{\partial \mathbf{z}} = \frac{\partial L}{\partial \mathbf{a}} \frac{\partial \mathbf{a}}{\partial \mathbf{z}} \text{ を求めます。}$$

$\frac{\partial L}{\partial \mathbf{a}}$ はベクトル、 **$\frac{\partial \mathbf{a}}{\partial \mathbf{z}}$ は対角行列**です

続き

$$\frac{\partial L}{\partial \mathbf{a}} = [L'_1, L'_2, \dots, L'_n]$$

$$\frac{\partial \mathbf{a}}{\partial \mathbf{z}} = \begin{pmatrix} h'_1 & 0 & \dots & 0 \\ 0 & h'_2 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & h'_n \end{pmatrix}$$

$$\begin{aligned} \frac{\partial L}{\partial \mathbf{a}} \frac{\partial \mathbf{a}}{\partial \mathbf{z}} &= [L'_1 h'_1, L'_2 h'_2, \dots, L'_n h'_n] \\ &= [L'_1, L'_2, \dots, L'_n] \odot [h'_1, h'_2, \dots, h'_n] \end{aligned}$$

実装は上記のアダマール積の形になります
(対角行列の場合のみ成立します)

✎ アダマール積とは

行列の対応する成分をそのまま掛け合わせる演算です
 $X = A \odot B$ ならば $X_{ij} = A_{ij} B_{ij}$ となります

微分式の見直し(隠れ層) #3

4. $\partial z / \partial \mathbf{W}$

$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$ を $\mathbf{W}$ で微分します

$$\frac{\partial \mathbf{z}}{\partial \mathbf{W}} = \frac{\partial (\mathbf{W}\mathbf{x})}{\partial \mathbf{W}} = \mathbf{x}^T$$

5. 出力層の勾配 $\partial L / \partial \mathbf{W}$

以上より $\frac{\partial L}{\partial \mathbf{W}} = \frac{\partial L}{\partial a} \frac{\partial a}{\partial z} \frac{\partial z}{\partial \mathbf{W}}$
について

$$\frac{\partial L}{\partial \mathbf{W}} = \begin{pmatrix} L'_1 \\ L'_2 \\ \vdots \\ L'_n \end{pmatrix} \odot \begin{pmatrix} h'_1 \\ h'_2 \\ \vdots \\ h'_n \end{pmatrix} \mathbf{x}^T$$

6. δ の導入

出力層と同様に δ を導入します

$$\frac{\partial L}{\partial \mathbf{W}} = \delta \mathbf{x}^T \quad \left(\delta = \frac{\partial L}{\partial a} \frac{\partial a}{\partial z} = \frac{\partial L}{\partial z} \right)$$

これも出力層と同様、他の微分項を求められます

☞ 「同様」というのは偶然ではなく、意図した設計です

$$\begin{aligned} \frac{\partial L}{\partial \mathbf{b}} &= \frac{\partial L}{\partial a} \frac{\partial a}{\partial z} \frac{\partial z}{\partial \mathbf{b}} = \delta \mathbf{1}^T \\ \frac{\partial L}{\partial \mathbf{x}} &= \frac{\partial L}{\partial a} \frac{\partial a}{\partial z} \frac{\partial z}{\partial \mathbf{x}} = \delta \mathbf{W}^T \end{aligned}$$

✎ $\delta \mathbf{W}^T$ は更に前の層に渡され同じ処理が行われます

✓ 理論側の説明はこれで完了です。次ページで実装します

誤差逆伝播法(実装)

MyLinearLayer#backward

d.py

```
def backward(self, dLdA, lr=1e-2, L2=0):
    """ 誤差逆伝播法でパラメータ更新 """
    # 項を求める
    dLdA * self.activation_prime(self.Z) #  $\delta$ 
    self.X.T @ dLdZ # Wの更新用
    np.sum(dLdZ, axis=0, keepdims=True) # bの更新用

    prev = dLdZ @ self.W.T # 次の層へ引き継ぐ微分値
    if prev != 0: # L2正則化項を追加
        dW += 2 * L2 * self.W

    # 下法を実行
    dW -= lr * dLdW # 重みパラメータ更新
    db -= lr * dLdb # バイアス更新
    dLdA_prev = dLdA # (理論的に)前の層へ伝播、(実装的には次の層)
```

でそのままです。以下補足

横は実装にあわせて変更してます

出力層に於いては `dLdA` 単体で `dLdZ` 相当です

出力層では `activation_prime` はおまけです

2関係は後述するので置いといてください

MyNetwork#backward

各層の `backward` を順番に呼び出しています。これによりメータ更新が完了します

04-backward.py

```
def backward(self, X, Y, lr=1e-2, L2=0):
    """ 誤差逆伝播法でパラメータ更新 """
    Y_pred = self.forward(X) # 順伝播
    dLdA = self.loss_prime(Y_pred, Y) # 出力層の損失
    for layer in reversed(self.layers): # 出力層から
        dLdA = layer.backward(dLdA, lr, L2) # パラメータ更新
    return dLdA
```

各関数の微分(誤差逆伝播用)

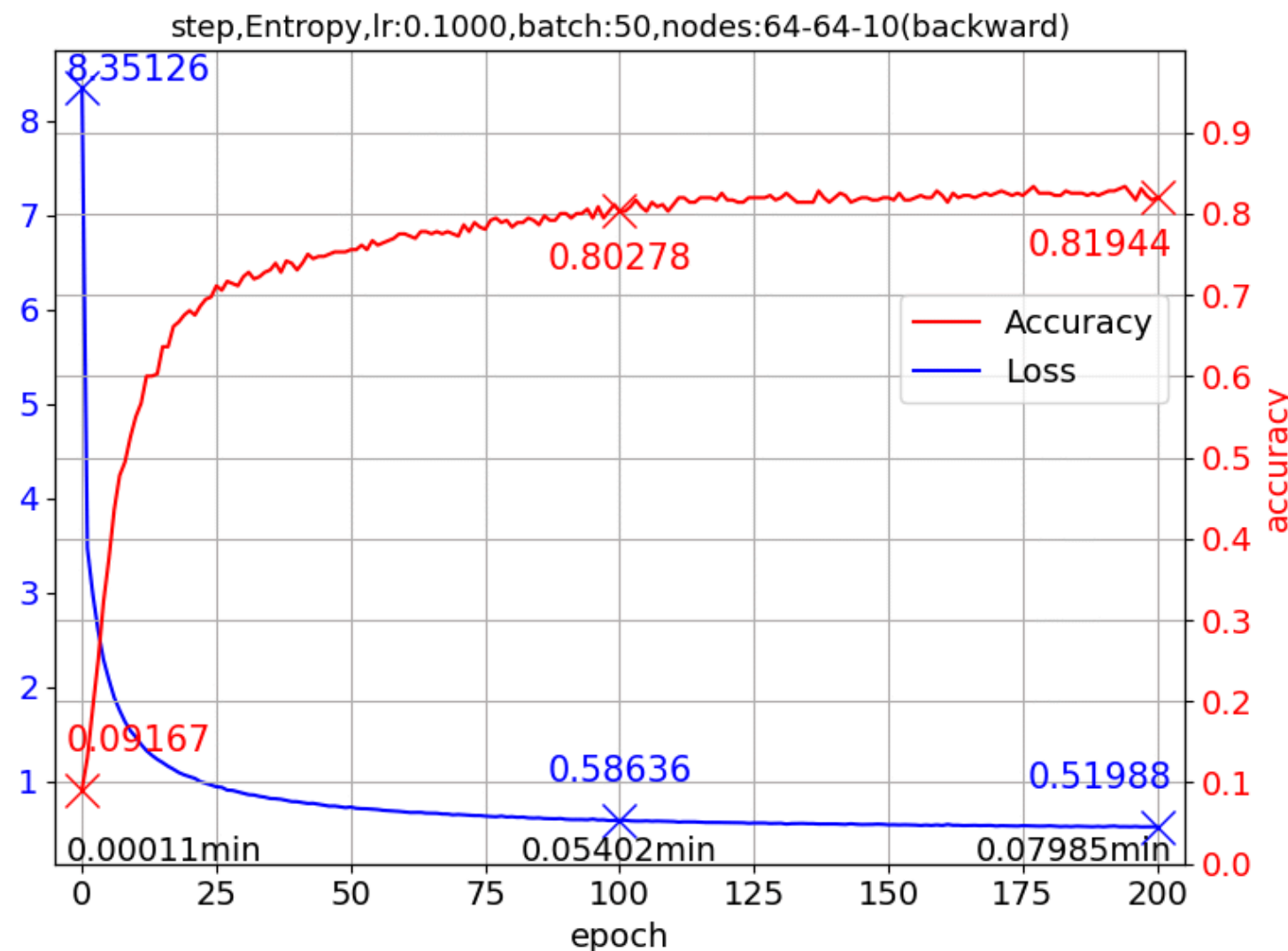
04-backward.py

```
def loss_cross_entropy_prime(Y_pred, Y):
    # one-hot前提で、Y_pred: (B, C), Y: (B, C)
    Y = one_hot_batch(Y) # 正解をワンホットベクトル化
    return (Y_pred - Y) / Y_pred.shape[0] # バッチ平均

def softmax_prime(z): # 簡易：損失関数が交差エントロピー
    return np.ones_like(z)
```


誤差逆伝播法(実行結果)

グラフとログ



```
step,Entropy,lr:0.1000,batch:50,nodes:64-64-10(backward)
Loss: 8.3512616600, Accuracy: 0.09167, Time: 0.00011[min]
epoch: 1 → Loss: 3.4700833541, Accuracy: 0.13056, Time: 0.00032[min]
epoch: 2 → Loss: 2.9877415189, Accuracy: 0.19167, Time: 0.00052[min]
<省略>
epoch: 198 → Loss: 0.5189589011, Accuracy: 0.82222, Time: 0.07947[min]
epoch: 199 → Loss: 0.5200432732, Accuracy: 0.81667, Time: 0.07966[min]
epoch: 200 → Loss: 0.5198817405, Accuracy: 0.81944, Time: 0.07985[min]
```

結果

- 損失値
(0.51→0.51)
 - ほぼ同じ
- 正解率
(81.9%→81.9%)
 - 同じ
- 時間(100エポック:
56分→3秒)
 - トータル5秒、激減

評価

- 損失値と正解率がほぼ同じなのはOK (実装は異なるが理論ベースは同じ)
- 計算時間が激減した。約1000分の1

自動微分と計算グラフ

自動微分の位置づけ

自動微分は理論と実装をつなぐ仕組みです

理論ベースでは誤差逆伝播法の本質はチェーンルールによる分解(とその結果としての計算量削減)です

実装ベースでは求めたいのは $\frac{\partial L}{\partial \mathbf{bm}_w}$ の値です

しかしモデル(数式)毎に各微分項を導出し実装するのは手間です。そこを自動化したのが自動微分です

考え方(具体例)

$$L = \log((x + y)^2)$$

について

- $a = x + y$ (加算)
- $b = a^2$ (2乗、pow)

(i)は順伝播に相当し

```
def forward(x, y):
    return np.log((x+y)^2)
```

のように定義すると内部で自動的に分解されます(分解=合成関数とし

続き

(i)の各式の微分(ii)は

- $\frac{dL}{db} = \frac{1}{b}$
- $\frac{da}{db} = 2a$
- $\frac{dx}{da} = 1$

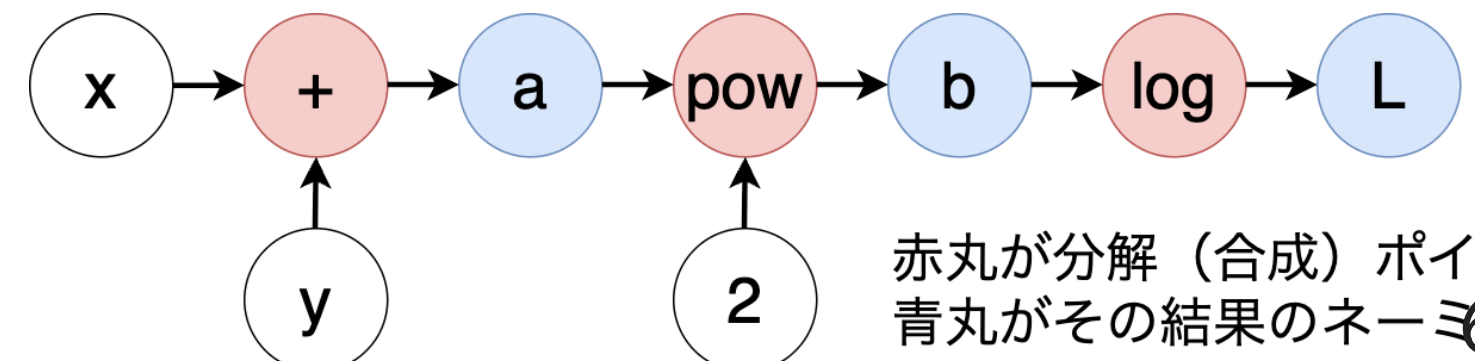
(i)の各式に対応した微分関数(ii)は予め用意されており、これも内部で自動的に割り当てられます

dL/dx を求める際は自動で下記が実行されます

$$\frac{dL}{dx} = \frac{1}{b} \cdot 2a \cdot 1$$

計算グラフ

計算グラフは自動微分を図示したものです



赤丸が分解(合成)ポイント
青丸がその結果のネー

誤差逆伝播法の説明はここまでです。簡易まとめ

理論的理解

- 位置付け: NNの勾配降下法における微分値計算
- 選択理由: (数値微分に比べ)計算量が削減できる
- 真髄: チェーンルールを応用して重複部分を削減

実装寄りの理解

- 自動微分: チェーンルールの分解と計算が自動
- 計算グラフ: 上記の視覚化

§ 9. 活性化関数の変更 & 学習率調整

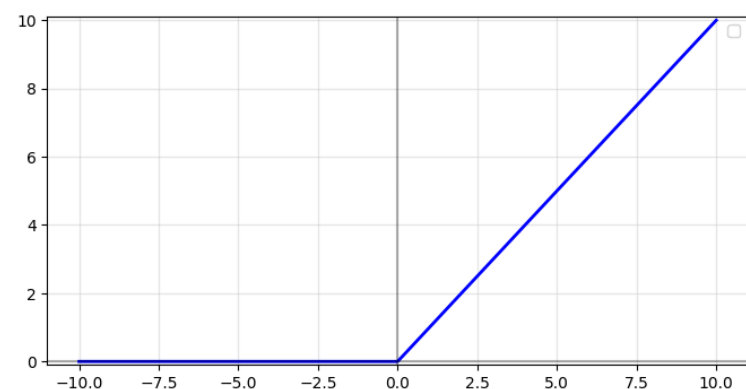
ReLU関数

背景

階段関数は x の値によらず微分値が常に0です。そのため、実は**隠れ層は全く学習してませんでした**(パラメータ更新なし)。モデル全体で学習してたのは出力層のみでした

よって隠れ層も学習するよう活性化関数を変更します

ReLU関数



ReLU関数は左図のような関数です

数式は次の通りです

$$y = \begin{cases} 0 & , x \leq 0 \\ x & , x > 0 \end{cases}$$

階段関数に対する利点ですが、単純に $x > 0$ の場合に微分値が1になるという点です(学習できます)

実装

`relu` はReLU関数本体

```
def relu(x):  
    return np.maximum(0, x)
```

`relu_prime` は微分。理論上 $x=0$ は微分不可ですが実装的には気にせず適当に0か1を返せばOKです

```
def relu_prime(z):  
    return (z > 0).astype(float)
```

学習実行部分は関数名を変えるだけです

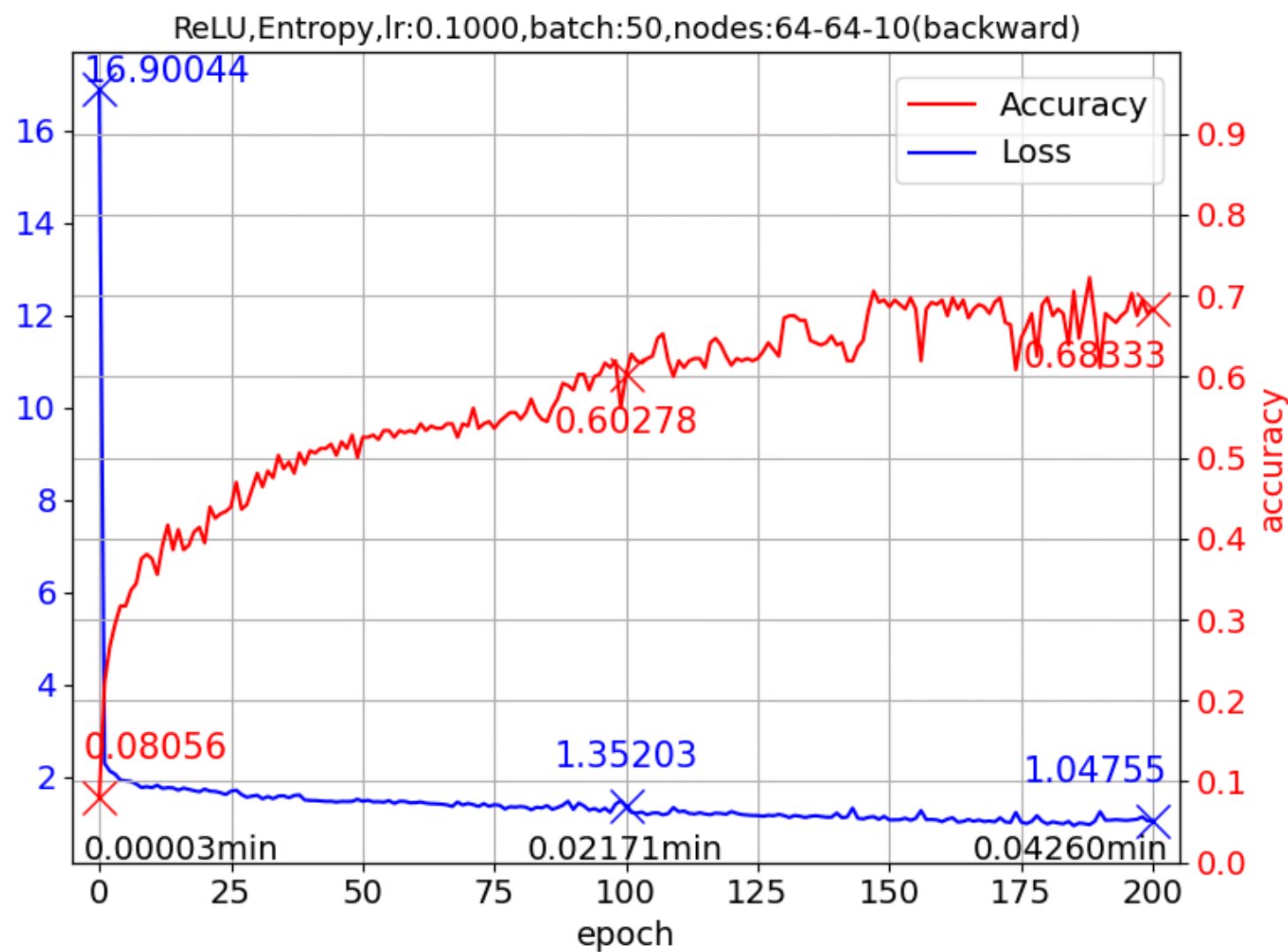
```
activation = 'ReLU'
```

関数マップにも登録しておきます

```
funcs_map = {  
    # <省略>  
    'ReLU': (relu, relu_prime),  
}
```

ReLU関数(実行結果)

グラフとログ



```
ReLU,Entropy,lr:0.1000,batch:50,nodes:64-64-10(backward)
Loss: 16.9004414256, Accuracy: 0.08056, Time: 0.00003[min]
epoch: 1 → Loss: 2.2860905147, Accuracy: 0.22222, Time: 0.00025[min]
epoch: 2 → Loss: 2.1199022689, Accuracy: 0.26667, Time: 0.00043[min]
<省略>
epoch: 198 → Loss: 1.1264676740, Accuracy: 0.69444, Time: 0.04223[min]
epoch: 199 → Loss: 1.0391083747, Accuracy: 0.67778, Time: 0.04242[min]
epoch: 200 → Loss: 1.0475498359, Accuracy: 0.68333, Time: 0.04260[min]
```

結果

- 損失値
(0.51→1.04)
 - 関数を変えたため
- **正解率**
(81.9%→68.3%)
 - 減少
 - 不安定(ギザギザ)
- 時間(100エポック:
3秒→1.2秒)
 - トータル2.5秒、やや減。誤差レベル

評価&方針

- **隠れ層も学習してるのに正解率が減少した**
→次ページで説明します

学習率の調整

学習率とは

学習率は勾配降下法の更新式に現れる η です

$$\mathbf{W}^{(t+1)} := \mathbf{W}^{(t)} - \eta \nabla L(\mathbf{W}^{(t)})$$

更新の方向(±)は ∇L で決定されますが、更新量は学習率 η によって大小を調整できます

- 小さすぎ→学習が遅い、進まない
- 大きすぎ→学習状況が不安定になる

いずれも損失値や正解率が収束しないという状態に陥ります

学習率の適正值

学習率の適正值は使用する関数やモデルの構成によって異なります

これらを変更した際、学習率の調整が必要となる場合があります

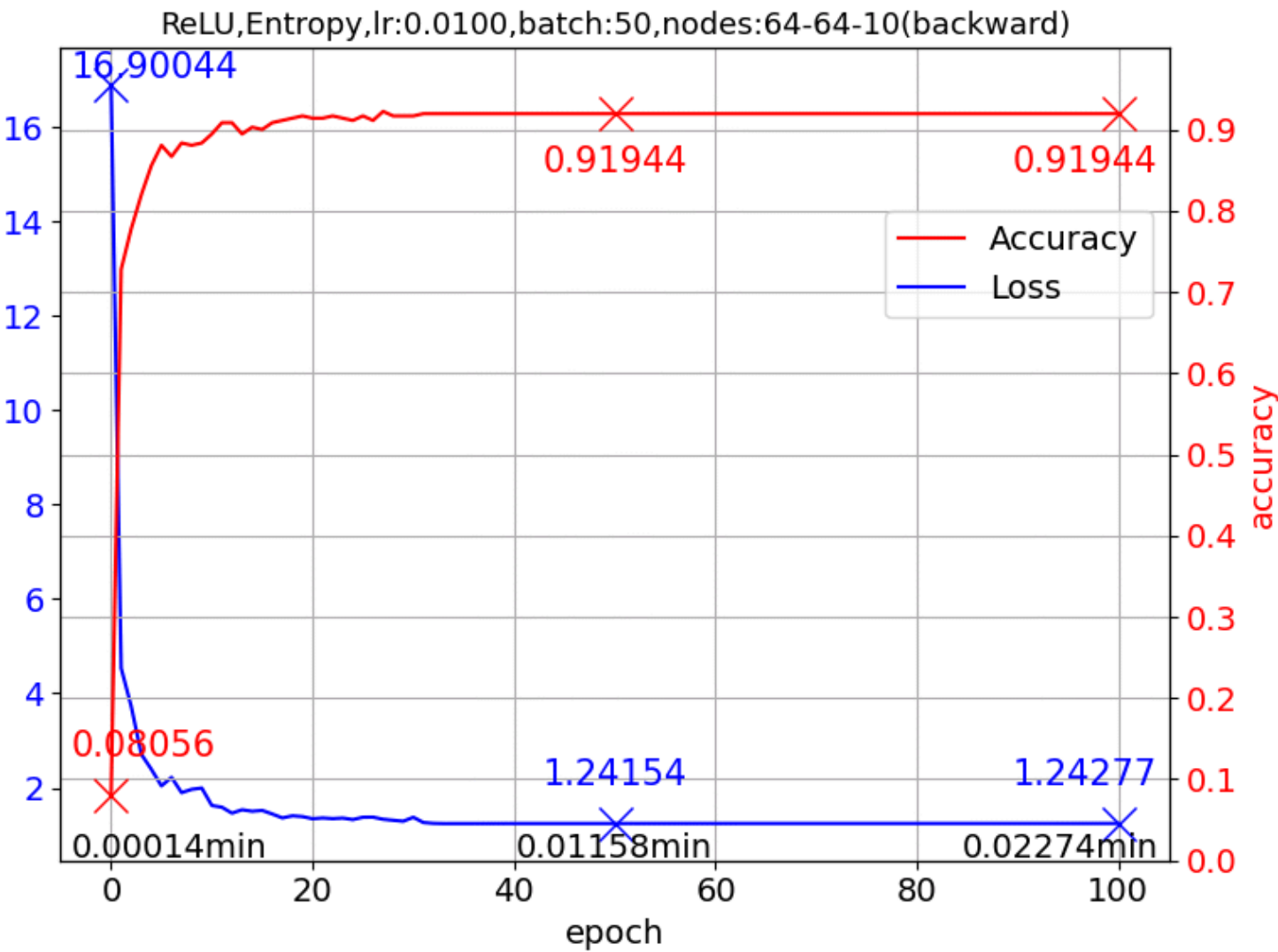
コード

学習率を変更します。他はそのままOKです

```
lr = 1e-2 # 前は1e-1
```

学習率=1e-2(実行結果)

グラフとログ



```
ReLU,Entropy,lr:0.0100,batch:50,nodes:64-64-10(backward)
Loss: 16.9004414256, Accuracy: 0.08056, Time: 0.00014[min]
epoch: 1 → Loss: 4.5367412226, Accuracy: 0.72778, Time: 0.00035[min]
epoch: 2 → Loss: 3.7252007072, Accuracy: 0.77778, Time: 0.00056[min]
<省略>
epoch: 98 → Loss: 1.2427389287, Accuracy: 0.91944, Time: 0.02235[min]
epoch: 99 → Loss: 1.2427537009, Accuracy: 0.91944, Time: 0.02255[min]
epoch: 100 → Loss: 1.2427682673, Accuracy: 0.91944, Time: 0.02274[min]
```

結果

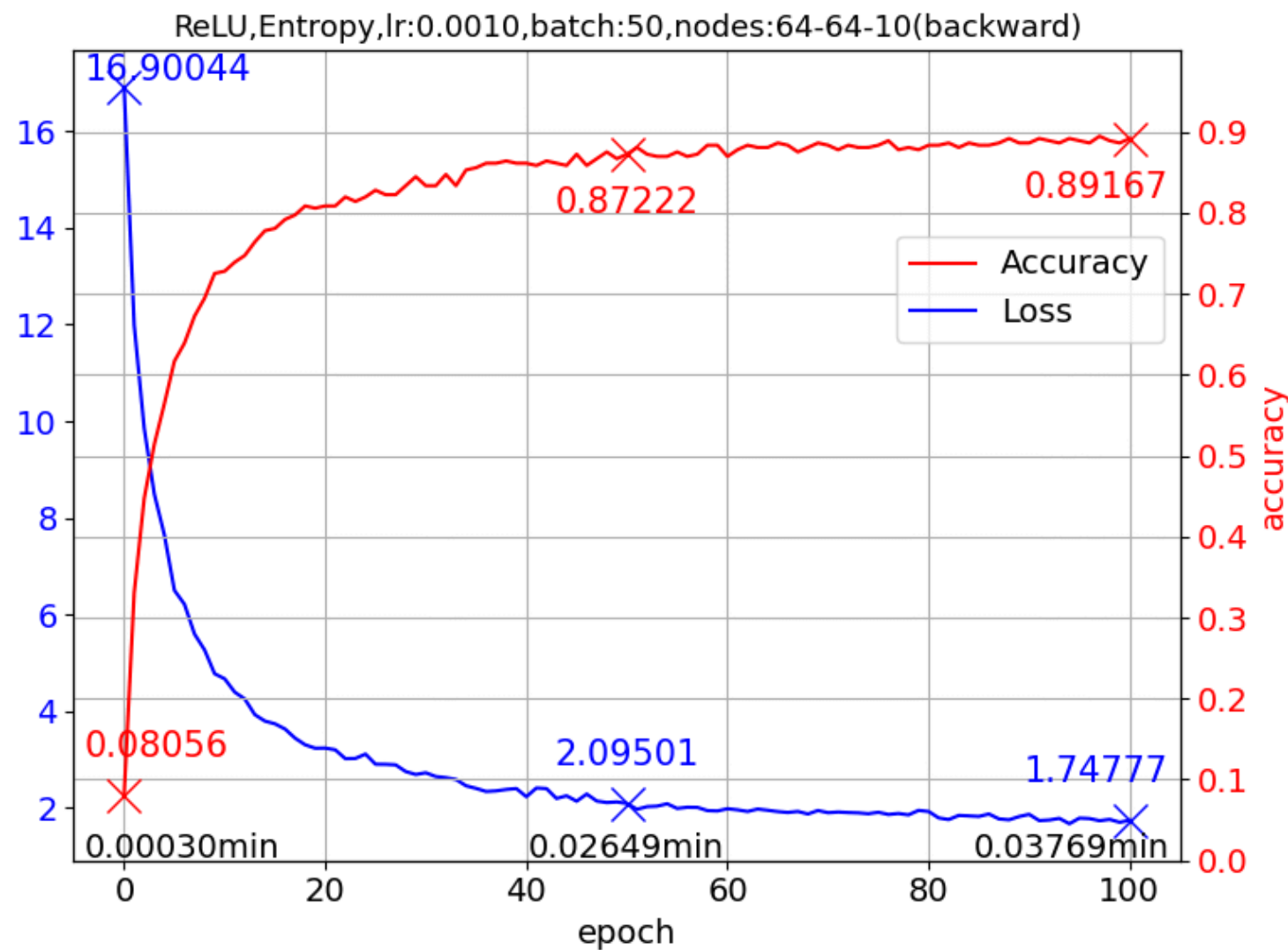
- 損失値
(1.04→1.24)
 - 上がってる
 - 収束はしている
- 正解率
(68.3%→91.9%)
 - 上がっている
- 時間(100エポック:
1.2秒→1.3秒)
 - 誤差
 - トータルエポック
数は減らしてます

評価

- 学習率の調整により
正解率が向上した

学習率=1e-3(実行結果、参考)

グラフとログ



```
ReLU,Entropy,lr:0.0010,batch:50,nodes:64-64-10(backward)
Loss: 16.9004414256, Accuracy: 0.08056, Time: 0.00030[min]
epoch: 1 → Loss: 11.9864697692, Accuracy: 0.33056, Time: 0.00053[min]
epoch: 2 → Loss: 9.8516214295, Accuracy: 0.44722, Time: 0.00084[min]
<省略>
epoch: 98 → Loss: 1.7651122357, Accuracy: 0.88889, Time: 0.03730[min]
epoch: 99 → Loss: 1.7058314431, Accuracy: 0.88611, Time: 0.03750[min]
epoch: 100 → Loss: 1.7477719293, Accuracy: 0.89167, Time: 0.03769[min]
```

結果

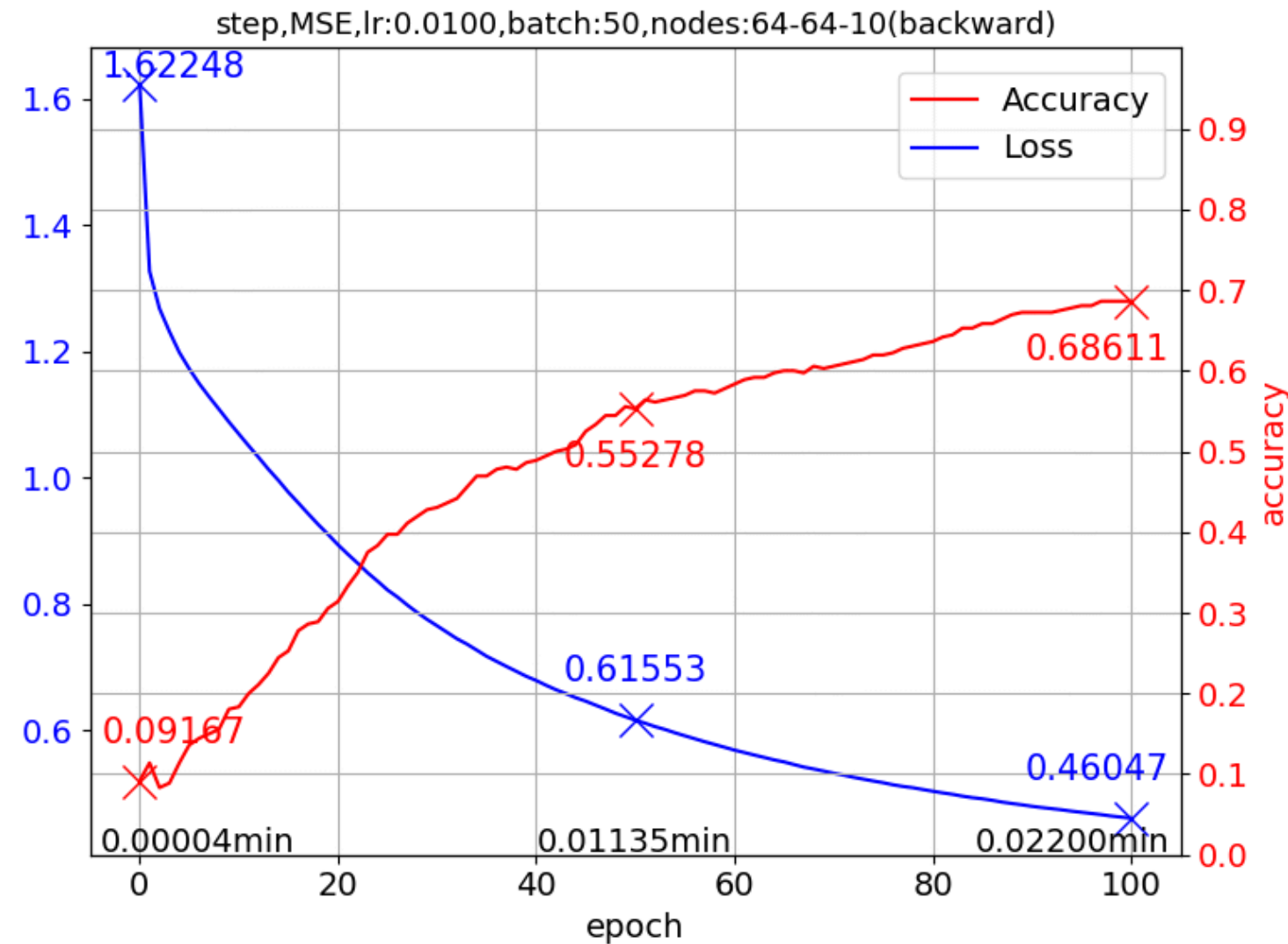
- 損失値
(1.24→1.24)
 - ほぼ同じ
- 正解率
(91.9%→89.1%)
 - 下がっているがまだ収束していない
 - 立ち上がりが遅くなった
 - 少し揺らいでる(不安定)
- 時間(100エポック:1.3秒→2.2秒)
 - 誤差レベル

評価

- 学習率をより減らしたら悪化した73/101

step関数で学習率変更(参考)

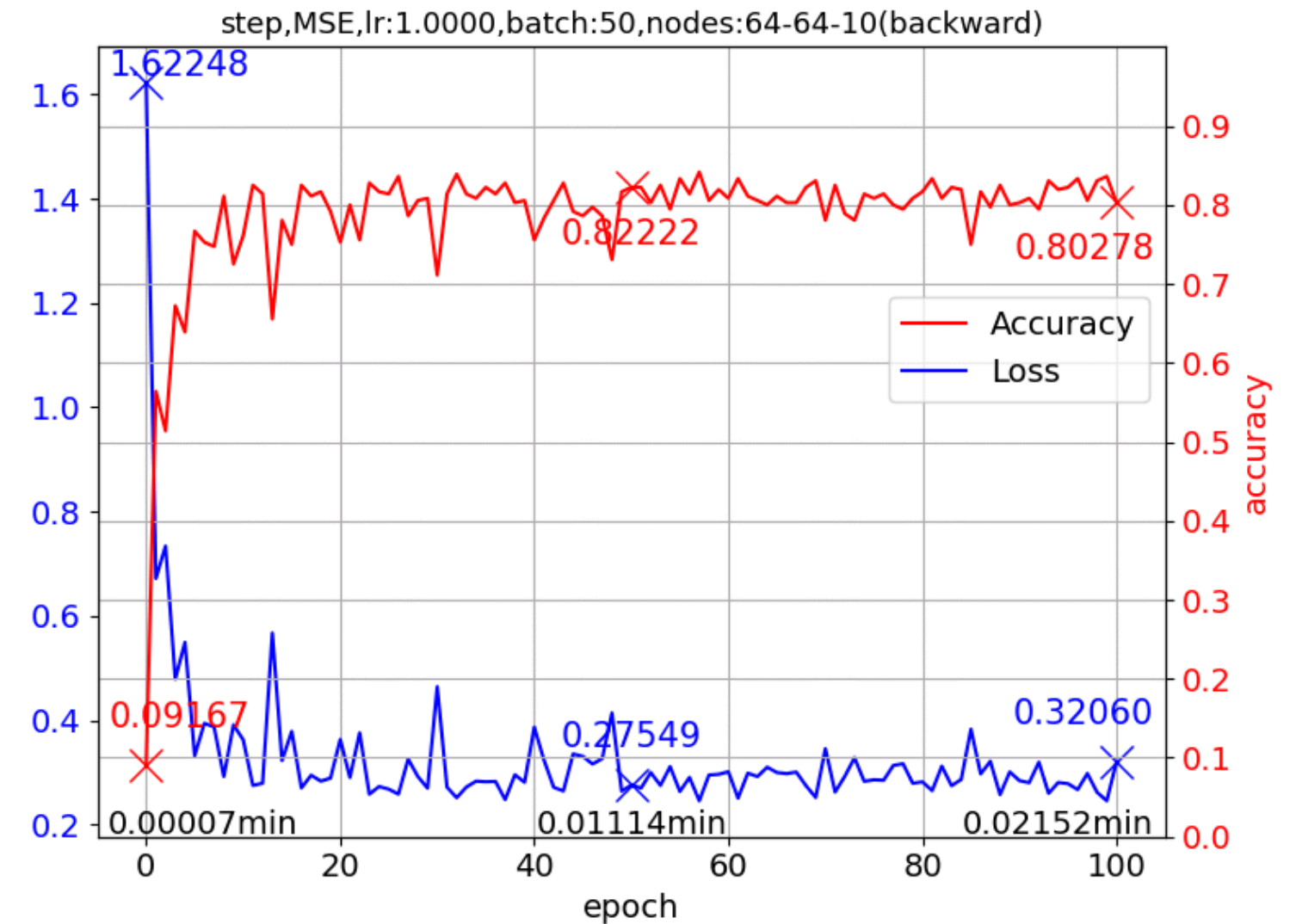
学習率 $1e-1 \rightarrow 1e-2$



- 損失値(0.51→0.46)
- 正解率(81.9%→68.6%)
- 明らかに収束していない

学習率を下げた結果、学習速度が低下した結果、収束せず(最終的な収束値は未確認)

学習率 $1e-1 \rightarrow 1$



- 損失値(0.51→0.32)
- 正解率(81.9%→80.2%)
- ギザギザ→不安定

学習率を上げた結果、学習が不安定になった

§ 10. 更なる改善手法の紹介

基本の改善手法(ミニバッチ学習、誤差逆伝播法)については説明完了です
ここからは更なる改善手法を紹介します

Xavier初期化とHe初期化

Xavier初期化とは

Xavier初期化とは重み W の初期値に $\sqrt{\frac{2}{n_{in} + n_{out}}}$ を掛けることです。 n_{in} は入力 x の次数で、 n_{out} は出力 a の次数です。直感的には次元によるノルム増加の補正で、次元数として入出力の次数の平均を用いてるイメージです。*Sigmoid* 系関数に使われます

He初期化とは

He初期化とは主に *ReLU* 系関数に使われており、重み W の初期値に $\sqrt{2}$ を掛けます。 n_{in} は入力 x の次数で、次元によるノルムの増加を抑えています。 $\sqrt{2}$ は *ReLU* 関数が $x < 0$ で $y = 0$ となるため実質的な重みの量が半分になることからの補正

実装

```
MyLinearLayer#__init__()
```

```
self.W = np.random.randn(input_size, output_size)
if he:
    self.W *= np.sqrt(2 / input_size) # He初期化
self.b = np.array([np.zeros(output_size)])
```

`he=True` の時に重み行列 `W` に対してHe初期化を行います。
バイアス `b` は変更ありません

実行部分

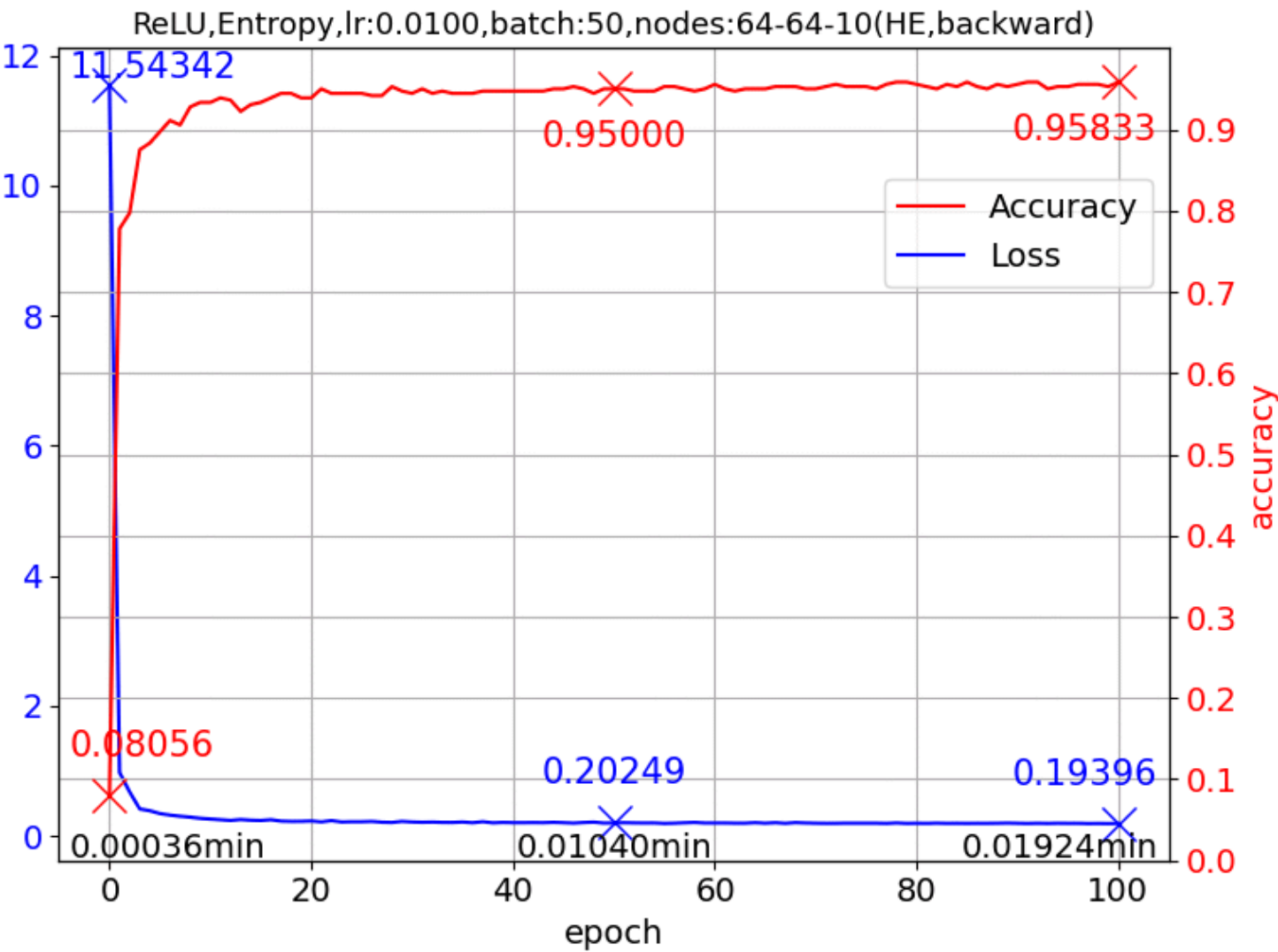
```
he = True # Falseだと実行しない（前のまま）
```

閑話

Xavier初期化もHe初期化もどちらも結局は重みパラメータの分散を一定の範囲に縮めることが目的です。（分散が大きすぎると学習の精度や効率が低下するため、その抑制です）

He初期化(実行結果)

グラフ



ReLU,Entropy,lr:0.0100,batch:50,nodes:64-64-10(HE,backward)
Loss: 11.5434236396, Accuracy: 0.08056, Time: 0.00036[min]
epoch: 1 → Loss: 0.9770382673, Accuracy: 0.77778, Time: 0.00128[min]
epoch: 2 → Loss: 0.6785440797, Accuracy: 0.79722, Time: 0.00150[min]
<省略>
epoch: 98 → Loss: 0.1938802557, Accuracy: 0.95556, Time: 0.01882[min]
epoch: 99 → Loss: 0.1925929632, Accuracy: 0.95278, Time: 0.01901[min]
epoch: 100 → Loss: 0.1939612528, Accuracy: 0.95833, Time: 0.01924[min]

結果

- 損失値
(1.24→0.19)
 - 初期値変更の為、あまり参考にならず
- **正解率**
(91.9%→95.8%)
 - 上がっている
- 時間(100エポック:
1.3秒→1.2秒)
 - 誤差レベル

評価

- He初期化により正解率が向上した
 - この段階からの4%は大きいです

数式の言語的解釈(みたいな例)

伝えたいこと

さきほど $\sqrt{n}$ を「長さの次元補正」と解釈しましたが
どこからそんな視点が生まれるかというと、
個人的には以下のように数式を言語的に捉えています
(あくまでイメージ。雰囲気や文脈にもよります)
データ $X = [x_1, x_2, \dots, x_n]$ があるとします
 Z を何らかの数値とします(何の数値かは不問)

n の割り算

- $\frac{Z}{n}$ → 「平均化」「標準化」「1つあたり」かな？
- $\frac{Z}{\sqrt{n}}$ → 「次元の長さ補正」「分散の補正」かな？
- $\frac{Z}{n^2}$ → 「密度化」「総当たり絡んでる？」

$S = \sum x_i$ の割り算

$S = \sum_{i=1}^n x_i = x_1 + x_2 + \dots + x_n$ とします
• $\frac{Z}{S}$ → 「割合化」「正規化」「確率化」かな？

$R^2 = \sum x_i^2$ の割り算

$R^2 = \sum_{i=1}^n x_i^2 = x_1^2 + x_2^2 + \dots + x_n^2$ とします
• $\frac{Z}{R}$ → 「単位化」「方向に着目」かな？
• $\frac{Z}{R^2}$ → 「逆二乗則」かな？(光・重力的あつかい)

以上はあくまで参考で

数式を単に計算と見るのではなく、何を意図している
かという解釈も大切ですよというお話でした

おまけ $F = k \frac{qQ}{R^2} = qE \rightarrow$ 粒子の作用から場の概念へ

L2正則化

L2正則化とは

L2正則化とは損失値に $\lambda \sum W^2$ を加えることです $\sum W^2$ は各層の重みパラメータ w_{ij} の二乗和、 λ は係数 (ハイパーパラメータ) です

L2正則化の意図

パラメータ単体の大きさを小さく保つ事です

仮に損失値が小さくなったとしても、パラメータ間の大小が極端に開く事は好ましくなく、そのため、パラメータ間の差を小さく保ったまま損失値全体を小さくするという動作を意図している

実装

`MyNetwork#loss()` → L2正則化を加算してます
(`loss_value += L2 * np.sum(layer.W**2)`)

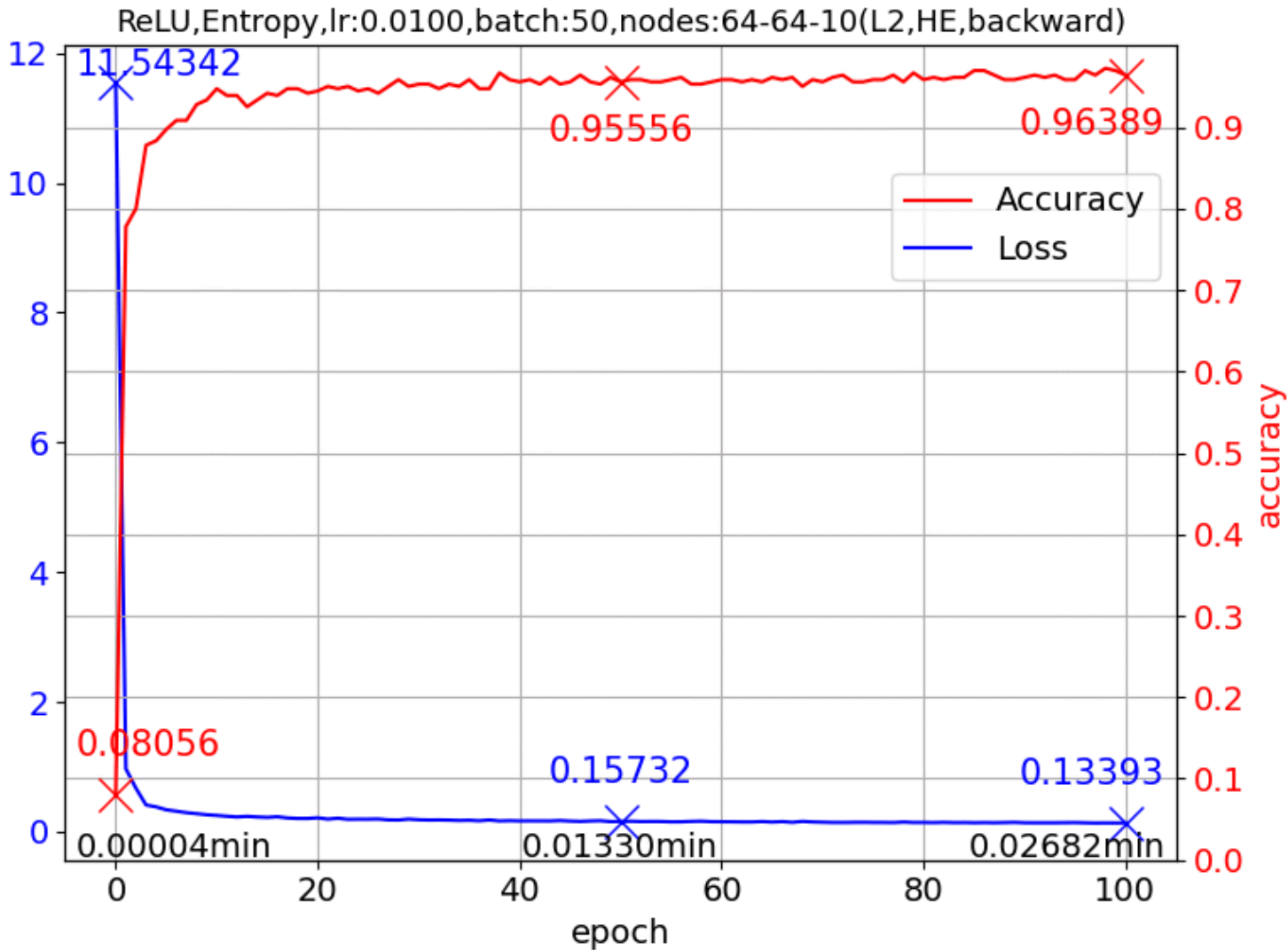
```
def loss(self, X, Y, L2=0):  
    """ 損失関数 """  
    Y_pred = self.forward(X)  
    loss_value = self.loss_fn(Y_pred, Y)  
    if L2 != 0: # L2正則化  
        for layer in self.layers: # 各線形層ごとに処理  
            loss_value += L2 * np.sum(layer.W**2) # 対象はWのみ  
    return loss_value
```

`MyLinearLayer#backward()` → 微分値を加算
(`dLdW += 2 * L2 * self.W`)

```
def backward(self, dLdA, lr=1e-2, L2=0):  
    """ 誤差逆伝播法でパラメータ更新 """  
    # <中略>  
    dLdA_prev = dLdZ @ self.W.T  
    if L2 != 0: # L2正則化項を追加  
        dLdW += 2 * L2 * self.W
```


L2正則化(実行結果)

グラフとログ



```
ReLU,Entropy,lr:0.0100,batch:50,nodes:64-64-10(L2,HE,backward)
Loss: 11.5434236396, Accuracy: 0.08056, Time: 0.00004[min]
epoch: 1 → Loss: 0.9688728989, Accuracy: 0.77778, Time: 0.00032[min]
epoch: 2 → Loss: 0.6681328541, Accuracy: 0.80000, Time: 0.00057[min]
<省略>
epoch: 98 → Loss: 0.1343407657, Accuracy: 0.97222, Time: 0.02631[min]
epoch: 99 → Loss: 0.1337073587, Accuracy: 0.96944, Time: 0.02656[min]
epoch: 100 → Loss: 0.1339324919, Accuracy: 0.96389, Time: 0.02682[min]
```

結果

- 損失値
(0.19→0.13)
 - 減少
- 正解率
(95.8%→96.3%)
 - 上がっている
- 時間(100エポック:
1.2秒→1.5秒)
 - ほぼ同じ

評価

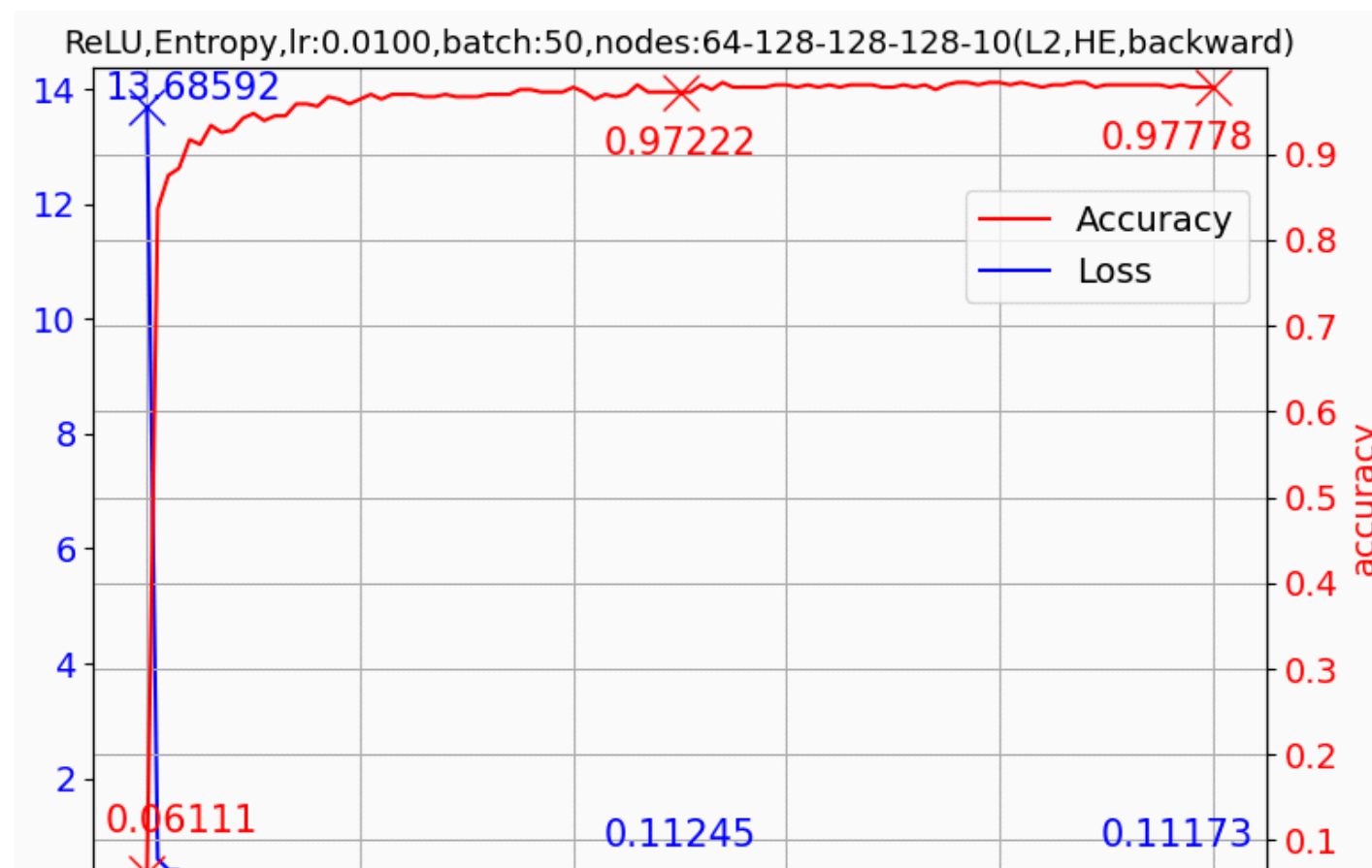
- L2正則化により正解率が向上した

4層化&ノード増加

層とノードを増やします

```
self.layers = [ # MyNetwork#__init__()  
    MyLinearLayer(input_size, hidden_size1, activation=activation),  
    MyLinearLayer(hidden_size1, hidden_size2, activation=activation),  
    MyLinearLayer(hidden_size2, hidden_size3, activation=activation),  
    MyLinearLayer(hidden_size3, output_size, activation='softmax'),  
]  
  
hidden_size1, hidden_size2, hidden_size3 = 128, 128, 128
```

グラフ(ログは省略)



結果

- 損失値
(0.13→0.11)
 - 減少
- 正解率
(96.3%→97.7%)
 - 上がっている
- 時間(100エポック:
1.5秒→7.2秒)
 - 上がっている

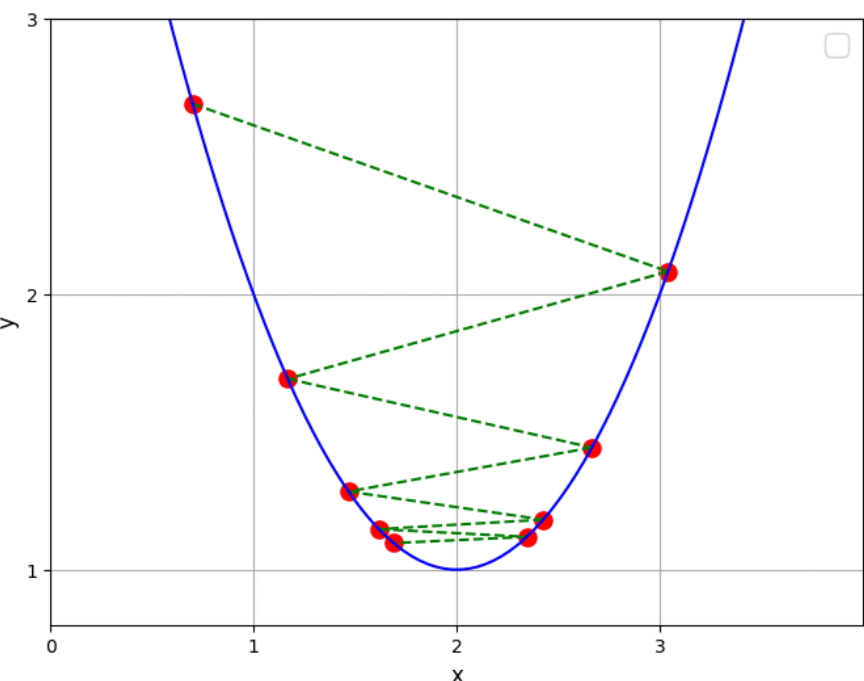
評価

- ノード数増加により
正解率が向上した
- 時間が増加した
 - 総パラメータ数が増えたので順当

ていげん

学習率逡減

学習率を小さくしていく



学習率が一定値の場合、左図のように最小値周りを行ったり来たりする状態に陥る場合があります
($x = 2$ 近辺)



コード(簡易実装)

最初の `global lr` と最下行が追加部分

```
def train_minibatch(X, Y):  
    global lr # スコープ対策  
    # エポック毎にループ  
    for i in range(max_epoch):  
        # <省略>  
        # バッチサイズ単位でループ  
        for j in range(0, num, batch_size):  
            # <省略>  
            lr *= lr_decay if (i+1) % decay_step == 0 else 1 # 学習率の更新
```

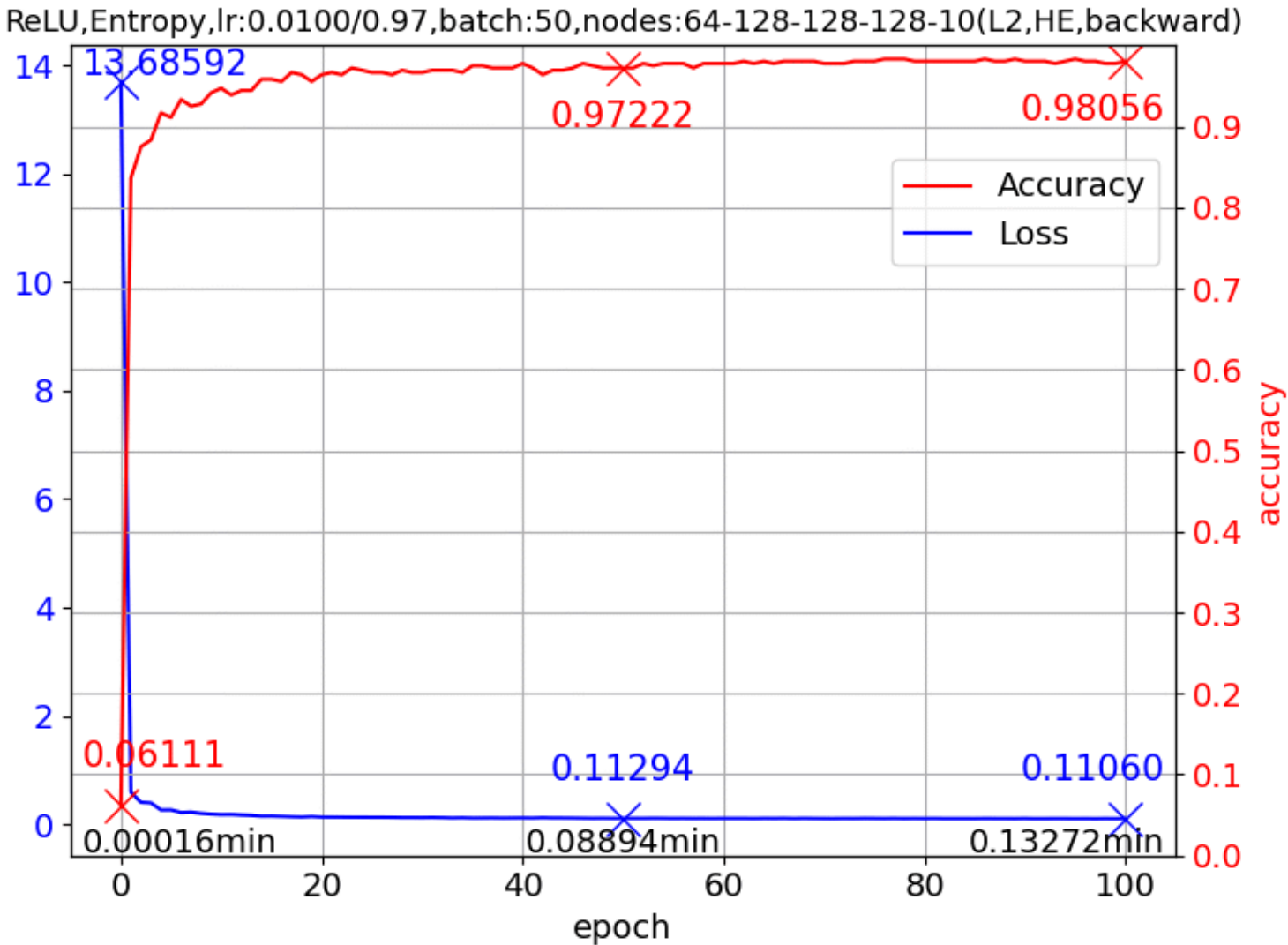
実行コード

```
lr_decay = 0.97 # これを1にすると逡減は無効  
decay_step = 10 # 10エポックごとに学習率を更新
```

② 逡減の意味
「逡減」は「だんだんと減らすこと」という意の日本語です
「減衰」や「調整」より適した表現と個人的には思ってます

学習率低減(実行結果)

グラフとログ



```
ReLU,Entropy,lr:0.0100,batch:50,nodes:64-64-10(L2,HE,backward)
Loss: 11.5434236396, Accuracy: 0.08056, Time: 0.00004[min]
epoch: 1 → Loss: 0.9688728989, Accuracy: 0.77778, Time: 0.00032[min]
epoch: 2 → Loss: 0.6681328541, Accuracy: 0.80000, Time: 0.00057[min]
<省略>
epoch: 98 → Loss: 0.1343407657, Accuracy: 0.97222, Time: 0.02631[min]
epoch: 99 → Loss: 0.1337073587, Accuracy: 0.96944, Time: 0.02656[min]
epoch: 100 → Loss: 0.1339324919, Accuracy: 0.96389, Time: 0.02682[min]
```

結果

- 損失値
(0.111→0.110)
 - ほぼ同じ
- 正解率
(97.7%→98.0%)
 - 僅かだけど、上がっている
- 時間(100エポック:7.2秒→6.6秒)
 - 誤差

評価

- 正解率は上がった
 - まあでも誤差レベルですが
 - 上昇幅は小さいけれど一応効果ありということも

§ 11. PyTorch

PyTorchとは

PyTorchとは

PyTorchは、Facebookによって開発された機械学習用のフレームワークです。ここまで解説したようなニューラルネットワークやその他の機械学習アルゴリズムの構築を行うためのAPIが揃っています
割と普及率が高いツールです

主な特徴は次のとおりです

- 研究者がよく使ってる(個人の感想)
 - 結構な論文がPyTorchで実装している
 - WEB解説でも結構多めです
- 動的計算グラフ
 - オリジナル数理モデルでも微分してくれる
- その他、GPU対応、音声・画像対応などなど

 他のライブラリもありますが何となく選んでます

なんで最初から使わないの？

実用上はPyTorchなどのライブラリを当然に使います。一から自分で実装することはないです

しかし今回のレクチャーでは、以下のような段階を踏んで学習を進めるため敢えてライブラリの紹介は最後としました

- ベース理論の理解
 - そまそまの基本理論を理解する
- コードレベルでの実装
 - 理論と実装の繋がりを体感する
 - 実装コードの動作を中身から理解する
- 既存ライブラリの利用
 - 学んだ理論と実装知識を土台として使いこなす

目指すのは内側からの理解です。まあ中身の理論を知らなくても使いこなせるかもですが、個人的にあまり好ましくないと感じています

PyTorchでの実装 #1

学習用データ準備

```
>
19)
device("cuda" if torch.cuda.is_available() else "cpu")

ts()
digits.data, digits.target
X_train, Y_train = \
    split(X_orig, Y_orig, test_size=0.2, random_state=19)

# PyTorch用に変換)
X_train = torch.tensor(X_train, dtype=torch.float32)
Y_train = torch.tensor(Y_train, dtype=torch.long)
X_test = torch.tensor(X_test, dtype=torch.float32).to(device)
Y_test = torch.tensor(Y_test, dtype=torch.long).to(device)

# ミニバッチ学習用に整形)
data_loader = torch.utils.data.DataLoader(TensorDataset(X_train, Y_train), \
                                           batch_size=50, shuffle=True)
```

的にPyTorchに合わせてます
 使用するコードです 後述します(今回は `cpu` を指定している)

ニューラルネットワーク

```
# ニューラルネットワーク定義
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.fc1 = nn.Linear(64, 128)
        self.fc2 = nn.Linear(128, 128)
        self.fc3 = nn.Linear(128, 128)
        self.fc4 = nn.Linear(128, 10)
        self._init_weights()

    def _init_weights(self):
        for m in self.modules():
            if isinstance(m, nn.Linear):
                nn.init.kaiming_normal_(m.weight,
                                       nonlinearity='relu')

    def forward(self, x):
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = F.relu(self.fc3(x))
        return self.fc4(x) # CrossEntropyLoss
```

`nn.Linear` は `MyLinearLayer` 相当です

`_init_weights` はHe初期化のためだけの処理です

PyTorchでの実装#2

学習準備

```

vice)
sEntropyLoss() # 損失関数=クロスエントロピー
ptim.SGD(model.parameters(), lr=0.01, \
decay=1e-4) # 最適化手法=SGD、L2の係数=1e-4
ptim.lr_scheduler.StepLR(optimizer, \
    step_size=5, gamma=0.95) # 学習率更新

```

riterion	は損失関数を指定
imizer	は最適化手法を指定
eduler	は学習率更新のスケジュール

状況表示

```

def graph(): # 省略

def print_metrics(epoch, X_true, Y_true):
    # 損失値、正解率を計算
    model.eval() # 検証モードに切り替え
    with torch.no_grad(): # 「微分計算しない」指定
        outputs = model(X_true) # 予測値(確率)を出力
        loss = criterion(outputs, Y_true) # 損失値計算
        pred = torch.argmax(outputs, dim=1) # 最大確率
        acc = (pred == Y_true).float().mean() # 正解率
    now = time.perf_counter()

    # グラフ表示用に値を保存
    loss_list.append(loss.to("cpu")) # .to("cpu")はCPU
    acc_list.append(acc.to("cpu"))
    time_list.append(now - start)

    # 現在値表示
    print(f'Epoch:{epoch+1:3d}, Loss: {loss:.8f}, Acc: {acc:.8f}, '
          f'Time: {(now-start)/60:.5f}[min]')
    return

```

loss, acc, now の計算を print_metrics にまとめてます

PyTorchでの実装#3

学習実施

```
# 計測値を準備
loss_list, acc_list, time_list = [], [], []
start = time.perf_counter() #計測開始
print_metrics(-1, X_test, Y_test) # 学習前の状況表示

# 学習ループ
for epoch in range(100):
    model.train() # トレーニングモードに切り替え
    for xb, yb in train_loader: # ミニバッチ学習を実行
        optimizer.zero_grad() # 微分値を0にリセット
        xb, yb = xb.to(device), yb.to(device) # GPU対応
        outputs = model(xb) # 予測値 (logits) を出力
        loss = criterion(outputs, yb) # 損失関数
        loss.backward() # 誤差逆伝播法
        optimizer.step() # 勾配降下法 (パラメータ更新)

    print_metrics(epoch, X_test, Y_test) # 検証
    scheduler.step() # 学習率更新

graph(loss_list, acc_list, time_list, title="PyTorch(CPU)")
```

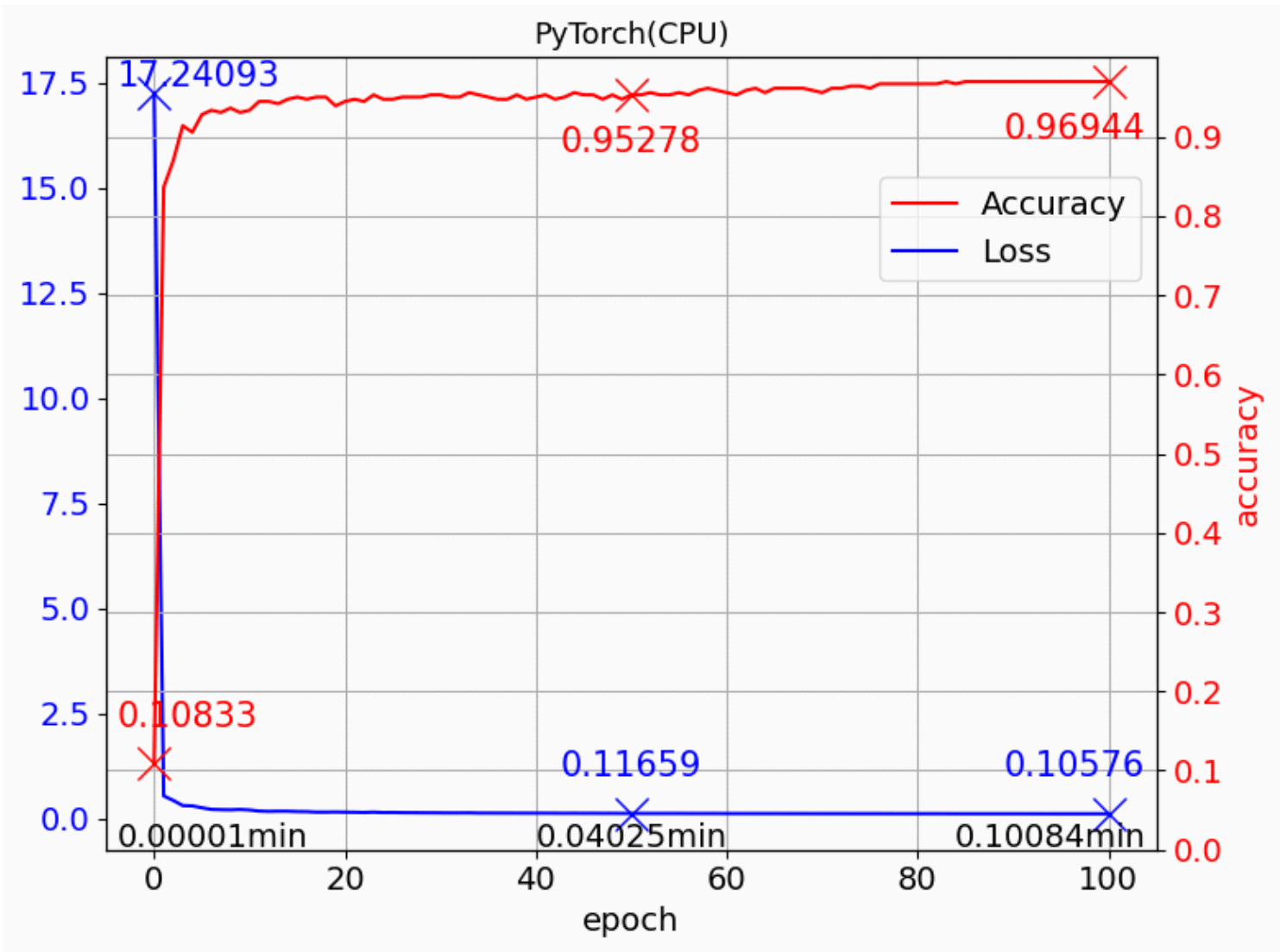
コメント

全体の流れはこれまでと同じです

- ミニバッチ学習用のデータを取得する
 - `for xb, yb in train_loader:`
- `forward` して予測値を出力する
 - `outputs = model(xb)`
- 損失関数を計算する
 - `loss = criterion(outputs, yb)`
- 勾配(微分値)を求める
 - `loss.backward()`
- 勾配降下法を実施
 - $\mathbf{W}^{(t+1)} := \mathbf{W}^{(t)} - \eta \nabla L(\mathbf{W})$
 - `optimizer.step()`

PyTorch(実行結果)

グラフとログ



Epoch:	0	Loss:	17.24092674	Accuracy:	0.10833	Time:	0.00001[min]
Epoch:	1	Loss:	0.53120923	Accuracy:	0.83611	Time:	0.00078[min]
Epoch:	2	Loss:	0.42357185	Accuracy:	0.86944	Time:	0.00153[min]
Epoch:	3	Loss:	0.30052987	Accuracy:	0.91389	Time:	0.00230[min]
<省略>							
Epoch:	98	Loss:	0.10609009	Accuracy:	0.96944	Time:	0.09883[min]
Epoch:	99	Loss:	0.10593439	Accuracy:	0.96944	Time:	0.09980[min]
Epoch:	100	Loss:	0.10575505	Accuracy:	0.96944	Time:	0.10084[min]

結果

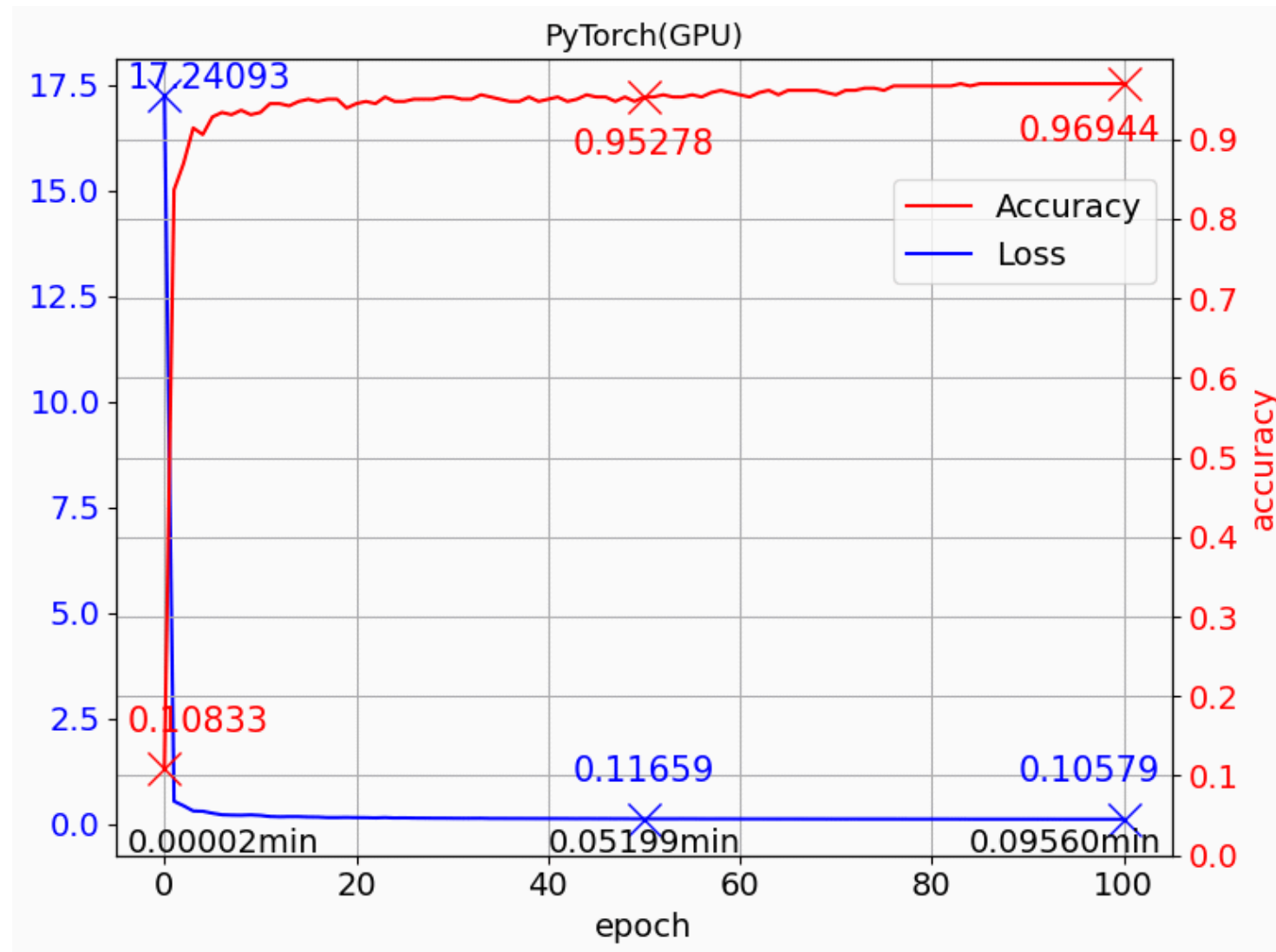
- 損失値 (0.110→0.105)
 - ほぼ同じ
- 正解率 (98.0%→96.9%)
 - ちょい低下
- 時間(100エポック:6.6秒→6.0秒)
 - 誤差レベル

評価

- 同じ手法を使っているため、結果はほぼ同じ (乱数シードで変化する程度の違い)
- 別ライブラリなのであくまで参考 89/101

PyTorch(GPU、実行結果)

グラフとコード



コード

下記のみ変更

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")  
#device = "cpu"
```

結果

- 損失値
(0.105→0.105)
 - ほぼ同じ
- 正解率
(96.9%→96.9%)
 - 同じ
- 時間(100エポック:6.0秒→5.7秒)
 - 誤差レベル

評価

- 手法は同じなので損失値と正解率は変わらず
- GPUは有効になっているが、モデルが小さい為処理速度にはほぼ差
90/101

§ 12. 閑話

本筋からは外れますが関連するトピックをいくつか取り上げます

概念&用語の解像度を高める#1

特徴量を指定する必要がない

ニューラルネットワークの特徴の1つとして「**特徴量を指定する必要がない**」ということがよく言われます
今回のシナリオに当てはめると次の通りです

- 「正解率98%まで持ってこれた」
- 「でも画像解析的な解説は一言もなかった」

パラメータ数

AIでよく1兆パラメータなど言われますがその**パラメータとは「学習によって更新される変数の個数」**です
今回のシナリオで当てはめると W と b です

ハイパーパラメータ

人手での設定が必要な変数で、学習はされません。今回の例ではバッチサイズや学習率、エポック数、ノード数、層数などです

GPUの重要性

ここまで見てきたとおり、パラメータの更新は微分計算によって行われます。また、パラメータは行列やベクトルで表されており、それらの演算も含まれます

微分や行列計算を高速で実行できるのがGPUです

GPUのメモリ

GPUで計算するためにはパラメータをGPU内のメモリに格納する必要があります

1パラメータあたり64bit=8bytesとすると
1億パラメータ/1GBとなり、仮に1兆パラメータのAIモデルだと1TBのメモリが必要になります
(必然的に複数GPUの並列化が必須です)

ローカルAIを設置しようとする、20GBのGPUがあったとして使えるAIモデルのパラメータ数は20億程度にないです

概念&用語の解像度を高める#2

量子化

メモリの話の続きで、1パラメータ=64bitの場合は1億パラメータ/1GBでした。

ならば1パラメータ=16bitにすれば4億パラメータ/1GBになります。これが量子化です

より多くのパラメータが扱える代わりに1パラメータあたりの精度は低下します。極端な話、1bit量子化なども実験されています(実用化はどうだったか失念)

初期パラメータが乱数の理由

パラメータがらみで、初期パラメータを乱数にする理由ですが、仮に同じ値を設定したとすれば、どのパラメータに対しても同じ微分値となり更新後の値も同じとなり、**要は実質的な学習がなされません**。そのため初期値として乱数が使われます

TFLOPS

GPUの性能を表す指標として使われるTFLOPS (1秒あたりの浮動小数点演算の回数)ですが、つまり**1秒に何回微分や行列計算ができるか**のお話になります(正確には回数そのものでなく、その比例係数)

ただし、そもそもメモリに載らないと計算できないので重要性はメモリの方が大きいかなと思っています(個人の感想)

量子コンピュータ

ニューラルネットワークにおける**学習の本質は勾配降下法**でした。この手法は**原理的に回数を繰り返す**必要があります

この**繰り返しを高速に実行できるようになる、もしくは繰り返し自体を減らす／なくすことができる**、という期待が高いのが量子コンピュータです

勾配消失問題

勾配消失問題とは

勾配降下法の基本式は次の通りです

$$\backslash \mathbf{bm} W^{(t+1)} := \backslash \mathbf{bm} W^{(t)} - \eta \nabla L(\backslash \mathbf{bm} W^{(t)})$$

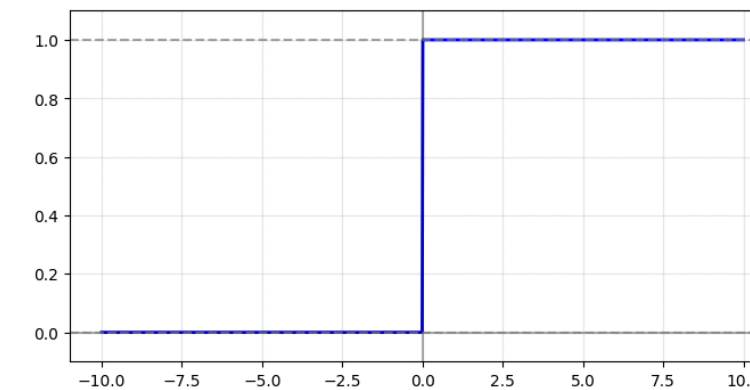
仮に $\nabla L(\backslash \mathbf{bm} W^{(t)}) = 0$ とすると $\backslash \mathbf{bm} W$ は更新されません。つまり学習されず、これを勾配消失問題と呼びます(実際には厳密に0でなくとも非常に小さな値に留まる場合も含まれます)

✎ 勾配消失問題が関わる技術要素

WEBで勾配消失問題の説明を眺めてたのですが誤差逆伝播法における問題という記述が散見されました

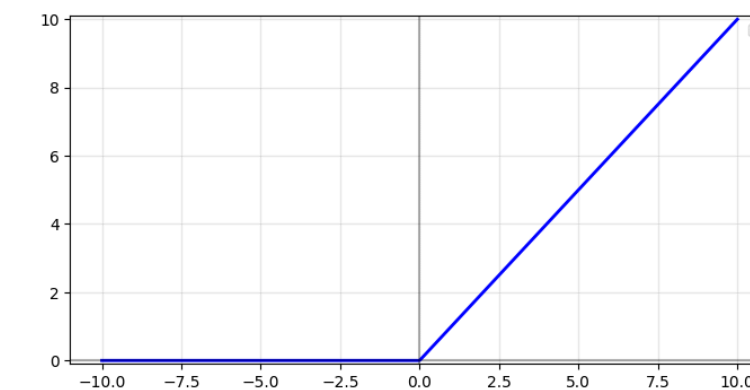
でも本来は「勾配降下法において」が順当な説明でしてその辺の区別を踏まえた上で参考にするのが良いかと

ステップ関数



これはあらゆる x に対して微分値が0なので一切学習されません

ReLU関数



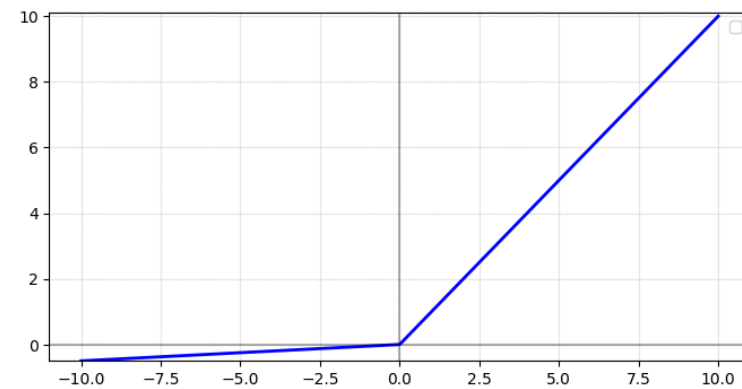
$$f' = \begin{cases} 0 & , x \leq 0 \\ 1 & , x > 0 \end{cases}$$

x がマイナスになると学習しなくなります(死んだReLU問題などと呼ばれます)

その他の活性化関数

LeakyReLU関数

「死んだReLU問題」を回避する目的の関数です



数式は以下。グラフは左図です

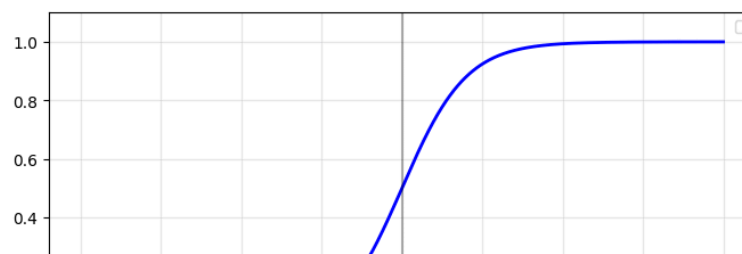
$$y = \begin{cases} \alpha x & , x \leq 0, \alpha > 0 \\ x & , x > 0 \end{cases}$$

✎ $\alpha > 0$ の理由

$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial x}$, $\frac{\partial L}{\partial y}$ で勾配方向を握ってるのは $\frac{\partial L}{\partial y}$ だから

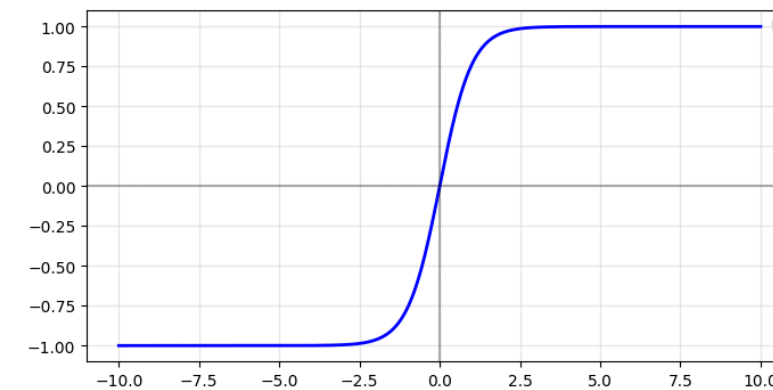
Sigmoid関数

知名度が高いです。最近の利用頻度は低いようです。



数式は以下。グラフは左図です

Tanh関数



数式は以下。
グラフは左図です

$$y = \tanh x = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Tanhも x が 0 から遠くなると勾配消失が生じます

しかし、0 付近での勾配はSigmoidよりも大きく学習効率が高いです(0 付近できゅっと締まってる)

あと原点对称です

✎ シグモイド型関数(シグモイド的／風／系関数)

稀にですがシグモイド「型」関数という表記もあります

この場合は特定の関数ではなく、S字型の関数を指してます

ユニバーサル近似定理

ユニバーサル近似定理とは

二層ニューラルネットワークにおいて、隠れ層のノードの個数に制限がなければ**任意の連続関数を任意の精度で近似することが可能**という定理です

証明の骨子

およそ下記のような流れです

- まずノード1つでスタート。直線一本が表現できる
- ノードを1つ増やすと、折線を表現できる(折れ目1つ)
- 近似したい関数に合わせて折れ目を必要なだけ増やす
- これを繰り返せばどんな関数でもいくらでも近似できる

これを知ってどうなるの？

実務で直接的に役に立つことは稀かと思います

しかし、理論的な精度の上限が無いということは基本知識として押さえておくべきかな、という位置付けです

✎ 敢えて使い所を探るなら

学習対象がグラフ的にどの程度凹凸があるかを把握できるならば、その凹凸数に合わせたノードを用意するという程度の助けにはなるかと
(でもこれは「正解ではない」です。実験の初手程度)

ノードの追加 vs 層の追加

問題

今、2層ニューラルネットワークがあったとし、隠れ層のノード数を10個とします

追加でノードを10個増やす場合、2層のまま(ノードの追加)か3層にする(層の追加)かどちらが良いか？という問題です

回答(正解ではないです)

傾向として層を増やす方が表現能力は高くなります

- ノードを増やす($10 \rightarrow 20$) $\rightarrow$ 表現力は2倍(= 20)
- 層を増やす($2 \rightarrow 3$) $\rightarrow$ 表現力は2乗(= 100)
※数値はイメージです

ひたすら層を増やせば良いのか？

「層を増やす方が表現力が高い」というのは各層にある程度のノードが存在した上でのお話です

実際、ノード数1で層の数を任意の個数とした場合に任意の関数を近似できるという証明はないです(ユニバーサル近似定理の層数バージョンは無い)

これを知ってどうなるの？

ノード数は時間パフォーマンスに直接影響します追加できるノード数には制限があります。その上で

目の前の課題に対して、ノード数を一定数追加する場合にまずは層数を増やしてみるという選択肢が優先的ということは基本的な知識として知ってた方が良いかな、という感じです

ノーフリーランチ定理

ノーフリーランチ定理とは

ノーフリーランチ定理とは、「ある学習アルゴリズムを用いた場合にそれがあらゆる問題に対して常に良い性能を示すことは不可能」という定理です

今回の課題でのイメージ

- 今回のモデル→数字に対して98%の正解率
- しかし、アルファベットに対しては4%の正解率

では、このモデルでありとあらゆる文字を学習したとして(漢字、アラビア文字、絵文字、象形文字、楔形文字、くずし文字など)

そのモデルがある程度どの文字も識別できたとしても、恐らく漢字特化やアラビア文字特化のモデルの方が性能高いんじゃない?といったイメージです

✎ 理論的にはそもそもの問題の範囲が森羅万象、非実在も含めたありとあらゆる問題なのであらゆる文字の100%識別モデルは存在するかもです

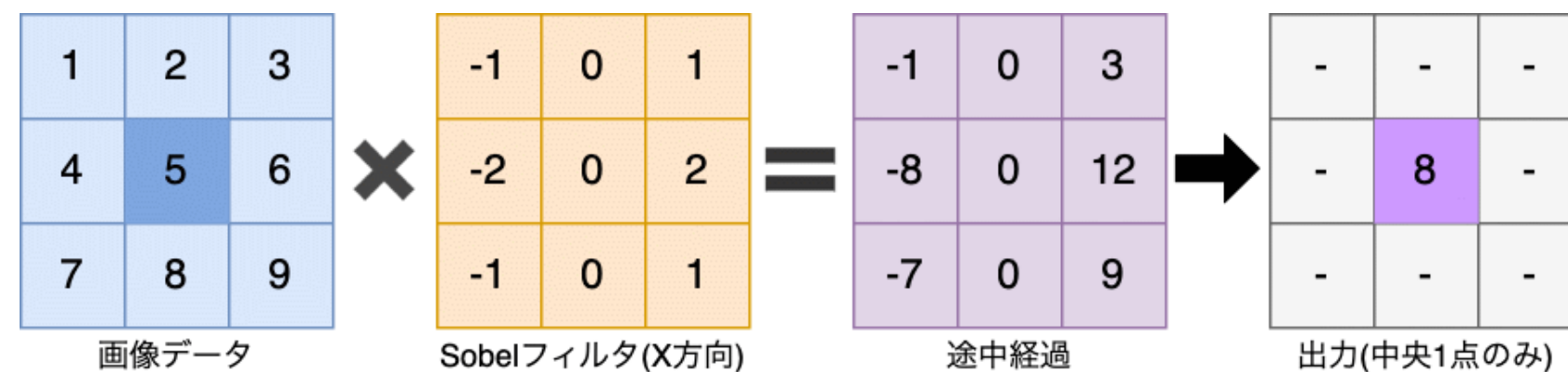
CNN(畳み込みNN)

CNNとは

NNに画像の特徴検出手法を取り入れたものです

畳み込みとは

元々は画像のフィルター処理の1種です。下記画像で説明します。左端の青色が画像データで、中央の5が処理対象のピクセルです。で、一つ右のオレンジがフィルターです。青とオレンジの対応する枠を掛け算した結果が紫の枠。紫枠の数字を全て足し算した結果が右端の8でこれがフィルター処理の結果です



このフィルター処理を画像の全ピクセルに対して行います

フィルター処理の結果

実際に先のフィルター処理を行った結果が下図です



特徴として輪郭が明確になっています。どのような特徴を抽出できるかはオレンジのフィルター内の各係数によります。先の例ではX軸方向の輪郭抽出です

CNNでの取り入れ方

CNNではフィルタという枠組み(3x3など)だけが用意されており、中の係数がパラメータとなります。対象画像に対してどのような係数のフィルタが最適なのかを自動で学習・設計する仕組みになっています

RNN(再帰型NN)

RNNとは

Recurrent Neural Networkの略で、時系列データや連続したデータの処理に特化したNNです

通常のNNとの違い

通常のNNでは学習用データは1件1件独立とみなしますが、RNNではあるデータとその次のデータに関連があるとみなして学習を行います

- 独立の例: 数字画像データ
→ 前の画像と次の画像に関連性はない(独立)
- 関連ありの例: 文章データ
→ 前の単語と次の単語に関連がある

しかし、ここまで紹介してきたNNの構成ではデータ間の関係性は加味できません。

その為そもそもの構成自体を変える必要があります

すみません

分量的 & 時間的に
作成が間に合わず
割愛します 🙇

アンケート:<https://forms.gle/7Kp9fuJK1kY1Jfk86>

おしまい

以上で終了です

今回でだいたい7年前くらいです。その7年前に何があったかということ
トランスフォーマーというアルゴリズムが提唱され色々ゲームチェンジしました
なので、次回はそれを取り上げます